

Loop-Work：一种面向处理器 RTL 工程的 仓库原生领域 Agent 构建方法

基于 ysyx-workbench 的 RV64 纵向案例研究

纵向工程证据范围：近四个月的持续开发记录

摘要：大语言模型已能生成寄存器传输级（Register Transfer Level, RTL）代码并调用仿真、综合和静态时序分析工具，但复杂处理器开发并非一次自然语言到 Verilog 的映射。体系结构语义、跨模块时序、软件工作负载、参考模型和物理约束必须在同一上下文中协同；仅有源码与 shell，难以稳定闭合长链任务。

本文提出 **Loop-Work**，一种面向处理器 RTL 功能开发与分层验证的仓库原生领域 Agent 构建方法。该方法以控制、知识、执行和证据四个责任平面承载工程任务，以任务内修订环连接分析、设计、实现与验证，再由跨任务能力环把可迁移经验固化为 instruction（声明式工程规则）、Skill（供 Agent 复用的操作规约）、端到端执行剖面（E2E，把规则、执行入口和终止条件连接成任务图）、测试与门禁。

本文以近四个月的 ysyx-workbench 纵向记录为案例，覆盖“一生一芯”课程项目从 C 阶段到 B 阶段的连续演进；C/B 是课程进度标签，不是本文定义的设计成熟度。工程对象从 C/汇编测试、Abstract Machine（AM，抽象计算机运行时与平台适配层）和 NEMU 参考机，推进到 RV32（32 位 RISC-V）五级流水核、ysyxSoC 接入及 RV64（64 位 RISC-V）双发射乱序核。NEMU 已支持整数乘除、原子、单/双精度浮点、压缩及位操作等扩展，覆盖机器/监管者/用户（M/S/U）特权级和启动所需设备模型，并在该参考平台内运行 Linux 6.6、进入 Ubuntu 22.04.5 root shell；AM 与 am-kernels 形成参考端和 RTL 端共用的应用、设备与体系结构测试；NPC（本项目的 RTL 处理器目标）则由基础整数主通路演进为支持上述扩展、控制状态寄存器、Sv39 虚拟内存、硬件页表遍历、指令/数据地址转换缓存和 16 项物理内存保护的小窗口双发射乱序核。同一设计身份下，113/113 项模块测试、177/177 项 official riscv-tests、61/61 项 AM 测试均通过；验证压力从 L0 模块事务、L1 全核与差分测试、L2 mini-system 推进到 L3 轻量 Linux。DiffTest（在指令退休边界比较 NEMU 参考状态与 NPC RTL 状态的差分测试）失配为 0。

上述产物来自同一闭环：Agent 生成并消费接口合同、RTL、testbench、负向变体和 AM 程序，NEMU 与 DiffTest 提供参考语义，系统回放扩大验证压力，Yosys、OpenSTA 与开源工艺库提供实现反馈。架构证据达到稳定条件后形成机器状态 ARCH_STABLE；与该状态一起发布的“收据”是记录实验身份、门禁结果和允许结论的机器可读凭证。 workflow 切片选择器（下文简称 Selector，不是 RTL 发射队列的就绪指令选择器）把因

果量测、PPA 资格和可回退 RTL 实验编入同一选择次序。owner-timing 在这里指 AXI 写事务从请求建立到 B 通道响应完成期间的责任者驻留量测；该切片在 CoreMark 与 Dhrystone 上各完成三次同身份重复。受控增加两个 B-response 周期的诊断实验识别出响应延迟敏感性，并只授权下一分析方向。该结果尚未授权生产 RTL 候选或工程晋级（promotion），5 ns 时序硬门仍失败，性能、功耗、面积三项联合资格（PPA）保持 UNQUALIFIED。本文报告一种工程方法在单仓库中的实例化及其可复核产物；现有实证范围止于处理器 RTL 功能开发、系统回放和预布局实现反馈，不把纵向共现解释为相对其他 Agent 方法的因果优势，也不把测试集合通过解释为覆盖率闭合，或以预布局代理结果替代物理签核。

关键词：处理器 RTL Agent；Loop-Work；仓库原生环境；分层验证；RV64；DiffTest；CPI/PPA

1 引言

1.1 问题背景与研究动机

在最初建立 RV32I 取指、译码、执行、访存和写回主通路，并把 AM 生成的裸机镜像装入处理器仿真时，Agent 已能较快补充译码、算术逻辑和局部控制代码。此时目标集中在基础整数指令和单发射五级流水，工程师可以直接指出源文件、运行命令和 GOOD TRAP 条件；即使一次生成遗漏 testbench、存储模型或退出协议，也能够较短的人工反馈路径内补齐。

任务随后由单核裸机运行扩展到 ysyxSoC 地址空间与 AXI 接口，再扩展到 RV64、浮点与原子指令、M/S/U 特权级、Sv39 地址转换、双发射乱序执行、OpenSBI/Linux，以及 CPI（每条退休指令的周期数）和 PPA 分析。此时一次 RTL 修改可能同时改变请求 owner、反压传播、flush 与精确异常顺序，并要求 NEMU 参考语义、AM 测试程序、NPC 提交状态和综合约束同步更新。代码即使能够编译，也可能因为遗漏架构前提、选择错误后端、使用过期日志或由过弱判定器给出绿色结果而无法进入后续开发。

这些连续任务揭示的瓶颈不再只是“生成速度不够”，而是领域知识怎样被送入当前上下文、自然语言目标怎样先被分析并转成受权任务、设计与测试怎样共同产生、NEMU/AM/NPC/EDA 的观测怎样互相校验，以及本轮反例怎样在下一轮不被遗忘。延长单个提示或增加一次人工审查能够缓解局部问题，却不能稳定保存仓库关系、工具入口和验收层级；会话或模型变化后，相同的工程事实仍可能被重新发现和重新踩坑。

本文据此把研究对象从“如何让模型生成一段 RTL”推进为“如何在处理器 RTL 工程中构建一个能够工作的领域 Agent”。本文提出 Loop-Work，把控制、知识、执

行和证据组织为四个责任平面：前两者使 Agent 在写代码前理解目标、边界和专业约束，后两者使其能够生成并消费 RTL、测试、反例与工具反馈。任务内闭环用于完成当前修改，跨任务闭环把稳定经验固化为 instruction、Skill、E2E、测试和门禁。候选是否可由下游消费由证据面判定，但它只是整套领域能力中的一环，而非方法的全部。

在 ysyx-workbench 中，NEMU、AM 和 NPC 不是三个并列项目：AM 产生可复用程序镜像，NEMU 提供指令级参考状态，NPC 是被开发的 RTL 目标，DiffTest 在退休边界比较二者；模块 testbench、L0-L3 系统验证以及 Yosys/OpenSTA/开源 PDK 又分别增加周期级、系统级和实现级反馈。本文将以这条真实数据流说明 Loop-Work 如何落地，并以 NEMU、AM 与 NPC 的纵向产物检验 Agent 工程能力扩展到了何种范围。

1.2 研究问题

全文围绕五个研究问题展开：

1. **RQ1（领域能力构成）**：怎样把分散在设计规范、源码、脚本、工具和历史记录中的处理器 RTL 知识，组织为 Agent 可发现、可调用并可维护的仓库原生环境？
2. **RQ2（目标到任务）**：怎样把工程师的自然语言目标转换为“先分析、后实施”的任务图，使 Agent 能够确定设计边界、选择规则和工具、分配责任并控制修改范围？
3. **RQ3（设计—验证闭环）**：Agent 怎样在 AM、NEMU、NPC 和 EDA 工具之间生成并消费架构方案、RTL、测试程序、反例和执行证据，并在设计增删后保持跨层一致性？
4. **RQ4（能力演进）**：编译失败、仿真反例、差分失配、系统缺口和 PPA 反馈怎样被固化为 instruction、Skill、E2E、测试和门禁，从而扩大后续任务可独立闭合的范围？
5. **RQ5（工程产物与边界）**：Loop-Work 在 ysyx-workbench 的 NEMU、AM 和 NPC 上形成了哪些工程产物，跨越了哪些验证压力层级，现有证据能够支持和不能支持哪些结论？

RQ 是 Research Question（研究问题）的编号。为避免后文只有编号而要求读者回翻，五问可用一条连续工程链理解：RQ1 先回答“领域 Agent 需要知道什么、这些知识放在哪里”；RQ2 回答“工程师的一句话怎样被拆成可授权、可停止的节点”；RQ3 回答“节点怎样共同产生 RTL、测试和反例，并消费 NEMU、AM、NPC 与 EDA 反馈”；RQ4 回答“本轮失败怎样变成下一轮可复用的规则”；RQ5 最后检查“这种方

法实际形成了什么产物，证据最多允许说到哪里”。例如，“优化 RV64 CPI”不会直接进入 RTL：它先由 RQ1 找到性能规则与既有反例，由 RQ2 形成基线、瓶颈普查、因果量测和候选复验节点，由 RQ3 运行 benchmark、只读探针与时序工具，由 RQ4 把无效量测和失败门禁固化，最后由 RQ5 报告可重复基线与仍未合格的 PPA 边界。前四问解释领域 Agent 怎样形成和演进，第五问检验这套方法在本案例中形成了什么。

1.3 主要贡献与研究范围

本文围绕上述问题给出五项贡献：

1. 提出一种面向处理器 RTL 功能开发的领域 Agent 构建观点：Agent 的有效能力由基础模型与仓库原生工程环境共同构成，而不能只用一次代码生成质量衡量。
2. 用接口化责任模型描述控制、知识、执行和证据四个平面，并给出任务内修订与跨任务能力固化两类反馈边，说明一次工程作业怎样运行以及经验怎样进入后续任务。
3. 给出从工程目标、写前分析、接口合同和 RTL 拓扑，到 testbench (TB)、负向变体、AM 程序、DiffTest、L0-L3 系统验证与 EDA 反馈的端到端任务流程，使设计与验证成为同一任务的共同产物。
4. 给出由 instruction、Skill、机器合同、E2E profile、受控执行和证据收据组成的仓库级参考实现；身份拒绝、负向资格、长跑失败关闭和有界晋级被编码为四项可审计工程准则。
5. 给出 RV32 single、ysyxSoC、NEMU/AM/NPC、RV64 双发射 OoO (Out-of-Order, 乱序执行)、分层 Linux 和 CPI/PPA 的纵向工程结果，并用机器可读主张、同身份回归和独立反例复核限定结论边界。

研究材料来自一个持续演化的仓库，本文据此讨论方法接口的可执行性、可审计性及其在所列 RV64 压力层级上的实例化。本文定位为工程方法与纵向案例研究，不提供四平面最小性或完备性定理，也不把纵向共现解释为生产率因果效应。全文依次讨论相关工作、问题与设计要求、方法与参考实现、案例研究设计、纵向工程证据、成果综合、研究不足与下一步攻克方向及结论。

2 相关工作与方法空缺

2.1 Agent、RTL 生成与 EDA (Electronic Design Automation, 电子设计自动化)

SWE-agent 说明 Agent-Computer Interface 会影响仓库级软件任务的执行方式 [1]; VerilogEval 用 testbench 通过率评价 Verilog 生成 [2]; ChatEDA 研究大模型对 EDA 工具链的规划与执行 [3]。MAGE 进一步用多 Agent 候选采样、调试和 Verilog 状态检查提高模块级生成结果 [13]。近期一项 CVDP (包含 RTL 生成、验证、调试与规格对齐任务的硬件设计问题集) 实验显示: 对多数受测模型, 直接套用基线工具 Agent 的 Pass@1 (每个问题只取一次采样时的通过率) 低于相应的无工具单次基线; 加入结构化系统提示后, 差距缩小, 部分配置持平或反超 [14]。这一结论只适用于该论文的模型、任务子集与测试驱动环境 (harness), 不说明 Agent 在一般 RTL 任务中更弱。NotSoTiny 进一步在结构更复杂的上下文模块补全任务上比较原生 testbench 与形式等价: 在 1114 个去重并验证的模块补全任务上, 语法有效输出有 95–97% 可通过原生 testbench, 而 11 个受测模型在形式等价下的完全正确率为 3.3–23.4%; 其中四个相对较高结果也仅为 19.0–23.4%[22]。其中时序设计采用有限深度的 `equiv_induct` ($k = 30$), 因而这一范围只描述其数据集、模型和验证协议, 却直接提醒读者: 正向 testbench 接受率不能替代独立的语义资格。

SWE-agent 的 Agent-Computer Interface (ACI, Agent 与计算机工具之间的交互接口) 主要约束模型怎样浏览仓库、编辑文件和调用工具; Loop-Work 不把四平面声称 ACI 的替代或普适泛化, 而把 ACI 类执行接口放入处理器 RTL 的领域合同中: 写入权限必须同时受架构规格、target/reference 身份、周期级观察点和分层验收边界约束。两者的直接差异因而不是“能否使用 shell”, 而是执行动作能否与硬件事务语义及证据身份共同进入裁决。本文没有在同任务、同预算条件下比较二者, 故这里只给出职责接口对照, 不报告相对效果。

ACE-RTL 以 generator、reflector 和 coordinator 维护迭代调试上下文。在 CVDP 的 cid003 规格到 RTL 任务上, 完整 ACE-RTL 的 Agentic Pass Rate (APR) 为 96.15%, 同类最强 Agent 基线 MAGE (ACE-RTL-Generator) 为 55.13%, 即提高 41.02 个百分点 [19]。该实验对每个问题进行五次独立运行; APR 表示多次运行中至少被一次解决的唯一问题数占问题总数的比例, 因而不能解释为单次生成成功率或跨任务平均相对提升。它与 Loop-Work 都把调试上下文置于单次生成之上; ACE-RTL 的 coordinator 聚合跨迭代错误、修复建议及其结果, 并在搜索停滞时触发 restart, 而 Loop-Work 控制面还把任务权限、实验身份、失败关闭和跨层晋级编码为必须检查的接口。该比较限于公开方法描述, 不推断 ACE-RTL 未披露的内部实现。NVIDIA 随后的官方技术发布又报告 Nemotron 3 Ultra 配合该 Agent 在九类 CVDP 任务上的平均 pass rate 为

97.1% [20]。本文以前一项预印本结果作为学术论证锚点；后一个数值来自未经同行评议的厂商技术博客，是平均通过率而非 Pass@1，只作补充背景，也不能延伸为系统级签核率。*Can AI Agents Really Complete RTL-to-GDS?* 中的 FluxEDA 个案研究在统一 RTL、工具环境和商用 55 nm 工艺条件下比较三类 Agent 执行架构，报告同一基础模型的 Agent 系统在固定 DeepSeek-V4 与 350 MHz 条件下的 0–100 端到端得分为 11.67 至 97.94，即相差 86.27 个绝对分值点；这不是 86.27% 的相对提升。其当前 v3 在 PicoRV32、55 nm、350 MHz、各配置仅运行一次的实验中，还给出两项端到端得分同为 13.33、但归一化 Token ROI（该论文定义的 Token 投入产出比）为 12.71 与 0.09 的 Agent 配置，二者相差约 141.2 倍 [21]；这不是纯 Token 数之比，也不能归因于单一模型或单一组件。Design Conductor 则报告从 219 词需求出发，在 12 小时内为 RV32I+Zmmul VerCore 形成满足 ASAP7 1.48 GHz 时序的 GDSII（集成电路版图交换数据）[25]。这些结果表明，相关系统已在各自任务与评价协议下报告了迭代反馈和端到端自治能力的进展；由于设计、工具、工艺与 oracle 均不同，它们不构成本文 RV64 OoO、Ubuntu 长跑或 PPA 资格的横向基线。

2.2 仓库级修复与 Agentic 验证

CVDP 将 RTL 生成、验证、调试和规格对齐组织为 783 个问题及 13 类任务 [15]；HWE-Bench 把真实历史硬件缺陷转化为六个开源项目中的 417 个仓库级修复任务，并指出定位、硬件语义与 RTL/配置/验证跨产物协调是主要失败环节 [16]。PRO-V-R1 使用程序化工具反馈训练自主 RTL 验证 Agent [17]；GoGoTB 则把执行控制、可演化知识和规格绑定覆盖闭合连接起来 [18]。这些工作已经覆盖角色分工、反馈循环、验证环境生成和仓库任务，因此本文不以“首次使用 Agent 闭环”为贡献。本文关注的是另一条轴：同一 repository-native 生命周期中，怎样把任务权限、适用知识、真实执行和证据资格保持为可区分状态，并让中断、旧身份、弱 oracle 和不合格 PPA 进入不同终态。

LLM4Cov 把确定性执行反馈用于离线 Agent 学习，并在 CVDP-ECov 的 Agentic evaluation 上报告：其 Stage-2 4B 模型的覆盖阈值 Pass@1 为 69.2%，平均覆盖率为 90.4% [26]。这里的“通过”表示测试平台完成执行且覆盖率超过专家预设阈值；仿真失败样本的平均覆盖率按零计。论文除另有说明外以五个样本估计 Pass@1，并允许模型依据仿真反馈迭代最多三轮。因此，69.2% 不是测试平台可运行率，也不是 RTL 功能正确率或完整验证闭合率。Si2 的 LBC-bench 则以 CVDP 为初始重点，公开 leaderboard 数据格式、提交标签和可复现入口 [28]。前者研究怎样提高覆盖闭合候选，后者提供公共比较基础设施；二者都可为后续对照实验提供任务或评价协议，但公开 leaderboard 本身不能充当隐藏 oracle，也不承担本文主张的独立真值责任。

2.3 Mutation 与硬件故障注入

Mantra 从真实硬件缺陷构造 mutation 算子，并以 mutation score 评价测试敏感度 [4]；基于 HDL 的故障注入研究也表明覆盖率受到故障模型、工作负载和注入技术影响 [5]。Firefly 在 Maveric Core 2 与 Bitty 两个对象上报告：组合 line/toggle coverage 分别达到 95% 与 90% 时，其注入 fault set 中仍分别有 28% 与 67% 未被检出 [23]。该结果不能外推到所有覆盖率或所有故障模型，但说明常规覆盖率与 fault sensitivity 不是同一概念。BugGen 使用多 Agent 生成、插入和验证 RTL 缺陷，在五个 OpenTitan IP 上最终缓存了 500 个唯一且可检测的 bug scenario；实验为每个 scenario 配置两个 mutation，因此共包含 1000 个 mutation [24]。其 94.2% functional accuracy 表示最多两次重试后的 scenario 生成成功率，并行执行吞吐为每小时 17.719 个唯一且可检测的 scenario。初次 mutation 结果中有 104 个能够通过语法检查、但未被既有 OpenTitan 回归检测。该 104 的统计母体是 1000 个 mutation，而不是 500 个最终 scenario；它们反映测试平台盲区，不是 104 个新发现的 OpenTitan 真实缺陷。

Firefly 与 BugGen 主要解决上下文相关 fault 的生成、验证基础设施诊断或数据集扩展；本文后述的负向资格化命题关注这些 fault 如何成为不可绕过的晋级资格。为此，“变体可编译、语义变化非空、错误被触发、错误传播到观察点、判定器拒绝、campaign 完整”必须同时成立。本文的 23 个定向 mutant 与上述论文的数据集没有共同分母；检测给定变体仍然不能证明 DUT（Design Under Test，被测设计）全局正确或 oracle 完备。

2.4 追踪、可信论证与构建来源

需求追踪研究关注需求来源和生命周期链接 [6]。W3C PROV-DM（Provenance Data Model，来源数据模型）以 entity、activity、agent 及其派生关系表达来源与责任 [29]；GSN（Goal Structuring Notation，目标结构化表示）和 SACM（Structured Assurance Case Metamodel，结构化可信论证元模型）则组织 goal/claim、argument、context 与 evidence 的可信论证 [30, 7]。in-toto 以 layout 和 link metadata 记录供应链步骤、授权 functionary，以及每一步的 materials、products 和 byproducts [8]。SLSA（Supply-chain Levels for Software Artifacts）v1.2 Build Provenance 则在 in-toto Statement 中以 subject 标识构建输出，并在 buildDefinition 中以 resolvedDependencies 描述已解析输入，在 runDetails 中记录 builder、执行 metadata 和 byproducts [9]。二者均可作为来源和完整性追踪提供结构，但材料来源真实并不自动证明 RTL 功能正确，论证结构完整也不等于仓库中的验证作业已经成功执行。本文不发明 provenance 或 assurance-case 语法，而把它们与 task contract、工程 oracle、失败关闭状态和晋级责任连接；这里的 oracle 是把日志、trace、计数器或波形转换为机器判定的参考模型、断言或规则，它本身也必须被资格化。

2.5 质量门、UVM 与工业准备度

持续交付中的 `quality gate` 研究讨论发布可信度与速度的权衡 [10]；UVM (Universal Verification Methodology, 通用验证方法学) 标准化验证组件和接口 [11]；ISO 26262-5 规定汽车硬件安全生命周期和工作产品 [12]。Siemens 于 2026 年宣布扩展与 NVIDIA 的合作：Fuse EDA AI Agent 已接入 Intelligence Center X，而将 NeMo Gym、OpenShell、Nemotron、Switchyard 和 CUDA-X 与基于物理的确定性 EDA 引擎持续校验相结合的扩展能力，官方说明为将在后续 AI-native EDA 版本中提供 [27]。同一材料中，STMicroelectronics 仅表示计划在其在研设计活动中测试和验证相关优势。因此，该材料能够说明产业产品路线正在转向长运行、受治理且持续校验的 Agent workflow，但不能作为相关组合已经完成独立验证或普遍部署的证据。

本文不声称符合 UVM、SLSA 或 ISO 26262，也不替代物理 signoff。本文将上述已有机制组合并约束到 repository-native 场景：把本地 `module/signal/transaction`、验证层级、实验身份、负向判定器能力和失败关闭发布编译成同一个可执行 workflow。

UVM 的 `sequencer`、`driver`、`monitor`、`scoreboard`、`objection` 和 `functional coverage` 已经提供成熟的验证组件与仿真生命周期语义；Loop-Work 所处层级更外侧，负责把一次 RTL 任务的规格、写权限、UVM 或非 UVM 执行入口、原始报告与晋级责任绑定到同一仓库身份。当前案例主要使用定向 SystemVerilog TB、断言、AM workload 与 DiffTest，尚未建立 UVM verification plan 或 coverage closure，因而不能把“113/113 测试通过”等同于功能覆盖率闭合。

形式验证与物理签核同样承担本文工具链不能替代的责任。Symbolic QED 等工作说明处理器不变量可以由符号方法和性质证明检验 [31, 32]；等价性检查、模型检查与断言证明回答的也不是 mutation 杀伤率。物理实现则还需要布局、时钟树、布线、寄生提取、多模式多角时序、功耗及 DRC/LVS 等对象。本文只把这些方法所需的输入身份、执行结果和失败状态预留在责任接口中；尚未接入的验证与签核环节统一列入第 11 节，而不在案例结果中视为已经完成。

表 1: 相关方向与本文方法的接口

方向	主要回答	本文补充的问题
Agent/RTL benchmark	候选是否通过给定任务或 testbench	将模块和仓库任务结果绑定到限定声明
Repository/verification Agent	如何修复真实缺陷、生成验证环境或闭合覆盖	长跑、身份和发布状态怎样失败关闭
Self-verifying EDA stack	物理引擎反馈、治理运行时和模型怎样进入自治执行	身份、新鲜度、负向敏感性、失败关闭与成熟度如何组成不可绕过的资格序列
Public leaderboard	模型与 Agent 提交怎样透明比较和复现	公开分数与隐藏 oracle、下游消费资格怎样分离
Mutation/fault injection	给定 fault set 下测试能否观察错误	变体非空性、传播、拒绝和 campaign 终态
追踪、来源与可信论证	claim、活动、来源、论据和证据如何关联	来源与结构之外，证据是否新鲜且足以支持声明
Quality gate/UVM	验证环境复用与发布控制	Agent 候选如何按本地工程合同失败关闭
Signoff readiness	何种物理证据可以交付	代理统计为何不能越级为合格 PPA

文献检索截至 2026 年 8 月 5 日；正式会议论文、camera-ready 作者稿、arXiv 预印本和产业技术发布在参考文献中分别标明。不同来源共同用于确定研究问题的位置，本文的方法命题仍由后续仓库证据和专门实验承担检验责任。

综上，近期工作已经分别推进了 RTL 生成、验证 Agent、EDA 工具调用、来源追踪和阶段门禁。本文选择一种把仓库知识、工程工具、分层验证和跨任务学习统一组织为处理器 RTL 领域 Agent 的工程方法。其补充不在于再增加一个 RTL 生成器，而在于说明模型如何依托仓库原生环境获得可持续的工程能力；身份、新鲜度、负向敏感性、失败关闭和有界晋级属于其中的证据与控制约束。当前论文检验的是这一组织方式在单个 RV64 仓库中的可实例化性，而非相对 SWE-agent、UVM 或其他 Agent 方法的平均优势。下一章从处理器任务的实际失效模式推导本文采用的责任接口。

3 处理器 RTL 领域 Agent 的构建问题与设计要求

相关工作表明，候选生成、工具反馈、来源追踪和阶段门禁已经分别取得进展，但它们尚未统一回答：一个通用模型怎样在真实处理器 RTL 仓库中获得分析架构、受控修改、构造验证、调用工具和吸收反馈的领域能力。本文先把这一缺口写成领域

Agent 的构建问题，再由失效模式推导方法所需的责任接口；候选验收是其中的证据责任，而不是问题定义本身。

3.1 处理器 Agent 任务的失效结构

处理器开发任务不能只写成“修改某个模块”。本文把任务定义为

$$\mathcal{T} = \langle c, O, K, U, Q, E \rangle,$$

其中 c 是待证明的工程主张， O 是 RTL、软件、配置与 workload 构成的对象集合， K 是必须遵守的领域知识， U 是授权动作， Q 是分层验收谓词与门禁， E 是必须保留的证据。开放式 Agent 往往只接收 c 的自然语言近似和部分 O ，因而在对象、动作和完成条件上留下歧义。任务还必须绑定实现责任与独立复核责任；具体角色接口在控制面小节统一定义。表 2 从实际工程压力推导出方法要求。

表 2: 处理器工程失效模式与方法要求

工程困难	开放式 Agent 的典型失效	方法要求	Loop-Work 机制
跨模块事务耦合	局部修改通过，却破坏 owner、flush、异常序或总线生命周期	最小充分且可扩展的任务边界	合同编译、专业不变量与范围扩展协议
target/reference 多配置	把另一设计、另一配置或旧日志当作当前结果	比较前绑定完整实验身份	design、build、workload、oracle 与 evidence 分层身份
小时级系统运行	进程退出、前台超时或信号中断被写成成功	成功必须由证据末端授权	失败关闭证据提交协议
CPI/PPA 多目标优化	用局部 CPI、generic cell 或理想时钟结果交换功能正确性	正确性硬门先于性能比较	分层门禁、单主假设的中等粒度切片和成熟度晋级
多会话历史增长	无关历史淹没当前问题，过期结论被重复消费	最小充分、带来源的历史选择	retained knowledge、evidence index 与 bounded recall
多 Agent 协作	节点越界写入、上下文污染或实现者自证完成	明确 owner、作用域与独立审查	机器合同、隔离派发与 reviewer/inspector

这些失效并不是六个彼此独立的脚本问题，而是“Agent 能力如何被构成”的四类责任缺口：目标和权限没有被控制；规则虽存在却没有被选择到当前任务；模型不能在确定环境中把方案转化为真实工程动作；工具虽产生观测，却没有形成可反驳、可复用的验证结论。若只增加提示长度或工具数量，知识、动作和验证仍可能彼此脱

节；若再把四类状态压成一个 GREEN，身份漂移、弱判定器和不完整长跑便成为不可表达状态。

3.2 由失效模式推导的设计原则

上述四类责任缺口导出六项设计原则：

1. 合同先于生成：先定义对象、输入、输出和成功条件，再允许 Agent 修改或运行。
2. 身份先于比较：baseline 与 candidate 身份不一致时，不计算正式改善。
3. 原始观测不可改写：新 checker 可以生成新解释，但不能覆盖原始 FAIL/PASS。
4. 正向与负向证据并存：正向 PASS 还要接受能够暴露目标错误的反例检查。
5. 局部接受与工程晋级分离：focused、module、system、PPA 分别承担不同结论。
6. 高成本验证受决策约束：长跑必须能够改变明确决策，并在开始前完成低成本排除。

这些原则共同要求一种带类型的闭环：控制责任把目标变成可执行任务，知识责任提供适用规则，执行责任同时产生设计、测试和原始观测，证据责任用正反例决定观测能够支持什么结论。增加模型参数或工具数量只能扩大潜在动作集合；在本文采用的分解中，四类责任需要能够分别检查并共同运行，潜在动作才可能转化为受约束的工程能力。该操作性分解服务于案例建模，不预设它是所有 Agent 系统的唯一结构。

3.3 术语的操作性定义

为使上述责任可以被复核，表 3 给出本文自定义术语的可执行含义。

表 3: 本文术语及其操作性定义

术语	定义
完成主张 (claim)	对功能、系统行为、性能或成熟度的陈述，例如“该 design 的 FENCE 顺序合同闭合”。
目的性主张集	按研究问题、验证压力和正负状态主动选择的可反驳主张集合；它不是随机样本，也不是“已完成事项”清单。
task episode / 任务事件	具有一个目标、待判断对象、可恢复验收条件和明确终态的一次连续工程事件；可跨多个目录。
证据边界	某组设计、配置、输入、命令、oracle 和原始产物最多能够支持的结论。

术语	定义
oracle / 判定器	把日志、trace、计数器或波形转换为 PASS/FAIL/GAP 的规则；它本身也需要资格化。
Loop-Work	从冻结合同到真实观测、差异分类、下一动作和可恢复终态的一次闭环工作轮。
task-run	一次确定性工程事件的结果档案；保存状态、指针、哈希和有界日志。
bounded recall	只按任务 profile、规范和关键词加载必要历史，避免过期结论淹没当前问题。
design-id	production RTL 规范文件集合的内容哈希，用于区分设计身份。
baseline / candidate	baseline 是同身份比较的冻结基线，candidate 是待检验候选；身份不同的两者不能计算正式改善。
mutation	能完成编译或 elaboration 的负向 RTL 版本，用于检查 testbench/oracle 是否拒绝指定错误。
owner	某个流水事务或状态在当前周期的唯一责任主体，用于阻止旧槽位或错误生产者继续授权结果。
qualification / 资格化	检查对象身份、原始观测完整性、判定器敏感性和父级硬门，从而限定一份结果能够支持哪类结论；资格化通过不等于证明全部未建模行为正确。
fail-closed	身份未知、证据缺失、清理失败、信号中断或 oracle 不合格时保持非 PASS。
promotion / 晋级	候选从局部开发证据进入更高成熟状态；它需要满足父级门禁。
CPI / IPC	每条退休指令的周期数 / 每周期退休指令数；必须绑定测量窗口和退休语义。
PPA	性能、功耗、面积（Performance, Power, Area）的联合工程指标；其资格取决于实现条件。
STA / 静态时序分析	Static Timing Analysis；在给定网表、时钟、I/O、库与工艺条件下检查路径时序，不运行动态指令流。
ARCH_STABLE / PERF_BASELINE	前者表示架构功能证据达到冻结条件；后者表示同身份性能量测可重复。两者都不等于物理 PPA 签核。
L0-L3 验证层级	L0 为模块事务，L1 为全核/AM/DiffTest，L2 为 OpenSBI mini-system，L3 为轻量 Linux；层级通过只支持各自范围。

术语	定义
DiffTest / 差分测试	在指令退休边界比较 NEMU 参考状态与 NPC RTL 状态，检查 PC、寄存器、CSR（控制状态寄存器）和特权状态等架构可见量。
instruction / 声明式规则	记录某类工程任务必须满足的不变量和硬门；例如 <code>rtl-agent-task-contract.instructions.md</code> 规定 RV64 子任务派发前必须声明目标、路径、命令、材料和成功条件。
Skill / 程序化任务能力	供 Agent 遵循的可复用操作规约与能力入口；Skill 规定步骤、参数、反例和失败出口，配套生成器才把字段写成机器合同。
E2E / 端到端闭包	End-to-End 的缩写；把规则、Skill、执行入口、观测点和终止条件连接为可运行任务图。
profile / 执行剖面	一类任务的节点、依赖、实现函数、责任主体、输入和输出清单。
raw observation / 原始观测	工具未经结论化处理的日志、trace、波形、返回码和运行状态。
manifest / 清单	对设计、配置、命令、产物路径、尺寸和哈希的结构化登记。
receipt / 收据	对已资格化证据及其允许结论的机器可读记录。
replay / 冻结重放	在设计、输入和原始观测不变时，用版本化判定器重新解释同一证据。
gate / 门禁	由固定验收谓词和执行入口组成的晋级检查。
hard gate / 硬门	不允许被其它收益抵消的门禁，例如功能回归或时序失败时，较低 CPI 不能换取晋级。
canonical / 规范唯一入口	被指定为能够产生正式证据的唯一文件、命令或对象；例如新任务合同必须经 <code>create</code> $\rightarrow$ <code>validate</code> $\rightarrow$ <code>render</code> ，不能用手改提示替代。
candidate-only / 仅候选结果	对象尚可用于诊断，但因合同未校验、边界被手改或审查未完成而不得被下游当作正式 PASS。
审查出口字段	<code>unknowns</code> 记录未知项， <code>scope_extension_request</code> 申请新增范围， <code>confidence_and_basis</code> 记录置信程度及其证据依据；三者都不能自动扩大当前权限。

术语	定义
GAP / UNQUALIFIED	GAP 表示必要义务尚未闭合；UNQUALIFIED 表示对象或量测缺少形成该类结论的资格。二者都不是 PASS。
canonical cohort / 规范证据组	绑定同一设计、构建、workload、判定器和证据身份的一组可比较运行。
Selector / 切片选择器	控制面从开放义务中选择下一项可证伪、可回退动作的机制；不是 RTL issue selector。
marker / 终态标记	日志中必须唯一出现的机器可检索事件，例如自然关机或指定违约被定向拒绝。
DUT	Design Under Test，被验证的设计；本文通常指 NPC 中当前冻结的处理器 RTL。
ROI	Region of Interest，性能计数有效的程序区间，由起止 PC 或 marker 冻结。
PDK	Process Design Kit，工艺设计套件；提供工艺相关库与实现条件，但“使用开源 PDK”本身不等于完成物理签核。

本文中的“合同”（contract）不是法律文件，也不总是同一种 JSON。它泛指一组可以被审查和拒绝的工程承诺；承诺的对象、载体和裁决者随层级不同。表 4 固定全文四种主要用法，后文出现“接口合同”“机器合同”或“量测合同”时均按此解释。

3.4 声明与分层实验身份

只使用 design-id 会遗漏 testbench、工具、workload 和判定器的变化。因此一次强结论至少由表 5 中的身份共同限定。

checker-only 修改若没有改变 DUT 输入、时序、终止或随机性，并且原始输入与日志完整，可以对冻结证据做 versioned replay（版本化重放）；production RTL、设备模型、仿真语义或 workload 发生变化时，旧执行不再属于新对象。该规则既防止串证，也允许十九小时级原始观测在身份不变时接受新的判定器解释。

当失效模式被归结为目标失真、知识失配、执行失控和验证误认后，所需方法便不再是增加一次生成，而是为这四类责任建立彼此可检查又能闭环协作的接口；它们构成 Loop-Work 领域 Agent 的顶层结构。

表 4: 全文合同术语的对象、载体与实例

合同类型	约束对象与典型载体	工程实例	与其它合同的区别
架构/接口合同	RTL 应实现的事务、周期、owner、优先级和可见行为；通常写在架构或模块.md	FENCE.I 要求旧取指包失效、前端 flush 后从规定 PC 重取	规定“硬件必须做什么”，不授权 Agent 读写路径
节点/机器任务合同	某次 Agent 节点可读取、写入、运行和交付什么；schema-v2 JSON 是授权真源	只读 DI-3 复核的 scope.write_paths=[], 且只允许 rg/sed/sha256sum	规定“本次作业可以做什么”，不证明 RTL 正确
量测合同	workload、ROI、计数器、重复次数、守恒式和非干扰要求；由 JSON、runner 与 checker 共同承载	CoreMark/Dhrystone 各三次，冻结起止窗口并核对 stats-on/off 的 cycles/retired	规定“数字怎样才可比较”，不直接给出优化根因
证据/验收合同	原始观测成为 PASS、FAIL、GAP 或收据所需的身份、marker、返回码、signal 与 cleanup 谓词	长跑只有命令、终态 marker、哈希和清理全部闭合才可发布 PASS	规定“观测最多能支持什么结论”，不修改原始日志

表 5: 分层实验身份

身份	绑定对象
design-id	production RTL 规范文件集合及参数化设计点
verification-id	testbench、assertion、reference、oracle 与 mutation 规则
build-id	filelist、define、工具版本、编译、elaboration 与链接配置
workload-id	镜像、binary、输入、ROI、seed、设备与延迟配置
experiment-id	上述身份与执行语义的组合身份
evidence-id	原始日志、结构化结果、状态和发布产物的内容哈希

4 Loop-Work 顶层方法

4.1 中心论点：模型与仓库环境共同构成领域能力

因此，本文的核心问题是：怎样在处理器 RTL 功能开发中构建一个能够工作的领域 Agent。本文所称 Agent，是能够读取仓库材料、调用工具并根据运行结果继续行动的大语言模型执行单元；Workflow 是带输入、输出、权限、成功条件、失败分流和证据责任的可执行任务图；Loop-Work（闭环工作轮）则把专业知识、工程执行和分层验证组织为这种 Agent 的外部能力结构，而不是一个更长的提示词。

本文的中心论点是：处理器 RTL 领域 Agent 的工程能力并非基础模型固有的代码生成能力，而是模型与仓库原生工程环境共同构建的系统能力；只有把项目知识、任务控制、工具执行和分层验证组织为可循环演进的工作流，并把任务反馈持续固化为可复用约束，Agent 才能稳定承担从架构分析、RTL 实现到功能验证、综合和系统运行的长链任务。四平面说明能力由什么构成，双闭环说明能力如何运行和增长；候选验收、身份绑定与失败关闭则是证据面保护后续工程节点的具体机制。

4.2 抽象顶层：四类责任状态与两条反馈边

Loop-Work 的根输入是工程目标 G 、当前状态 S 、约束边界 Γ 与验收准则 A 。控制面把根输入编译为合同、权限、owner、门禁和终止条件；知识面返回当前节点适用的规则、事实和程序化能力；执行面在授权范围内产生候选与原始观测；证据面绑定实验身份、判定器和证据资格；控制面最后作出接受、返工、保持缺口或终止的裁决。其责任状态与裁决写为

$$s = \langle C, K, X, V \rangle, \quad d(s) \in \{\text{PASS, FAIL, GAP, UNQUALIFIED, BLOCKED}\}.$$

这里的 C 是控制责任， K 是知识适用性， X 是实际候选与运行状态， V 是身份绑定、oracle 与证据资格。图 1 给出全文唯一的动态总览： C_{prepare} 和 C_{decide} 是同一控制面的准备端与裁决端， $K/X/V$ 分别对应另外三个平面。内层闭环把当前任务的反例和证据缺口送回当前合同，外层闭环只把重复且可迁移的缺口送回可复用能力。双闭环是四平面主干上的两类反馈边，不是第五和第六个平面。

上述结构不是叠加在 RTL 开发之外的管理流程，而是 RTL 生产本身的责任骨架。对处理器修改，四平面主干具体化为下列对象流：

工程目标 → 架构宪法/模块规格 → 机器任务合同 →

{可综合 RTL, TB/断言/负向 RTL, AM C/汇编程序} →

L0 模块验证 → L1 全核/AM/DiffTest → L2/L3 系统回放 → 实现反馈（按需）。

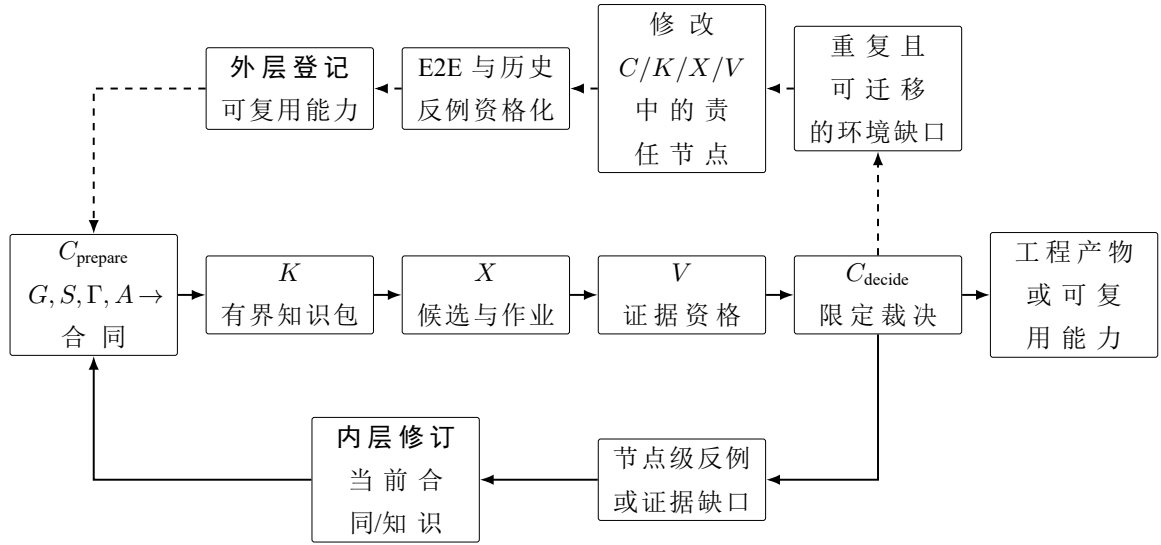


图 1: Loop-Work 顶层方法：四类责任状态沿主干流动，内外闭环以不同半径返回

架构与模块规格先固定事务 owner、周期、优先级和可见行为；任务合同再给出允许修改的依赖锥以及客观终止条件。RTL、testbench 和 AM 程序不是事后依次补齐的附件，而是同一合同下共同生成的三类对象：RTL 实现状态转换，TB 观察逐周期合同并主动注入违约，AM 程序从软件侧激活架构可见语义。执行结果分别由模块 oracle、NEMU-NPC DiffTest、系统终态和实现工具消费；任一层失败都返回最早失效的架构、实现或验证对象，而不是继续向下游传播。第 6 节先把这条对象流映射到 ysyx-workbench 的环境构件，再以一个完整 RTL 修改实例逐项展示其落地方式。

4.3 证据与控制环的四项工程设计准则

领域 Agent 要能够持续工作，仍须防止错误结果污染后续节点。Loop-Work 因而对四类关键失效给出表 6 中的工程设计准则。D1-D4 是由案例失效模式推导出的可审计接口要求；“违规事件”用于检查参考实现是否遵守准则，而不是 Popper 意义上的理论证伪实验。后文先说明各责任面的接口和工程实例化，再报告案例中观察到的拒绝事件；当前材料不估计采用这些准则后的平均效果。

四项准则回答“参考实现必须阻止什么”；它们不直接估计相对其他 Agent 架构的生产率。这个区分把设计要求、参考实现和案例观察放在同一条推理链上，又使三者承担不同责任。

表 6: Loop-Work 证据与控制环的四项工程设计准则

准则	期望约束	实现条件	违规事件
D1 身份拒绝	不同实验身份的观测不得支持当前候选	design、verification、build、workload 与 evidence 身份可核验	身份不匹配仍形成可消费 PASS
D2 负向资格	正向通过还需检查判定器能否拒绝指定非等价错误	mutant 可编译、非空、被触发并传播到观察点	合格 mutant 存活而结果仍获晋级
D3 失败关闭	长跑只有在终态 marker、命令、清理和信号状态共同闭合后才能发布 PASS	状态提交与证据发布不可分割	中断或证据不完整仍遗留可消费 PASS
D4 有界晋级	局部结果受父级门禁约束，跨任务能力受复用和稳定条件约束	回滚基线、父级 gate 与外层固化参数均被版本化	局部失败越级，或未满足固化条件的对象被标记为已晋级

5 Loop-Work 四平面的接口化描述与职责边界

5.1 静态职责树与可区分状态

在第 4 节的动态总览基础上，本章把同一方法拆成静态职责树、平面合同和案例反例，回答每一类状态的输入、输出、权限边界以及为何在本文案例中不能被无类型 GREEN 位替代。

在第 4 节定义的根输入和 $C/K/X/V$ 主干上，图 2 把动态序列投影为静态职责树：控制面是生命周期编排根，知识、执行和证据面以类型化接口接受其请求。该图只表达从属和职责分解，不表达时间先后。

表 7: 四个面的输入、输出与不可越过的职责边界

平面	主要输入	类型化输出	本面无权完成的动作
控制面	G, S, Γ, A 与上一节点状态	任务类别、节点合同、owner、门禁和终止决定	不替代专业事实、工具执行或证据判定
知识面	当前节点查询、工程作用域与来源要求	有界上下文包、专业不变量和可调用程序	不直接修改设计，也不以文档存在宣称规则已经执行
执行面	节点合同、知识包、允许命令和输入身份	候选变更、返回码、raw log/trace/report 与运行状态	不根据退出码自行宣称工程 PASS
证据面	原始观测、身份、oracle、fault set 与验收谓词	PASS、FAIL、GAP、UNQUALIFIED、BLOCKED、receipt 与索引	不改写原始事实，不越过未闭合门禁授权晋级

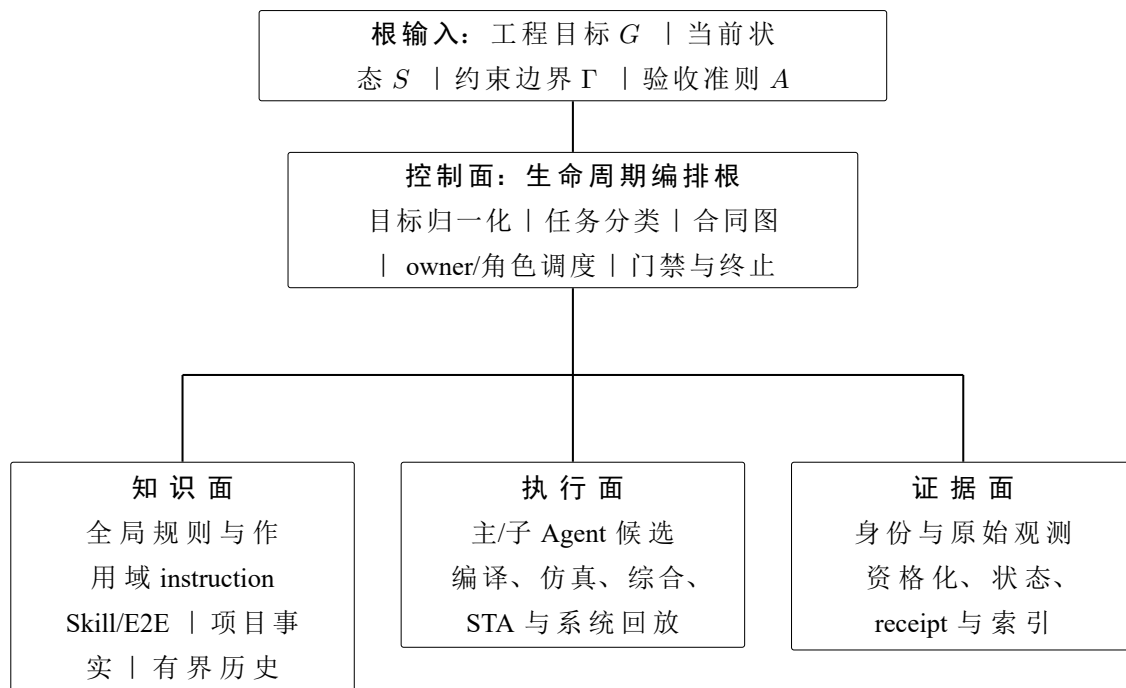


图 2: Loop-Work 的树状职责架构; 返回接口与运行顺序在图 5 中给出

四平面不是四个必须独立部署的进程或目录，而是裁决函数必须分别观察的四类责任状态。对第 4 节定义的 $s = \langle C, K, X, V \rangle$ ，若省略某一坐标后存在两个投影相同而正确裁决不同的状态，则该坐标不能被无损省略。表 8 给出本案例中的最小状态对。

表 8: 案例中省略一个责任状态后出现的不可区分状态对

被省略状态	其余状态相同的两个事件	正确裁决为何不同	单一既有机制不足之处
控制 C	同一源码、规则与证据；一次是只读 <code>review</code> ，一次是获准修改的 <code>development</code>	前者写入即越权，后者必须产生变更与验证证据	ACI 能调用工具，但不自动给出当前 <code>owner</code> 、写权限与终止合同
知识 K	同一历史 <code>PASS</code> 来源真实；一个合同仍适用旧接口，另一个合同已更换配置/事务语义	前者可引用，后者只能保持 <code>GAP</code> 并补充当前规则	<code>provenance</code> 能证明“曾由何物产生”，不自动证明“现合同仍适用”
执行 X	同一合同与判据；一项作业到达证据末端，另一项被 <code>TERM</code> 中断且 <code>marker</code> 缺失	前者可进入资格化，后者必须 <code>FAIL</code>	<code>assurance case</code> 或静态 <code>gate</code> 可组织主张，却不自动执行进程和清理状态机
证据 V	两个命令均返回 <code>0</code> ；一个绑定当前身份并能拒绝目标 <code>mutation</code> ，另一个消费旧日志或弱 <code>oracle</code>	前者可限定 <code>PASS</code> ，后者只能 <code>GAP/UNQUALIFIED</code>	<code>quality gate</code> 可算谓词，若未绑定身份和负向敏感性仍会产生假绿

表 8 支持的是案例内辨识能力：在本文选择的裁决模型中，若省略对应状态，所列事件将无法仅凭其余字段区分。ACI、`quality gate`、W3C PROV、GSN/SACM 可以分别实现这些接口的一部分；实现也可以把多类字段封装在同一数据库或程序中。本文不据此证明四平面具有全局最小性或完备性，也不排除三类、五类或其他编码方案；尚未完成的比较与证明列入第 11 节。

这一分解也不与最新 Agent 架构或产业 `self-verifying stack` 争夺同一层次。ACE-RTL 的 `generator`、`reflector`、`coordinator` 是候选生成和调试上下文的角色分解 [19]；Siemens 公开材料列出的确定性 EDA engine、NeMo Gym、OpenShell、Nemotron/Switchyard 与 GPU 加速是技术栈构件 [27]。前者可承载 K/X 的迭代，后者可分别承载执行、治理和判定接口，但这些物理构件怎样部署并不自动决定裁决状态怎样编码。就公开材料而言，它们没有报告与本文相同的 `design/build/workload/oracle/evidence` 强身份元组、`mutation` 资格和成熟度全序；这里应理解为“未报告同一责任接口”，而不是断言其系统不存在等价内部机制。反过来，Loop-Work 的四面也不替代专用模型、物理引擎或安全运行时；它只规定这些构件的输出在成为下游结论前必须保留哪些可区分责任状态。

本文的贡献定位于责任状态的显式接口化，而非新增底层工具能力：权限、知识适用性、真实执行和证据资格在既有系统中可能由不同工具隐式承担，Loop-Work 的工程增量是把它们编码为可版本化、可审计的责任字段。对表 8 中的案例状态对，这种编码保留了责任归属、旧证据失效和失败关闭消费的差异，并为 D1–D4 提供可检查接口；它不是带有 soundness、completeness 或 complexity 定理的形式系统。

目标进入体系后，首先必须确定它属于哪类工程活动、允许改变什么以及在何种条件下结束，因此先从控制面展开这棵树。

5.2 控制面：从根目标到权限化任务图

控制面回答四个问题：本轮究竟要证明什么，谁拥有当前节点，允许改变哪些对象，以及哪些结果能够终止节点。它先把自然语言请求归一化为任务类别，再把类别映射为权限、证据义务、收尾门禁与留存档位。因此，分类不是给任务贴标签，而是一个全定义路由函数

$$\mathcal{R}(\tau) = \langle W_\tau, E_\tau, Q_\tau, L_\tau \rangle,$$

其中 τ 为任务类别， W_τ 为写入权限， E_τ 为最低证据义务， Q_τ 为终止时允许或必须执行的门禁， L_τ 为 task-run 留存级别。每一种类别都必须得到四个值：“零门禁”和“禁止写入”也是明确的路由结果，并非该类别没有经过控制面。

表 9: 任务类别对写入权限、证据、门禁和留存的完整映射

类别	W_τ : 允许写入	E_τ : 最低证据	Q_τ : 终止门禁	L_τ
review	禁止; 只读源码、spec 与证据	可定位的审查发现、反例或明确无问题范围	零门禁; 结论本身即交付	none
analysis	禁止; 只读研究	调用链、数据流、状态或根因的来源指针	零门禁; 不得借分析身份落盘	none
docs	仅普通文档与文档资产	来源锚点; 可构建文档还需编译与渲染检查	不自动派生业务门; 显式文档门可选	none
development	合同内 RTL、C/C++、脚本或软件	变更指针及至少一项真实开发验证	实现阶段执行受影响 domain test; 不自动追加环境 E2E	compact
verification	禁止修改 production 源码; 可产生运行证据	配置、命令、返回码、日志及 PASS/GAP 边界	要求指定验证 evidence; 不追加环境门	none
environment	AI 规则、Skill、profile、合同与维护工具	定向自测、反例及路径对应的环境证据	只运行显式路径映射到的固定 gate pointer	compact
longrun	合同内 runner/配置; 运行中不混入未登记改动	design/config identity、marker、哈希、signal、cleanup	必须由 fail-closed 状态协议授权 PASS	durable
cleanup	仅精确列出的缓存、编译物或归档目标	删除前 preview 与删除后 result 清单	目标路径复核; 涉及索引卫生时运行对应门	compact
release	发布合同授权的源码、清单与包	完整 manifest、门禁结果和独立复核边界	release 门禁; 失败不得形成正式发布	durable

表 9 也解释了“分类决定是否写源码、需要何种证据”之间的因果关系: review/analysis 的决定是禁止写入且不增加实现门; development 的决定是允许受限写入但必须已有真实业务验证; verification 的决定是保持 production 源码只读并提交运行证据; environment、longrun 和 release 则分别增加环境指针、失败关闭或发布门禁。若用户扩大范围导致这些值改变, 控制面必须重新分类, 不能沿用旧类别的权限继续执行。

5.2.1 “大目标”与图节点的关系

本文所称大目标并非原始对话全文, 而是归一化后的复合验收目标

$$G = \langle \text{claim}, \text{objects}, \text{hard constraints}, \text{acceptance} \rangle.$$

例如，一次 RV64 性能基线复核的大目标是：在同一设计、配置和 workload 身份下，核算双发射计数器端点与 cycle/slot 守恒，检查多次运行和统计开关的非干扰性，保留 A1（第一次原始性能基线执行）和 A2（同一冻结配置下的第二次原始独立执行）的原始 FAIL，并在物理条件不足时维持 PPA 为 UNQUALIFIED。该目标同时包含多个可独立失败的义务，不能交给一个“查看全部然后判断”的超大节点。控制面因此把它拆成依赖图：身份冻结、计数器核算、重复性检查、失败状态复核、独立反例审查和聚合裁决分别成为节点。

图节点不是一段伪代码，而是一个具有输入端口、授权边界和输出端口的合同，如图 3 所示。上游只能提供合同列出的输入；节点内部动作受 owner 与权限包络约束；下游只能消费已经声明的产物和证据。缺少 evidence 的节点不能成为下游硬依赖。

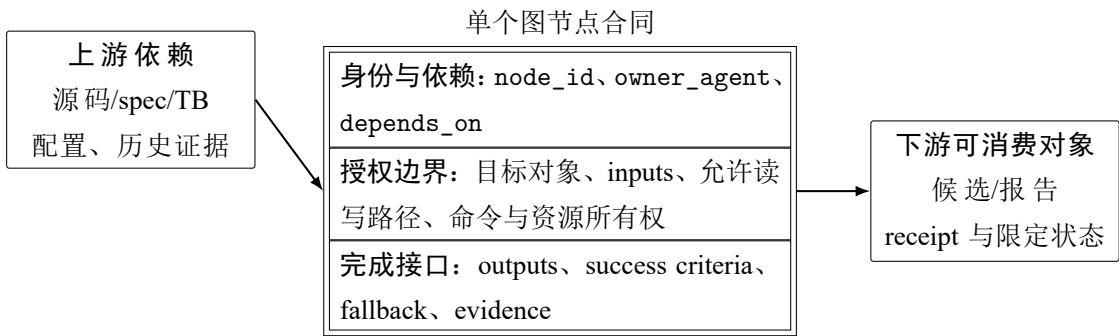


图 3: 图节点的最小合同：依赖、权限和完成接口属于同一个可审计对象

5.2.2 调度器、实现者、审查者与 oracle

在前述任务定义与设计要求的基础上，控制面进一步固定以下四类角色及其接口：

1. **调度器**：维护节点依赖、读写权限、owner 分配与状态迁移；它决定哪个节点可以开始，但不替代节点内的专业判断。
2. **实现者**：在合同授权范围内生成候选，并运行合同指定的定向验证；它提交候选和原始观测，无权用自己的总结宣布独立审查通过。
3. **审查者**：作为独立只读节点，不继承实现者的结论，依据同一验收谓词寻找反例、转录错误和越级声明；其 PASS 只覆盖合同列出的材料，不能替代未运行的 testbench、综合或 STA。
4. **判定基准 (oracle)**：由参考模型、断言、checker 或形式化谓词提供机器可判定的比较标准。它是验收接口而不是人格角色，也不能在输入身份不一致时给出有效裁决。

控制面至此给出了问题、权限、角色和终止条件，但它本身不提供 RTL 事实、专业规则与历史经验；这些输入由知识面按当前节点构造。就表 6 的准则链而言，控制面把 D4 所需的父级门禁、回滚基线和晋级阈值写入合同，使“局部接受”与“允许晋级”成为两个可区分状态。

5.3 知识面：从开放仓库到有界上下文包

知识面回答“当前节点需要知道什么、这些内容从何而来、哪些结论允许复用”。其输出不是文件名列表，而是带来源的有界上下文包

$$B_i = \langle R_i, F_i, H_i, P_i, \Pi_i \rangle,$$

其中 R_i 是不可违反的规则， F_i 是当前源码、spec、testbench、配置和工具事实， H_i 是按当前 profile 与主题检出的历史证据， P_i 是可调用的 Skill、E2E 与 oracle， Π_i 是每一项内容的路径、版本或哈希来源。控制面以节点合同为查询条件，知识面只返回完成节点所需的最小充分集合；全量聊天、无关日志和过期快照不进入 B_i 。

表 10: 知识面的层级、输出语义及其边界

知识层级	回答的问题	进入节点的形式	单独不能证明什么
全局	所有任务都不能越过哪些	任务分类、语言、记录和	不能替代模块
policy/AGENTS	行为边界	安全约束	spec 或业务测试
作用域	当前模块、接口或 PPA 路	硬约束、验证顺序、接口	文件存在不等于
instruction/spec	径必须保持什么不变量	谓词与晋级条件	不变量已被执行
项目事实	当前设计、配置、TB、工	合同允许路径内的源码片	当前事实不自动
	具与 workload 实际是什么	段和身份元数据	成为跨任务稳定知识
Skill	某类任务应当怎样重复执行	参数化步骤、合同生成器、	Skill 自测不等于
		固定命令目录与反例出口	被测设计正确
E2E	哪些规则、Skill、执行入口和观测点必须连成闭包	执行剖面的依赖、节点函数、输入输出和终端条件	任务图可展开不等于业务目标已经通过
oracle / 判定器	原始观测满足什么条件才可形成限定结论	比较器、断言、目标标记和裁决规则	给定判定器通过不等于未建模错误均可见
bounded recall	哪些已发布历史与当前节点相关	受 profile、主题和预算限制的摘要及 evidence 指针	历史 PASS 不能授权身份已经变化的新候选

这里需要区分 **instruction**、**Skill** 和 **E2E**。**instruction** 是声明式规则，规定“什么不能错”；**Skill** 不是知识本身，而是把已经选定的专业规则编译为可重复执行的步骤、合同和产物模板；**E2E** 也不是“完成知识”，而是把规则、**Skill**、执行入口、观测点和终止条件组织为可执行闭包，并检查这些对象之间的接线与证据链。控制面依据任务类别与路径选择三者，执行面调用其中的操作，证据面消费其中的验收谓词。因而 **Skill/E2E** 虽以文件实现，却不是与四面平行的第五层：它们首先属于知识面，只有被控制面选择后才在执行面运行，并由证据面裁决。

下面的代码清单直接摘自工作区生效的 `.github/instructions/rtl-generation-workflow.instructions.md`。为控制篇幅，清单仅省略各阶段内部的枚举项，保留原文中的触发条件、硬门和先后关系。它说明 **instruction** 不是对仓库文件的介绍，而是一段可由控制面查询、并能阻止执行面过早写入 RTL 的领域程序。

代码 1: 生效 RTL Instruction 的原文承重句摘录

```
1 ## 阶段 0 — 接口/控制契约先行
2 改动位于 `npc/rv64/**` 且触碰握手 / stall /
3 flush·redirect·trap / 异常序 / 访存序 / 投机恢复,
4 或跨 >= 2 module 边界时, 在进入阶段 1 之前,
5 必须先冻结六类跨模块契约, 并把受影响模块的
6 SPEC-TEMPLATE §2/§3 填满。
7
8 硬门槛: 填不出契约的格子 = 尚未理解上下游
9           = 禁止进入阶段 1 写 RTL。
10
11 ## 阶段 1 — 需求
12 把任务转写为可验证的需求清单。
13 ## 阶段 2a/2b/2c/2d
14 协议规则 -> 状态机 -> 不变量 -> 数据通路约束。
15 ## 阶段 2e — RTL 级电路拓扑
16 在写任何一行 RTL 之前, 必须先输出 RTL 级电路拓扑;
17 自审端口、状态、复位优先级和共享资源后才翻译为 RTL。
18 ## 阶段 3 — RTL
19 数据通路骨架 -> 状态机/控制 -> 协议接口
20           -> 不变量对应的断言或注释。
```

清单中的阶段 0 把跨模块事务先冻结为六类接口/控制合同; 阶段 1–2e 再把局部目标展开为需求、协议、状态、不变量、数据通路和电路拓扑; 只有这些对象可审查, 阶段 3 才获得写 RTL 的权限。控制面并非仅仅“读过”该文件, 而是把阶段产物写入任务合同和节点依赖; 执行面据此生成 topology、RTL、断言和 testbench; 证据面检查违约变体是否被 oracle 拒绝。因而, 该 instruction 把工程师“先弄清上下游再改代码”的经验转化为可观察的前置条件, 并允许审查者从缺失的 owner、周期或优先级直接判定任务尚未准备好。

以本地 RV64 子任务为例, rtl-agent-task-contract.instructions.md 没有保存某次对话答案, 而是把“审查一段 RTL”拆为工程域、任务类型、可读路径、可写路径、规范命令、必读 RTL/spec/TB、交付物和成功条件。相应 SKILL.md 规定标准操作和失败出口, 配套生成器再把字段写成带版本与 SHA-256 的 JSON 合同; E2E 检查 instruction、Skill、合同生成器、执行节点与证据入口是否接通。这样, 一条接口经验才从供模型阅读的文字, 转化为能够限制写入、拒绝越界命令并接受反例检验的工程能力; 合同检查通过仍只证明任务边界清楚, 不能替代该 RTL 的 testbench、DiffTest、综合、静态时序或 PPA 判定。

知识面还承担外部知识进入工程的净化边界。一次 task-run 的原始日志只进入证据索引; 被反复验证且跨任务稳定的结论才进入 retained memory; 需要持续约束动作的结论进入 instruction; 可重复操作进入 Skill; 需要自动组合和验收的能力才进入

E2E。这样，工作区不会把“发生过”误写成“以后都成立”，也不会把所有历史过程重新灌入模型。

有界上下文包把开放工程目录压缩为当前节点能够消费的确定输入，并以来源、版本和适用域为 D1 的身份拒绝及 D4 的版本化晋级提供知识侧条件；执行面据此产生候选和原始观测。

5.4 执行面：在合同授权内产生候选与原始观测

执行面只回答“实际做了什么”。它接收节点合同和有界上下文包，调用主 Agent、子 Agent、编译器、仿真器、DiffTest、综合、STA 或系统回放，输出三类对象：候选变更 Δ_i 、原始观测 O_i 和运行状态 σ_i 。执行面无权把三者自行解释为 PASS；即使命令返回 0，身份、日志完整性、oracle 和清理状态仍需交给证据面资格化。

单个节点只有一个对外 owner。实现者、验证者和只读审查者可以是节点图中的不同 owner，但不能同时争用同一工作区写权限或同一工程 shell。对于本地 RV64 RTL，机器合同进一步冻结 allowed_paths、write_paths、结构化 command/mode/purpose、必读上下文、交付物与成功条件。子 Agent 以隔离上下文执行；所需设计事实由合同显式提供，而非依赖父会话中未登记的叙事。

图 4 给出本地 RV64 节点从父图到证据提交的执行路径。只有合同创建、结构校验和渲染绑定均通过，owner 才得到执行权；若发现范围不足，当前节点返回 scope_extension_request，由控制面创建新版本合同并重新经过校验，旧合同不会被口头扩大。

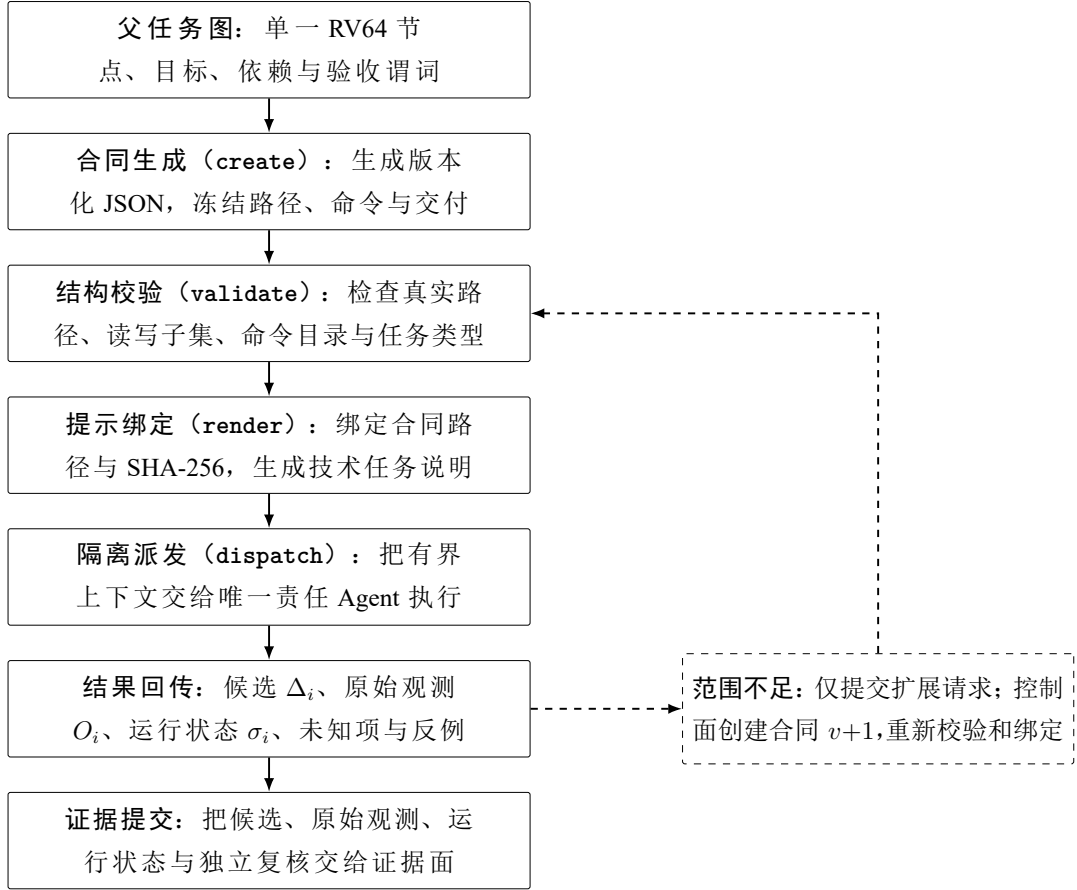


图 4: 本地 RV64 子任务合同的生成、校验、隔离执行与版本化扩展

上述流程的控制逻辑可压缩为代码 2。伪代码只描述分支与状态迁移；图 3 中的合同字段仍是节点定义的规范表达。

代码 2: 合同化 RV64 节点的最小调度算法

```

1 for node in topological_order(task_graph):
2     contract = create_versioned_contract(node, bounded_context)
3     require validate(contract)
4     prompt = render_with_path_and_sha256(contract)
5     result = dispatch(node.owner, prompt, inherit_parent_history=false)
6
7     if result.scope_extension_request:
8         replace_with_new_contract_version(node, result.request)
9         retry_from_validation(node)
10    else:
11        submit_to_evidence_plane(result.delta,
12                                result.raw_observation,
13                                result.run_state,
14                                result.review_report)

```

短命令与小时级作业共享同一合同接口，但状态保存不同。短命令可以直接回传返回码和 bounded log；长时间仿真、综合、STA 或系统回放还必须持续记录 stage、signal、evidence-complete 与 cleanup，防止交互终止或进程退出被误当作工程完成。这些原始状态构成 D3 失败关闭的执行侧前提，但仍只是执行事实；下一节才定义它们怎样成为可接受证据。

5.5 证据面：从原始观测到限定结论

证据面回答“哪些执行结果可以被相信，以及只能在什么范围内被相信”。它将执行面的 Δ_i, O_i, σ_i 与设计、配置、工具、workload、oracle 身份 I_i 绑定，再按验收谓词 Q_i 资格化：

$$\mathcal{A}_i = \text{qualify}(\Delta_i, O_i, \sigma_i, I_i, Q_i).$$

资格化至少检查对象一致性、原始日志完整性、预期终端 marker、正反例判定、返回码、信号与清理状态。独立审查报告可以成为 O_i 的一个输入，但审查文字不能覆盖 raw log，也不能把合同外对象纳入结论。

表 11: 证据面的状态及其可消费边界

状态	形成条件	下游允许的动作
PASS	本节点的身份、运行、证据、oracle 与清理全部满足合同	仅在该节点声明范围内作为硬依赖
FAIL	已观察到合同内反例、命令失败或中断状态	定位、回滚或生成下一实验；保留原始失败
GAP	输入、身份、观察点或覆盖不足，无法作出真假判定	请求补证或扩展合同；不得自动当作 PASS
UNQUALIFIED	作业完成，但量测或物理条件不足以支撑性能/PPA 结论	仅保留诊断价值，不允许晋级或 Pareto 比较
BLOCKED	上游合同、上下文、发布或基础设施前提未闭合	停止下游消费，先修复阻塞对象

对于长时间作业，最小 PASS 条件为

$$\text{PASS} = C_{\text{run}} \wedge C_{\text{evidence}} \wedge C_{\text{cleanup}} \wedge \neg C_{\text{signal}}.$$

所以“进程退出 0”只满足 C_{run} ，并不等于工程成功。证据不完整、清理失败或收到 HUP/INT/TERM 时，状态必须失败关闭。原始 FAIL 可以由修订后的 oracle 在冻结输入上产生一个新的 replay 结论，但旧状态不得被追溯改写。

本文最终把第 k 轮有效交付定义为

$$\mathcal{D}_k = \langle \Delta_k, \mathcal{E}_k, \mathcal{P}_k, \mathcal{V}_k \rangle,$$

其中 Δ_k 是工程变更, $\mathcal{E}_k$ 是正向与负向证据, $\mathcal{P}_k$ 是设计、构建、workload、oracle 与原始产物的来源绑定, $\mathcal{V}_k$ 是限定判定。任何一项缺失, 产物只能保持候选、GAP 或 UNQUALIFIED; 若缺失源于上游合同、上下文、发布或基础设施前提未闭合, 则状态为 BLOCKED。这些状态均不能成为有效交付。至此, 四个面的职责已经定义完整; 下一节只说明它们如何完成一次任务, 不再引入新的组成层。身份绑定、目标 mutation、原子终态和父级门禁分别使 D1、D2、D3、D4 获得可裁决接口; 其案例观察统一在表 24 中报告。

5.6 四面协同的一次数据流

一次任务沿图 5 的实线主干前向运行。控制面首先产生节点合同 N_i ; 知识面依据合同构造有界上下文包 B_i ; 执行面产生候选与原始观测 $X_i = \langle \Delta_i, O_i, \sigma_i \rangle$; 证据面返回资格化对象 $\mathcal{A}_i$; 最后仍由控制面依据任务图决定接受、返工、保持 GAP 或终止。图中 C_{prepare} 与 C_{decide} 是同一控制面的两个生命周期端点, 不是两个控制面。

四类对象不能混在一条“成功日志”中: N_i 证明动作被授权, B_i 证明上下文有来源, X_i 记录实际执行, $\mathcal{A}_i$ 才给出可消费结论。任一对象变化都会使其后的对象失效。例如 design-id 或 workload 改变后, 旧 X_i 不能继续支持新的 $\mathcal{A}_i$; oracle 改变时可以对冻结 O_i 产生 versioned replay, 但不能重命名旧的 raw 状态。

5.7 外层能力固化的参数化条件

图 1 已经给出外层反馈边。一次环境改动不能因为文件已经写入就被称为可复用能力; 它至少要进入规则真源 $R(z)$ 、接入可发现入口 $W(z)$ 、通过正例与目标反例资格 $Q(z)$, 并被后续业务实际调用 $A(z)$ 。重复成立次数、跨合同复用次数和稳定窗口由项目参数 $\theta = (r_{\min}, u_{\min}, t_{\min})$ 表示, 外层固化条件写为

$$\text{Ext}_{\theta}(z) = R(z) \wedge W(z) \wedge Q(z) \wedge A(z) \wedge [n_{\text{replay}} \geq r_{\min}] \wedge [n_{\text{reuse}} \geq u_{\min}] \wedge [\Delta t \geq t_{\min}].$$

该式是参考实现的操作性判定规则, 而非经统计校准的普遍定律。阈值必须在观察正式结果之前按任务风险、复核成本和陈旧代价登记; 未满足条件的对象仍可作为单次实现或候选能力使用, 却不能据此声称已经形成稳定的跨任务能力。相应地, 环境条件 Γ 下的有效 Agent 能力写为

$$\mathcal{C}_{\text{eff}}(\Gamma; \theta) = \{\mathcal{T} \mid \mathcal{T} \text{ 能在固定验收标准下形成某个 } \mathcal{D}, \text{ 且新复用构件满足 } \text{Ext}_{\theta}\}.$$

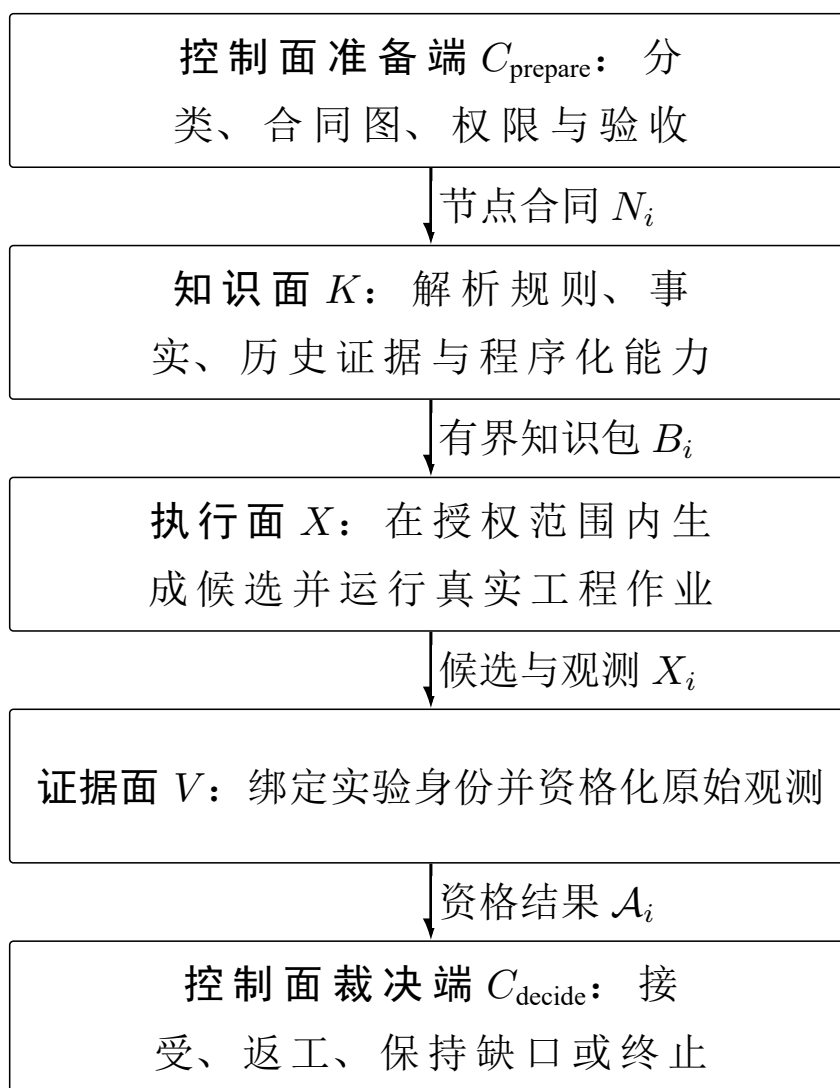


图 5: 一次任务对四面的调用顺序; 该图只表示前向数据流, 不表示反馈周期

只有验收标准不变且 $C_{\text{eff}}(\Gamma_1; \theta) \subsetneq C_{\text{eff}}(\Gamma_2; \theta)$ 时，才称环境能力扩大。本案例的主张映射尚未为每个历史构件完整编码 `replay` 次数、复用次数和稳定窗口，阈值本身也未做敏感性分析。因此本文不把纵向里程碑解释为这一严格集合关系已被证明；它们只说明若干构件进入了后续任务。阈值校准与逐构件审计统一列入第 11 节，Token、代码行和脚本数量不能替代该审计。后文的双闭环运行机制将在不增加新架构元素的前提下，接口化描述两条反馈路径的状态、触发条件和停止规则。下一章先把上述接口映射为参考实现，再进入双闭环的运行过程。

6 Loop-Work 在 ysyx-workbench 中的工程实例化

6.1 映射原则与边界

四平面方法不要求使用某一种语言或目录结构。本工作区的参考实现选择 Markdown/JSON 表达专业知识与机器合同，以 C 保存低开销任务生命周期，以 shell 连接工程工具，以 Python 实现合同校验、证据索引和原子发布。本章严格按照控制、知识、执行、证据的顺序展开；七张逐文件、逐字段追踪表统一移至附录 A，正文只保留决定数据流、责任边界和判定范围的实现机制。表 28 给出完整职责映射。实现载体可以跨面：例如 E2E 的定义属于知识面，选择属于控制面，运行属于执行面，结果资格化属于证据面；因此不能用“某个文件属于哪个目录”代替接口分析。沿这一映射，D1 落到身份 manifest 与 freshness 检查，D2 落到 mutation 与 oracle 资格，D3 落到长跑状态提交和原子发布，D4 落到父级门禁、Selector 与成熟度全序。这里列出的是准则的实现观察点，而不是用文件存在替代准则成立。

6.2 可执行领域环境：AM、NEMU、NPC 与 EDA 的协同关系

Loop-Work 的四个责任平面只有落到真实工程对象上才构成领域 Agent。ysyx-workbench 中最基本的执行闭环由 AM、NEMU 和 NPC 构成：AM 负责把 C/汇编程序、运行时和平台适配链接为同一 RISC-V 工作负载；NEMU 在软件中执行该工作负载并给出参考架构状态；NPC 则以 Verilator 运行被开发的 RTL 处理器。DiffTest（差分测试）在 NPC 每次退休指令后比较程序计数器、通用寄存器、浮点寄存器、确定性 CSR 和特权态，使“参考语义”与“硬件实现”在同一程序身份上闭合。Yosys、OpenSTA 与开源 PDK 位于另一条支路：它们不回答指令是否正确，而把同一冻结 RTL 转化为网表、面积和时序观测，使功能候选接受实现可行性反馈。

图 6 给出这套环境的主数据流。它同时说明为什么缺少其中任一环都会削弱 Agent：没有 AM，测试程序和平台接口难以在参考端、目标端间复用；没有 NEMU，NPC 缺少独立的逐退休参考；没有 NPC，软件模型无法成为 RTL 交付证据；没有

testbench、分层系统 workload 与 EDA 工具，Agent 只能观察程序终态，不能定位周期合同或判断实现条件。

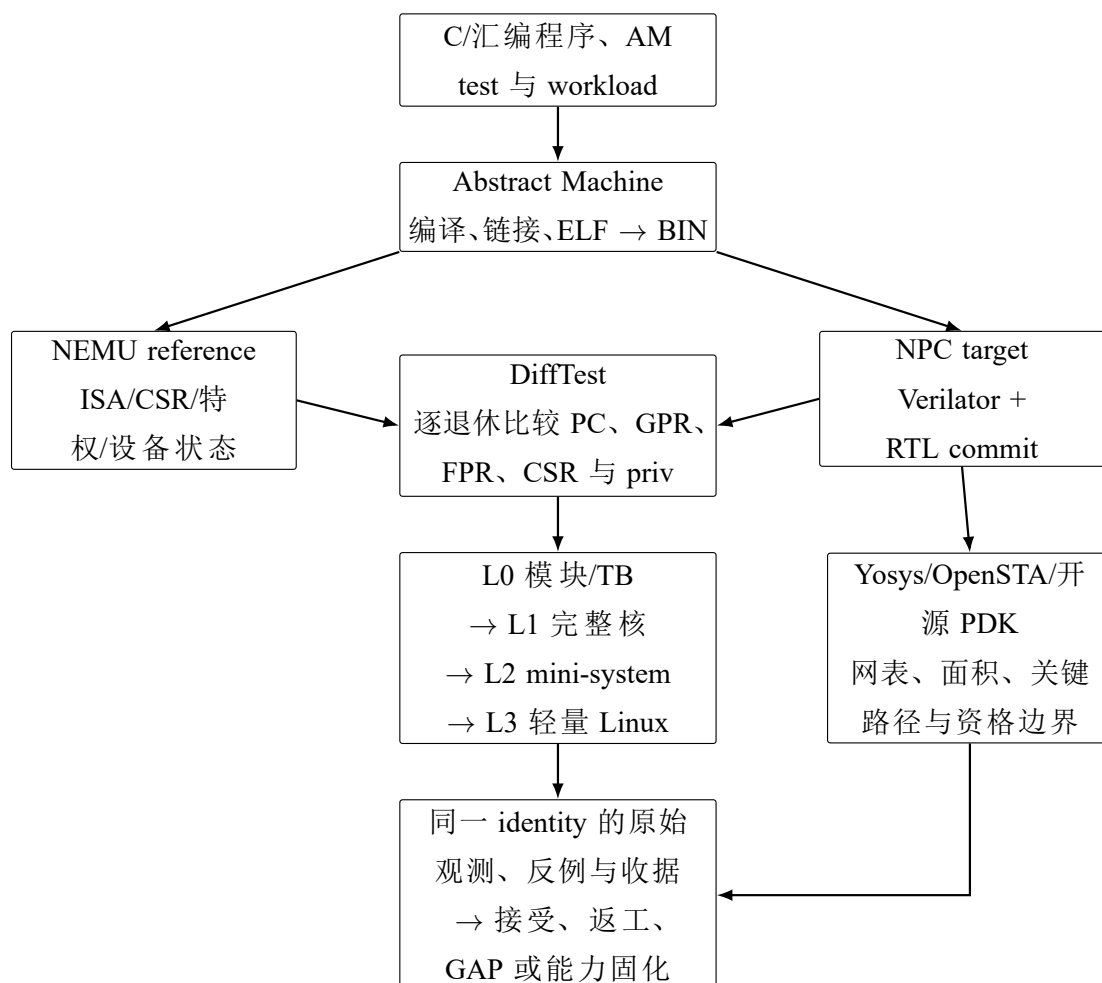


图 6: ysyx-workbench 中领域 Agent 的工程数据流：程序、参考语义、RTL 目标、分层验证与实现反馈

图 6 只表达执行面与证据面的工程数据流，因此不把 Goal、Selector 或任务回传箭头叠加在 NPC 方框上；这些控制面关系分别在后文的根目标合同与中等粒度切片选择中展开。

这一关系由真实构建入口承载。abstract-machine/Makefile 依据 ARCH 分离 ISA 与 platform，编译 C/C++/汇编、链接 AM 与 klib，再产生 ELF/image；abstract-machine/scripts/platform/nemu.mk 与 platform/npc.mk 把同一类 BIN 分别交给 NEMU 或 npc/sim。后者再按配置选择 single、SoC 或 RV64 后端。nemu/src/cpu/diffTest/ref.c 暴露内存、寄存器、执行步进和中断接口，NPC 端据此完成逐退休比较。工程环境因此不是一组工具名，而是一条具有明确上游、下游和观测语义的可执行数据通路。

分层证据进一步固定了每种观测的结论范围：L0 检查受影响 module/transaction 的 TB、立即断言和负向变体；L1 合取 module、official、AM 与 DiffTest；L2 运行 OpenSBI 与 M→S→U、Sv39、timer/PLIC/UART、原子操作和自然关机；L3 运行 Linux 6.6/PID1，并覆盖 process/COW、timer、tmpfs、UART IRQ 与唯一终态。完整 Ubuntu 22.04 属于显式授权的可选再认证，不能由低层 PASS 自动触发。该成本递增结构使 Agent 在局部修改时先使用低成本反例，在跨模块和系统风险出现时再升级验证。

6.3 控制面的实现：生命周期、代次与门禁路由

控制面位于目标合同与真实工程节点之间。它不负责实现 RTL，也不替代业务验证，而负责回答：当前工作属于哪类任务、哪些路径由本轮拥有、哪些证据已经形成、何时需要独立复核，以及哪组固定门禁能够授权终止。其上游是用户目标、分类策略和显式作用域；下游是图执行器、domain gate、证据面与归档器。没有这一层，Agent 可以在多轮对话中继续修改，却无法机械区分“同一候选的复核”与“已经产生新候选”。

参考实现采用一个轻量生命周期控制器。scripts/agent-flow.sh 是可移植启动适配器：仅当 C 源变化或缓存二进制不存在时，使用 C11 -O2 -Wall -Wextra -Werror 编译临时文件并原子替换，随后把仓库根和全部参数交给控制器。代码 3 展示了这一真实入口。临时文件、trap 清理和原子 mv 共同避免半编译二进制成为后续任务的隐式状态。

代码 3: agent-flow.sh 的编译缓存与原子启动入口

```
1 repo_root=$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)
2 source_file="${repo_root}/scripts/agent-flow.c"
3 binary_dir="${AGENT_FLOW_BIN_DIR:-${repo_root}/.github/cache/agent-flow}"
4 binary="${binary_dir}/agent-flow"
5
6 mkdir -p "${binary_dir}"
7 if [[ ! -x "${binary}" || "${source_file}" -nt "${binary}" ]]; then
8     temporary="${binary}.tmp.${BASHPID}"
9     cleanup_compile() { rm -f -- "${temporary}"; }
10    trap cleanup_compile EXIT
11    "${CC:-cc}" -std=c11 -O2 -Wall -Wextra -Werror \
12        "${source_file}" -o "${temporary}"
13    chmod 0755 "${temporary}"
14    mv -f -- "${temporary}" "${binary}"
15    trap - EXIT
16 fi
17 exec "${binary}" --repo "${repo_root}" "$@"
```

真正的状态与门禁路由位于 `scripts/agent-flow.c`。这种 C+shell 分工不是论文贡献本身，而是为日常短任务降低解释器和全量 profile 开销的实现选择。FlowMeta 保存 task、class、archive mode、status、完整计时字段和 generation；后者承担旧证据失效语义。代码 4 中的 record 路径只要登记或再次触碰对象，就递增代次并把状态恢复为 ACTIVE；读取门禁缓存时，代次、gate ID 和 PASS 状态必须同时匹配。由此，新候选不能消费旧代次 PASS。

代码 4: agent-flow.c 的真实状态字段与两个非连续分支（省略处显式标记）

```
1 typedef struct {
2     char task_id[MAX_ID];
3     char task_class[MAX_CLASS];
4     char archive_mode[16];
5     char status[MAX_STATUS];
6     int overhead_target_percent;
7     int64_t started_ms;
8     int64_t initial_work_ms;
9     int64_t gate_ms;
10    int64_t paused_ms;
11    int64_t pause_started_ms;
12    int generation;
13 } FlowMeta;
14
15 /* ... command_record() 的路径/门禁登记前部从略 ... */
16 if (touched) {
17     ++meta.generation;
18     snprintf(meta.status, sizeof(meta.status), "ACTIVE");
19     if (write_meta(state_dir, &meta) != 0) {
20         return 2;
21     }
22 }
23
24 /* ... gate_pass_for_generation() 的逐行解析前部从略 ... */
25 if (atoi(raw_generation) == generation && strcmp(raw_id, gate_id) == 0 &&
26     strcmp(raw_status, "PASS") == 0) {
27     snprintf(result->id, sizeof(result->id), "%s", raw_id);
28     snprintf(result->status, sizeof(result->status), "%s", raw_status);
29     result->elapsed_ms = strtoll(raw_elapsed, NULL, 10);
30     snprintf(result->log_path, sizeof(result->log_path), "%s", raw_log);
31     result->reused = 1;
32     found = 1;
33 }
```

生命周期操作的抽象语义如下。命令名只是参考接口：

固定 gate registry 不接受 Agent 临时拼接任意 shell 字符串；它把显式路径映射到已注册的验证入口。因此，控制面只选择“哪一个既有判定器承担本次责任”，不能为获得绿色结果临时改写判定命令。任务生成新路径、重新分类或补登记门禁时 generation 递增，旧候选缓存立即失效；只有同一代次的候选、审查和正式结束可以复用门禁结果。这样，“完成”由 class policy、作用域、真实 evidence 和固定 checker 共同决定，而不是由模型自由解释。

代码 5 给出这一映射的三个真实分支。修改端到端配置时选择“执行剖面接线检查”，修改任一 Skill 时选择“Skill 审计”，修改 RV64 子任务合同的规则、生成器或策略时还要增加“RTL 合同检查”。

代码 5: agent-flow.c 从显式修改路径到固定门禁的条件节选（共同错误返回分支从略）

```
1  if (strcmp(path, "scripts/agent-e2e.sh") == 0 ||
2      starts_with(path, "scripts/e2e/") ||
3      starts_with(path, ".github/e2e/"))
4      add_gate(gates, "profile-bindings");
5
6  if (starts_with(path, ".github/skills/"))
7      add_gate(gates, "skill-audit");
8
9  if (starts_with(path,
10     ".github/skills/prepare-rtl-task-contract/") ||
11     strcmp(path,
12     ".github/ai-env/contracts/agent-env-rtl-task-contract.json") == 0 ||
13     strcmp(path,
14     ".github/instructions/rtl-agent-task-contract.instructions.md") == 0)
15     add_gate(gates, "rtl-task-contract");
```

因此，路径不是事后报告中的装饰信息，而是控制面选择验证责任的输入；固定门禁也不会因为 Agent 更换措辞而变化。

6.4 知识面的实现：专业知识如何编译为可执行能力

知识面的关键不是存放更多 Markdown，而是把不同语义的知识放入不同载体，并沿固定编译链进入执行。instruction 表达“什么不能被违反”，Skill 规定“怎样稳定地产生产物”，machine contract 固定“本次允许读写与运行什么”，profile/E2E 表达“这些能力怎样组合才能证明完成”。它们的关系为

professional invariant → executable procedure → task contract
→ graph node → domain evidence.

例如，接口策略要求在修改 RTL 前冻结握手、stall、flush/redirect、异常序、访存序和投机恢复；RTL 生成策略又要求先明确需求、状态机、不变量、数据通路与拓扑。任务 Skill 规定怎样选择任务类型、怎样调用生成器以及哪些越权组合必须拒绝；配套 Python 生成器再把这些义务参数化为 allowed_paths、write_paths、结构化 command/mode/purpose、success criteria 和反例出口。若发现范围不足，子节点只能请求新版本合同，不能通过一句补充说明改变旧边界。profile 再把合同绑定到 owner、

函数、输入输出和依赖闭包，最后由真实 testbench、DiffTest、系统或 EDA 节点承担业务结论。

6.4.1 从接口经验到机器合同

知识编译链可以由一条真实 RV64 规则说明。任务合同 instruction 规定，任何子任务都必须显式给出表 31 中的字段；缺项不得派发。这些字段不是为了增加文档篇幅，而是分别封闭一个常见失效源：目标含混、源码范围无限、只读审查意外写入、自由拼接命令、缺少必要上下游，以及“工具返回 0”被直接解释为技术完成。

代码 6 直接摘录该 instruction 的“派发前硬门”。其中读写范围、命令三元组和成功条件共同构成授权边界；它没有规定 Agent 必须得到 PASS，而是保留 unknown、反例、GAP 与范围扩展出口。

代码 6: RV64 子任务合同 instruction 的逐字摘录（方括号为省略标记）

```
1 # RTL Agent Task Contract
2 ## 派发前硬门
3
4 每个子任务必须具备以下字段；缺任一项不得派发：
5 1. `engineering_domain`: 固定为 `local-rv64-rtl`，正文说明是本地
6    Verilog/SystemVerilog 数字电路设计、验证或 PPA。
7 2. `task_kind`: `read-only-review | implementation |
8    verification | ppa-analysis`。
9 3. `goal`: 单一、可验证，不夹带父目标的全部历史。
10 4. `allowed_paths`: 规范化的仓库相对路径集合；不含绝对路径、
11    `..` 和工作区外路径。
12 5. `write_paths`: 允许落盘的最小子集。`read-only-review` 必须为空；
13    实现任务必须显式列出。
14 6. `allowed_commands`: 每项固定为 `command/mode/purpose` 三元组；
15    `mode` 只能是 `read-only` 或 `write-within-scope`。
16 [第 6 项的 canonical catalog 细则及第 7 项从略]
17 8. `deliverables` 与 `success_criteria`: 分别声明输出和客观验收条件。
```

Skill 本身不是合同生成器。它的 Markdown 正文先固定能力分档、调用次序、越权拒绝、范围扩展和失败出口；rtl_task_contract.py 才执行实际的 create/validate/render。代码 7 直接展示 SKILL.md 中先于合同产物生效的约束。由此可以区分：Skill 是可复用操作规约，schema-v2 JSON 是某次任务的机器授权真源，rendered prompt 只是绑定该 JSON 路径与 SHA-256 的派生视图，而不是第二份合同。

代码 7: SKILL.md 的关键约束逐字摘录（非连续，方括号表示省略）

```
1 # Prepare RTL Task Contract
2 ##  workflow
3 1. 只给予任务所需的最小充分上下文；不附带整轮历史或无关日志，
4    也不删掉已知必要的上下游、调用链或证据来源。范围尚未收敛时，
5    从模块级只读输入集合开始。
6 2. 用脚本创建契约。只读复核使用 `read-only-review`，落盘实现使用
7    `implementation`，验证与 PPA 分别使用 `verification`、
8    `ppa-analysis`。
9 3. 校验并渲染；新派发必须走 canonical `create → validate → render`，
10   把渲染结果原样放进子 agent 提示，不要另写一份可能漂移的边界说明。
11 [本项关于 fork_turns、上下文隔离及哈希绑定的中间段落从略]
12   JSON 未通过 validate、或派发文本手工改写了渲染后的工程范围/
13   状态边界时，该轮结果只能登记为 `candidate-only`。
14   `scope_extension_request` 必须生成新的 versioned JSON，重新
15   validate/render 并绑定新 SHA；不能在旧提示后口头追加范围。
16 工程命令不是自由文本。`--allow-read-command` 与
17 `--allow-write-command` 只接收 canonical `COMMAND`；生成器从 JSON
18 catalog 填入固定的 `mode/purpose` 枚举和中文标签，任务不能自定义
19 purpose。详细意图写在 `goal/deliverables/success_criteria`，不会增加
20 命令变体。只读任务只允许 canonical 只读命令类，写型命令必须同时
21 声明最小 `write_paths`。
22 [同一文件后续关于材料模式及 dispatch 的段落从略]
23 ## 能力分档
24 - 默认只读探索：`workspace-files + read-only-review`，允许在模块级
25   白名单内自主寻找遗漏、反例和必要调用链。
26 - 限定材料复核：`prompt-supplied-self-contained`，只用于冻结事实的
27   第二遍逻辑检查。
28 - 实现/验证/PPA：使用相应 `task_kind`，显式声明最小写路径和
29   write-scoped 命令；不要因模板惯性一律降级为只读 reviewer。
30 渲染提示必须允许子 agent 报告 `unknowns`、显式假设、反例、
31 替代假设、`scope_extension_request` 与 `confidence_and_basis`。
32 信息不足时允许 `inconclusive`，禁止强制 PASS；不得设置固定
33 blocker/发现数量上限。
34 ## 边界
35 - 本 skill 只规范任务边界和证据，不替代 RTL 四段式、接口契约、
36   功能回归、综合/STA 或 PPA hard gate。
37 [同一文件“workflow”中的 canonical 校验与渲染入口]
38 python3 .github/skills/prepare-rtl-task-contract/scripts/rtl_task_contract.py validate <
39   contract.json>
40 python3 .github/skills/prepare-rtl-task-contract/scripts/rtl_task_contract.py render <
41   contract.json>
```


允许路径由用户范围、必读材料和合同自身组成；命令模式与用途来自固定目录；交付和成功条件原样进入 JSON。合同校验不是格式检查。生成器会拒绝“写路径不属于可读路径”“只读审查声明写入”“写型命令没有写路径”以及“无工具模式仍携带命令”等组合。该边界只有进入 E2E 才成为可发现能力：执行剖面的一行同时绑定节点名、实现函数、责任主体、规则/Skill/生成器输入和预期输出；节点函数再实际运行生成器的审计、自测与命令行自测。代码 8 给出实际接线，表 32 进一步区分环境自测与 DUT 业务验证。

代码 8: 合同能力在 E2E 任务图中的真实接线（换行仅为排版）

```
rtl-task-contract | agent-system |
e2e_agent_system_rtl_task_contract | agent-system |
agent-env-rtl-task-contract.json;
rtl-agent-task-contract.instructions.md;
prepare-rtl-task-contract/SKILL.md;
rtl_task_contract.py |
create/validate/render and out-of-contract-path mutation
must be checked
```

这个例子给出四层不同的可信性：instruction 证明硬门被明确表达，Skill 规定可重复操作，合同生成器证明规则能够形成受限机器对象，E2E 证明能力接线可运行；只有最后的业务验证才证明合同内的 RV64 技术主张。把这些层级合并成一句“E2E 完成知识”，会同时夸大环境检查的结论并掩盖真正的业务 oracle。

6.5 执行面的实现：确定环境、图闭包与节点作业

执行面的参考实现把知识面选出的 profile 展开为有序节点，并为每个节点建立确定的工具环境。其物理主链到“产生原始观测”为止；report、manifest 和 publication 属于下一节的证据面，不能因为它们由同一 runner 调用就混成一个职责。

代码 9: profile 从任务身份到原始节点观测的执行主链

```
1 task_slug + selected profile
2   -> rebuild active live index
3   -> brief(non-history focus, bounded budget)
4   -> recursively resolve @include profile closure
5   -> bind node/module/function/owner/inputs/outputs
6   -> load deterministic tool environment
7   -> dispatch node function
8   -> raw evidence/<node_id>.log + rc + runtime state
```

图执行器只承担通用编排，不把全部工程语义塞入一个 shell。profile 使用 node/module/function/owner/inputs/outputs 六元组；@include 递归展开可复用子图；module library 实现具体节点函数。scripts/agent-env.sh 在所有非交互节点之前固定仓库根、NEMU、AM、NPC、Yosys/STA、RISC-V 工具链与可选 JDK 路径，scripts/agent-run.sh 将同一环境提供给手工或单节点命令。因此，profile 的上游是机器合同与能力索引，下游是具有确定环境的 domain command，而不是另一个自然语言规划器。

6.5.1 工程执行后端及其原始产物

NEMU、NPC、AM 和 EDA 工具不是论文结论，也不是同一种“仿真器”。它们位于执行链的不同位置，分别产生参考端观测、目标端观测、工作负载镜像或实现分析报告。为避免将工具名称误读为验收结果，表 33 给出各组件的工程含义、直接产物与证据边界。

这组后端形成两条可区分的数据流：程序源码经交叉工具链与 AM 形成冻结工作负载，随后分别进入 NEMU 参考端和 NPC 目标端；DiffTest（差分测试）逐条比较两端退休后的架构状态，系统级判定基准则检查更长事务的终止条件。RTL 另经 Yosys 形成网表，再由 OpenSTA 消费网表与约束形成时序观测。前一条回答功能状态是否一致，后一条回答结构与时序代理量如何变化，二者不能互相替代。

环境自身的 agent-system profile 是该模式的一个实例。九个物理配置项通过一个 discovery include 展开为十一节点，但论文不把九行文件当作方法；表 34 按方法责任把节点重新分组。

证据与边界：环境自身的运行证据。一次已发布环境自验证中，profile 在 42 s 内展开 11 个节点，manifest 记录 PASS=11/11 和 13 个 ordinary evidence payload；manifest 入库后，evidence API 共可查询 14 个资产。completion-publication.md 绑定 task report、manifest、context brief、profile resolve、evidence index、dispatch log 与 nodes.tsv 七份 SHA-256。更能说明失败关闭的是另一次故障注入：十一业务节点均为 PASS，但 context brief 因独立 focus 不足产生 WARN，overall

仍被发布为 `blocked`；随后使用具有工程语义的 `OooIntBackend` task slug 重跑才形成 `completed publication`。这里被拒绝的不是 RTL，而是一条缺少可信启动上下文的环境完成声明。

这一 profile 验证的是 AI 环境自身是否可发现、可执行和可追溯；它不能证明 NPC、NEMU、Linux 或 PPA 正确。业务目标必须使用自己的 `testbench`、`DiffTest`、系统或 EDA evidence。这种边界是方法的一部分：环境 GREEN 与设计 GREEN 不能互相替代。

6.6 证据面的实现：资格化、失败关闭与原子发布

6.6.1 长跑终止语义适配器

长时间仿真、综合、STA 或系统回放的进程状态只属于执行面；工程 PASS 属于证据面。Loop-Work 因而规定一个最小提交条件：

$$\text{PASS} = C_{\text{run}} \wedge C_{\text{evidence}} \wedge C_{\text{cleanup}} \wedge \neg C_{\text{signal}},$$

其中 C_{run} 表示主命令正常结束， C_{evidence} 表示规定 `marker`、哈希和检查已到末端， C_{cleanup} 表示配置恢复与临时资源清理成功， C_{signal} 表示收到 HUP、INT 或 TERM。进程返回 0 只是第一个条件，不能单独提交工程状态。

代码 10 是 `scripts/task-run-status.sh` 的关键提交分支。runner 必须显式调用 `task_run_status_mark_evidence_complete` 才能置位证据完成；最终提交同时检查主命令、清理和信号状态。任何条件不成立时，脚本写入带 `stage`、证据完成位、`cleanup` 返回码和 `signal` 的 FAIL，而不会因为 shell 最终返回 0 留下绿色状态。

代码 10: task-run-status.sh 的证据授权与失败关闭提交（错误处理从略）

```
1 task_run_status_mark_evidence_complete() {
2     TASK_RUN_STATUS_EVIDENCE_COMPLETE=1
3 }
4
5 task_run_status_finalize() {
6     local command_rc="${1:?command return code is required}"
7     local cleanup_rc="${2:-0}"
8     local final_rc="${command_rc}"
9     local status_line
10
11     if [[ "${TASK_RUN_STATUS_EVIDENCE_COMPLETE}" -eq 1 &&
12         "${command_rc}" -eq 0 &&
13         "${cleanup_rc}" -eq 0 &&
14         "${TASK_RUN_STATUS_SIGNAL}" == "none" ]]; then
15         local pass_write_rc=0
16         if _task_run_status_write "PASS"; then
17             return 0
18         else
19             pass_write_rc=$?
20         fi
21         status_line="FAIL rc=${pass_write_rc} stage=${TASK_RUN_STATUS_STAGE}"
22         status_line+=" evidence_complete=${TASK_RUN_STATUS_EVIDENCE_COMPLETE}"
23         status_line+=" cleanup_rc=${cleanup_rc}"
24         status_line+=" status_write_rc=${pass_write_rc}"
25         _task_run_status_write "${status_line}" || return "${pass_write_rc}"
26         return "${pass_write_rc}"
27     fi
28
29     if [[ "${final_rc}" -eq 0 ]]; then
30         if [[ "${cleanup_rc}" -ne 0 ]]; then
31             final_rc="${cleanup_rc}"
32         else
33             final_rc=1
34         fi
35     fi
36     status_line="FAIL rc=${final_rc} stage=${TASK_RUN_STATUS_STAGE}"
37     status_line+=" evidence_complete=${TASK_RUN_STATUS_EVIDENCE_COMPLETE}"
38     status_line+=" cleanup_rc=${cleanup_rc}"
39     if [[ "${TASK_RUN_STATUS_SIGNAL}" != "none" ]]; then
40         status_line+=" signal=${TASK_RUN_STATUS_SIGNAL}"
41     fi
42     _task_run_status_write "${status_line}" || return 1
43     return "${final_rc}"
44 }
```

参考实现 `scripts/task-run-status.sh` 是执行面与证据面的终止语义适配器：`runner` 在启动时写 `RUNNING`，随工程推进更新 `stage`，只有全部证据检查结束后才能置 `evidence-complete`，最后再提交状态。`test-task-run-status.sh` 对附录表 35 的八类路径逐项断言；因此代码 10 的意义不在于四个条件本身，而在于它为所有小时级 `runner` 提供了统一、可故障注入的提交协议。该协议正是十九小时 `Ubuntu` 事务、综合和 `STA` 能够在中断后保持可审计边界的公共下游接口。

6.6.2 证据原子发布、有界召回与方法自验证

证据提交以后仍不能直接进入长期知识。发布器把节点标识、来源执行剖面、实现模块、责任主体、函数、状态、输入、输出和证据路径组成节点元组，再与原始观测及运行身份绑定；对应机器字段为 `node/source-profile/module/owner/function/status/inputs/outputs/evidence`。系统依次生成节点表、调度日志、任务报告、清单和证据索引；随后重算普通证据的路径、尺寸与 `SHA-256`，验证执行剖面闭包和事件顺序，最后才生成完成标记。完成态先同步暂存文档，再在单个数据库事务中复核完成标记、发布记录与产物集合；任何净化、索引、哈希或数据库步骤失败都会撤销本轮发布。参考实现主要位于 `scripts/e2e/lib/report.sh`，但其抽象职责是“证据原子提交”，不是格式化报告。

下一轮只从已发布对象中选择最小充分历史。已发布的 `instruction`、`Skill` 和执行剖面保持可查询；稳定 `memory` 与任务档案可查询；原始日志只向上下文提供路径、尺寸、`SHA-256`、标记和有界摘要。启动摘要受执行剖面、非历史主题和 `2400-token` 硬预算约束；历史运行通过 `run-id` 与证据索引按需读取。这样，证据面可以向知识面反馈已验证结论，而不会把全部聊天和重型日志再次灌入模型。

方法自身同样接受分层验证。轻量模式只检查控制器与 `shell` 状态协议；目标级模式追加 `schema`、`policy`、`artifact`、`Skill`、合同和 `profile` 绑定审计；`release` 模式再构建相对路径包、扫描本机路径与私有 `marker`，并复核数据库与交付边界。参考实现由 `agent-maintain.sh` 编排，并由 `package-ai-dev-env.sh` 形成可迁移包。两者位于外层能力闭环的末端：只有规则、执行入口、反例 `gate` 和交付包同时成立，一项环境修改才算从个人经验变成可复用工程能力。

6.7 RTL 生产闭环的工程实例：从架构合同到分层反馈

前述映射说明了环境构件各自承担的责任，但领域 `Agent` 是否真正可用，要看它们能否共同完成一次处理器修改。以下以“在保持 `RV64` 架构语义、精确异常和既有回归的前提下，把候选接纳与退休路径由单发射扩展到双发射，并建立 `CPI/PPA` 反馈入口”为贯穿场景。这个任务不是某一个文件的修改：它同时影响取指包、跨槽相关

性、反压、redirect/flush、ROB 顺序退休、计数端点、AM workload、NEMU 参考状态和 NPC 观测。Loop-Work 首先构造这种依赖关系，然后才允许生成 RTL。

6.7.1 根目标驱动的写前分析与候选生成

RV64双发射 OoO 核持续开发以“RV64 双发射 OoO 核持续开发、Selector 驱动优化与 CPI/PPA 晋级 Goal”为控制面的根合同。该 Goal 不要求每轮重建一套流程资产，而是要求在功能、架构、系统和实现硬门内，每轮推进一个真实且有界的工程切片。代码 11 直接给出该 Goal 的目标与主循环；其中 Selector 指 workflow 中的优化切片选择器，不是 RTL 发射队列中选择就绪指令的 issue selector。

代码 11: RV64 持续开发 Goal 的根合同摘录

```
1 最终目标：持续开发本地 RV64 双发射乱序执行核，
2 在不破坏功能正确性、架构一致性、系统可启动性
3 和实现硬门的前提下：
4     1. 闭合架构债务、验证债务和历史缺陷；
5     2. 建立可信、可复现、可比较的性能与 PPA 基线；
6     3. 使用中型优化选片器选择下一项工程工作；
7     4. 通过定向测量、RTL、回归、综合与 STA 降低 CPI；
8     5. 控制面积、时序和功耗退化，形成 Pareto 候选。
9
10 唯一主循环：
11     1. 轻量分类，只读取当前切片直接需要的文件；
12     2. 校验 design-id、baseline、receipt 与 selector input；
13     3. 输入不变则复用决策，实质变化才重跑 Selector；
14     4. 一次只执行一个主切片，先写清对象、周期与观测；
15     5. 先跑定向门，具备晋级可能时才升级高成本验证；
16     6. 把新测量、区间、硬门和因果结论写回 canonical
17     evidence，再选择下一切片。
```

因此，源代码修改不是主循环的起点。Agent 先把根目标合同与冻结 design-id、架构/性能收据、历史正反例、瓶颈普查和 PPA 资格组合为写前分析对象，列出候选的 RTL module/signal/transaction、依赖锥、周期与协议不变量、可观测量、验证成本和回滚边界。Goal 中的“中型优化选片器”即本文所称的“中等粒度 Selector”，二者指同一控制面机制。Selector 先排除身份不一致、缺少验证入口、破坏硬门或无法安全回退的候选，再选出一个中等粒度切片。这里的“中等”不按代码行数或文件数定义，而指一个主要假设、有界影响范围、明确不变量和可独立关闭的证据链。

被选切片随后编译为任务合同：允许读写的 RTL/spec/TB 范围、定向 testbench 与 AM workload、NEMU-NPC 对照方式、综合/STA 条件、成功阈值和失败处置均成为机器字段。执行面只有在合同校验后才获得写权限，并按需生成或修改 RTL、testbench、负向变体和 AM 程序；证据面再把模块检查、全核回归、DiffTest、分层系统回放及实现反馈绑定到同一实验身份。满足合同的候选才进入下一轮，功能回归、归因不足或资格条件缺失则分别触发返工、回滚或显式 GAP。这样，根目标合同保持方向稳定，单切片使每轮变化可解释，而不是让一次自然语言请求直接跨越分析、实现和晋级。

6.7.2 一次合同实例：双发射切片的授权对象

第 6.4 节已经说明 instruction-Skill-E2E 怎样产生受限任务；这里仅展示它在双发射切片中形成的机器合同。代码 12 转录一次 DI-3 静态复核的真实 schema-v2 字段：scope 保存读写和命令权限，context 保存必读材料，空 write_paths 表明这是只读复核；节点 owner 由上层 profile/dispatch 绑定，不写入合同 JSON。

代码 12: DI-3 双发射静态复核的 schema-v2 JSON 字段（省略处显式标记）

```

1  "schema_version": 2,
2  "contract_type": "local-rtl-task",
3  "task_id": "v13s-di3-static-current-review",
4  "engineering_domain": "local-rv64-rtl",
5  "task_kind": "read-only-review",
6  "goal": "复核 DI-3 的 pair formation、plain-memory capability
7  resident 与双 memory terminal owner 三个静态红项，判定属于
8  checker 锚点漂移还是 production RTL 架构回归。",
9  "scope": {
10   "workspace_root": ".",
11   "allowed_paths": [
12     [首项诊断证据路径从略]
13     "npc/rv64/eval/ppa/tools/architecture_hard_gates.py",
14     "npc/rv64/design/arch/rv64-architecture-ppa-contract.md",
15     "npc/rv64/vsrc/scheduling", "npc/rv64/vsrc/memory"
16     [其余已授权路径及合同自身路径从略]
17   ],
18   "write_paths": [],
19   "allowed_commands": [
20     {"command": "rg", "mode": "read-only",
21      "purpose": "search-allowed-paths"},
22     {"command": "sed", "mode": "read-only",
23      "purpose": "view-selected-lines"},
24     {"command": "sha256sum", "mode": "read-only",
25      "purpose": "hash-declared-artifact"}
26   ]
27 },
28 "context": {
29   "required_files": [
30     ".github/instructions/rtl-agent-task-contract.instructions.md",
31     "npc/rv64/design/arch/rv64-architecture-ppa-contract.md",
32     "npc/rv64/eval/ppa/tools/architecture_hard_gates.py"
33     [其余必读文件从略]
34   ],
35   "material_mode": "workspace-files",
36   "supplied_material": []
37 },
38 "deliverables": ["逐红项给出 RTL 信号链、checker 条件、反例、
39 替代解释与最小下一实验；定位 checker 漂移或 RTL 回归。"],
40 "success_criteria": ["逐项给出 PASS、GAP 或 inconclusive；引用真实
41 文件与信号，并保留 unknowns 与 scope_extension_request。"]

```

validate 会拒绝路径穿越、只读任务写入、写命令没有写路径、自由修改命令用途和 no-tools 模式携带命令等组合；render 再把同一 JSON 的路径与 SHA-256 写入

子 Agent 输入。子 Agent 若发现遗漏模块，只能提出 `scope_extension_request`，由控制面创建新版本合同并重新校验，不能在旧提示后口头扩大权限。该实例 JSON 的哈希为 `1fbe840f9c4215f485662e57f210e48012aafe9857ab51516104963b057ca451`；此哈希只绑定该 JSON，不绑定被读取的架构合同、RTL 或测试。由此，Agent 仍可自主追踪调用链和提出反例，但探索范围、写入范围和结论范围不会在执行中静默漂移。

第 6.4 节代码 8 所示 E2E 节点只证明这一合同机制可被发现、执行并拒绝越界输入；本实例的 RTL 正确性仍由下游 TB、DiffTest 和系统层节点判断。这样，知识编译机制与一次实际合同各自只出现一次：前者解释“怎样生成并约束”，后者解释“本轮具体授权了什么”。

6.7.3 架构合同、RTL、TB 与 AM workload 的共同生成

合同冻结后，Agent 并非只输出一个 RTL 差分，而是形成一组互相约束的工程对象。控制面把目标编译为架构、实现、验证和性能节点；知识面选择 ISA/特权规范、架构宪法、模块 spec、Instruction、任务 Skill、历史反例和 CPI/PPA 全序；执行面再按表 12 推进。这里的固定步骤构成任何 RTL 修改都不能绕过的生产主干，风险触发步骤则按事务影响面增加，避免把所有局部改动无差别升级为系统长跑。

表 12: RV64 RTL 生产闭环中的固定节点与风险触发节点

类型	节点	Agent 产生或消费的对象	进入下一节点的条件
固定	架构宪法与模块规格	消费 ISA 与既有设计；产生事务 owner、生命周期、周期、优先级、可见行为和回滚边界	受影响合同能够先于源码接受审查
固定	机器任务合同	产生允许读写路径、命令、验证对象、交付物与终止谓词	合同校验通过，依赖锥与权限不再含混
固定	RTL、TB 与负向变体	共同产生 RTL、定向 TB/断言和可编译违约版本	正向行为通过，目标违约被唯一 marker 拒绝
固定	同身份基线复验	消费修改前 inventory；执行修改后同范围或更强回归	验证范围不弱于基线，design、build 与 workload 身份一致
触发	AM 与 DiffTest	现有 workload 不能激活架构语义时生成 C/汇编及 BIN；交给 NEMU 与 NPC	PC、GPR、FPR、CSR、priv 及终态满足合同
触发	L2/L3 系统验证	当修改涉及特权、MMU、中断、设备或长事务时，消费系统 workload	对应层级的完整 case 与唯一终态闭合

续下页

表 12: RV64 RTL 生产闭环中的固定节点与风险触发节点（续）

类型	节点	Agent 产生或消费的对象	进入下一节点的条件
触发	CPI/PPA	当修改声称性能或结构收益时，消费冻结 workload、网表、SDC、库与 PDK	功能硬门、量测合同和物理资格按全序成立

下面用同一条 FENCE.I 指令生命周期展示表中四类源文件如何闭合，而不是分别举四个无关例子。位于 npc/rv64/design/arch/ 的顶层架构宪法 ooo-core-architecture.md 规定：凡是修改 instruction lifecycle、state owner 或 flush/redirect 路径，必须先更新宪法，再更新模块 spec，最后才能改 RTL。据此，取指桥规格把自修改代码（Self-Modifying Code, SMC）的可见性先写成代码 13 中的合同。普通 store 不承诺立即改变取指视图；真正的架构边界是 FENCE.I 提交后产生 mmu_flush，清除旧 I\$/ITLB 状态并重定向、重取后继 PC。

代码 13: 取指桥架构规格中的 FENCE.I 可见性合同摘录

```

1 # npc/rv64/design/specs/ooo-fetch-axi-bridge.md
2 ### R2.5 ordinary-store / FENCE.I 可见性合同
3
4 - 普通 store 的 issue、完成、提交与 AXI write fire
5   都不得逐项 invalidate I$；写后未执行 FENCE.I 的
6   自修改代码不要求取指看到新字节。
7 - FENCE.I 保持 stop/drain 序列化，在 commit 后生成
8   mmu_flush_i，整体清 I$/ITLB，并沿既有
9   redirect/refetch 重新取得 fence 后继 PC。
10 - 永久结构门必须贯通
11 FENCE.I commit -> mmu_flush -> fetch cache/ITLB clear;
12 旧 store-fire 拓扑必须为 RED。

```

修复前的 task-run 反例记录 fencei-fix-plan.md 先冻结因果链与原始 DiffTest 差异；分层验证闭合后，后续规格化再把已经验证的语义固化为上述 R2.5 可见性合同。代码 14 保留反例记录的关键字段：已有用例经间接跳转，在 store 退休后才取目标指令，因而无法观察“旧 uop 已经进入 ROB”这一窗口；新反例把被改写指令放在 FENCE.I 的直线后继，使 NPC 提交旧的 addi +1，而 NEMU 读取新字节并执行 addi +2。这份.md 的作用不是解释一个 PASS，而是在动 RTL 前冻结“怎样才算真正复现缺陷”。

代码 14: FENCE.I 反例记录中的根因与原始差分摘录

```
1 # fencei-fix-plan.md — Step A 反例
2 root cause:
3 1) fence.i = 纯 no-op; 全仓无 retire 触发 flush/redirect。
4 2) store->ifetch 只失效 fetch-packet cache 的 valid 位,
5   绝不 flush 流水/ROB。
6 3) 直线 fall-through 上紧邻 fence.i 的目标 L 会以旧字节
7   进入 ROB; store 只失效 cache, 陈旧 L 仍可直接提交。
8
9 [npc-diff] mismatch at dut commit
10   pc=0x000000008000002c inst=0x00150513
11 [npc-diff] x10 ref=0x0000000000000066
12           dut=0x0000000000000065
13
14 existing tests: indirect jalr fetches L after store retirement;
15 new test: store -> fence.i -> fall-through L in the same/next packet.
```

修复后, 局部 TB 不以“程序最终退出”为唯一 oracle, 而是直接消费规格中的旧包、flush 和重取三个事件。代码 15 来自 npc/rv64/testbench/tests/tb_ooo_fetch_axi_bridge.sv: 先填入旧取指包并证明 warm hit, 随后施加 mmu_flush, 明确要求相同 PC 不再命中旧 cache、不产生 stale response, 最后从 AXI 接收新包。该 TB 能拒绝切断桥内“mmu_flush 输入 → cache/ITLB clear → refetch”链路的可编译负向 RTL, 却不能单独证明上游“FENCE.I commit → mmu_flush_i”接线贯通; 上游接线由永久结构门和 AM/DiffTest 端到端用例共同约束。

代码 15: 取指桥 TB 对旧指令、flush 与重取的逐事件观测摘录

```
1 mmu_flush = 1'b1;
2 tick();
3 mmu_flush = 1'b0;
4 #1;
5 tb_check1("V10B FENCE.I pulse returns bridge idle",
6           dut.state_q == S_IDLE_TB, 1'b1);
7 start_fetch_ctx("V10B FENCE.I post-update request accepted",
8                FUSION_PC1, `PRIV_M, {`XLEN{1'b0}});
9 #1;
10 tb_check1("V10B FENCE.I rejects stale cache hit",
11           dut.cache_hit_w, 1'b0);
12 tb_check1("V10B FENCE.I exposes no stale response",
13           fetch_rsp_valid, 1'b0);
14 drive_fetch_packet("V10B FENCE.I replacement packet refetch",
15                   FUSION_PC1, FUSION_BEAT3);
16 expect_rsp("V10B FENCE.I replacement packet response",
17            RESP_OK, RESP_OK, FUSION_BEAT3);
```

模块 TB 证明内部事件链，AM 程序则从软件可见层制造同一个反例。代码 16 来自 `am-kernels/tests/cpu-tests/tests/smc-fencei-trigger.c`。程序在运行时把目标指令从 `addi a0,a0,1` 改写成 `addi a0,a0,2`，再执行 FENCE.I；同包和下一包两种布局分别压住取指宽度与包边界。AM 把该文件链接成同一 BIN，一路交给 NEMU reference，一路交给 NPC target；DiffTest 在目标指令退休拍比较 x10，从而把体系结构语义直接连接到 RTL 观测。

代码 16: AM 自修改代码程序中的 FENCE.I 触发序列摘录

```

1 unsigned long smc_fencei_probe_samepkt(unsigned long seed);
2 unsigned long smc_fencei_probe_nextpkt(unsigned long seed);
3
4 asm(
5  ".text\n"
6  ".option push\n"
7  ".option norvc\n"           // 强制 4 B 指令：保证取指包/对齐计算精确
8  ".p2align 3\n"
9  ".globl smc_fencei_probe_samepkt\n"
10 "smc_fencei_probe_samepkt:\n"
11 "  la    t0, 1f\n"           // t0 = 被改写指令 L 的地址
12 "  li    t1, 0x00250513\n"   // 新指令: addi a0,a0,2
13 "  sw    t1, 0(t0)\n"       // SMC store: 用新字节覆写 L
14 "  .p2align 3\n"
15 "  fence.i\n"
16 "1:\n"
17 "  addi  a0, a0, 1\n"       // 文件旧字节；正确执行应被重取替换
18 "  ret\n"
19 ".option pop\n"
20 );
21
22 int main(void) {
23   check(smc_fencei_probe_samepkt(100) == 102);
24   check(smc_fencei_probe_nextpkt(200) == 202);
25   return 0;
26 }

```

四份源文件形成一条可追踪闭环：架构.md 定义何时必须看到新指令；反例.md 固定旧实现为何会错及哪个观察点能区分；TB 把该观察点下沉为逐周期 oracle；AM 程序把同一语义提升到软件可见层并交给 NEMU/NPC 共同消费。修复前的 x10=0x65/0x66 差异因此不会被“既有 fence 测试通过”掩盖；修复后的模块 marker、AM check 与 DiffTest 收敛又分别约束内部事件、程序语义和参考一致性。若任何一层失败，反馈沿对象依赖返回到对应 spec、RTL、TB 或 workload，而不是在论文中把后续层的 PASS 倒写为前层正确。冻结记录分别保存了

[V10B-FENCEI-STALE-INSTRUCTION] PASS 与 smc-fencei-trigger PASS，使修复前原始 FAIL、模块级消费者证明和软件级回归能够被分开复核。

同样的生产主干适用于双发射扩展。模块级先构造同周期双接纳、跨槽 RAW/WAW、lane1 受 lane0 阻塞、redirect 清除年轻事务、精确异常与顺序退休等正向场景，再故意删除相关性检查或清空条件，确认错误版本虽可编译却会被目标 oracle 拒绝。若现有软件不能激活某个交互，Agent 在 am-kernels 中增加定向 C/汇编程序，经 AM 链接为同一 BIN；该 BIN 分别进入 NEMU reference 与 NPC target，DiffTest 比较每次退休后的架构状态。Agent 因而既产生 RTL、TB 和程序，也消费负向变体、参考语义与工具反馈。

风险继续增大时，同一 design-id 进入 L0-L3。模块事务先在 L0 定位，完整核的 official/AM/DiffTest 在 L1 复核架构可见行为，OpenSBI mini-system 在 L2 检查 M→S→U、Sv39、中断和设备，Linux 6.6/PID1 在 L3 检查进程、文件系统、UART IRQ 和有序关机。只有修改声称 CPI/PPA 收益时，同一 RTL 才进入 Yosys/OpenSTA/PDK 支路；若 SDC、宏、活动率或工艺条件不足，结果用于定位而保持 UNQUALIFIED。这条链使“增减 RTL 后整体仍正确”不再依赖一次全量测试的模糊印象，而是依赖受影响语义沿验证压力逐层升级。

6.7.4 反馈如何改变下一轮 Agent 能力

每个失败首先沿当前任务的责任边返回：RTL 事务错误回到实现/合同，身份漂移回到构建和输入，oracle 不敏感回到 TB/反例，系统终态不足回到对应 L2/L3 节点，PPA 条件不足则停在实现资格。只有同类缺口重复出现、能够抽象成稳定规则并在后续任务中复用时，外层闭环才更新环境。例如，跨模块 flush/owner 错误被固化为接口 Instruction，重复派发歧义被固化为合同 Skill，场景串证被固化为 NEMU-only、NPC-only 和 integrated profile，长跑误终止被固化为 fail-closed 状态协议，系统缺陷则尽可能下沉为 AM 测试、模块 TB、立即断言或 compile-success mutation。

因此，环境修改和处理器开发并非两条平行工作：更复杂的 RTL 任务暴露环境缺口，环境将已验证经验转成下一轮可执行约束；新的约束又使 Agent 能闭合更长的工程链。一次工作轮最终产生的不只是代码，还包括架构合同、测试 workload、反例、原始观测、限定收据和必要的环境能力更新。这正是本文所称“模型与仓库环境共同构建领域 Agent”的具体含义。

7 四面协同的双闭环运行机制

顶层方法已经定义四类责任状态和两条返回路径，前一章给出了同构参考实现。本章只沿时间轴描述这些既有接口：内层闭环优化当前任务，外层闭环把重复且可迁移的缺口固化到控制、知识、执行或证据面。双闭环不是新的架构层，也不改变四面

的职责边界。就四项设计准则而言，内层闭环以身份核验、负向资格和终态提交落实 D1–D3，外层闭环及 Selector 以父级门禁和稳定条件约束 D4；第 5 节定义接口，本章说明这些接口何时触发。

7.1 状态、事件与反馈对象

本文把第 k 个工作轮的状态写为

$$x_k = (D_k, C_k, V_k, L_k, P_k),$$

其中 D_k 为设计快照， C_k 为配置与输入身份， V_k 为验证器/oracle 版本， L_k 为未闭合义务账本， P_k 为当前成熟度。这只是状态记录的类型定义，并不是预测模型。本轮合同冻结有限验收谓词集合 $Q_k = \{q_1, \dots, q_n\}$ ，并为每个谓词给出期望状态 $\hat{o}_k(q)$ 。真实工具产生的观测状态限定为

$$o_k(q) \in \{\text{PASS}, \text{FAIL}, \text{GAP}, \text{UNQUALIFIED}\}.$$

身份不一致、证据不完整和 freshness 失效形成 blocker 集 B_k ；本轮未闭合项被操作性地定义为

$$E_k = \{q \in Q_k \mid o_k(q) \neq \hat{o}_k(q)\} \cup B_k.$$

BLOCKED 不是工具观测值，而是 $B_k \neq \emptyset$ 时控制面对下游消费作出的阻塞裁决。这里没有抽象的“差值”或待学习的反馈函数： o_k 必须来自编译、仿真、testbench、综合/STA、退出状态、计数器或身份哈希； E_k 的每一项必须能回指原始观测。动作选择遵循固定优先级。若 $B_k \neq \emptyset$ ，保持 GAP 且不猜测性修改 RTL；若候选触发 hard gate，先回滚并复验；若确认是 oracle 缺陷且原始日志完整，只生成版本化 replay；其余情况从能够区分现有竞争假设的实验中，按 focused、module、system、PPA 的成本层级选择最低层实验，同层并列时采用合同中预先登记的顺序。 x_{k+1} 是动作执行后由收据记录的新状态，而不是公式预测值。由此，反馈规则可以由另一位工程师按相同合同复核，无法判别时则保持 GAP。

7.2 内层交付闭环：失败不是统一重试

同一条 FAIL 对应的正确动作可能完全不同。表 13 给出工作区已实际使用的分流。

表 13: Loop-Work的反馈分类与下一动作

观测类型	判别问题	正确动作
DUT 功能或周期失败	是否定位到首个错误事务及 现有合同违反	最小 RTL 修复，新 design-id，focused 后逐层回归
身份/freshness 失败	当前结果是否属于不同 RTL、 filelist、配置或 workload	重建或重跑证据；不修改 RTL 猜测原因
oracle 假阳性/假阴性	DUT 是否到达终态，原始日志是否完整	修正 oracle，做正/反例测试，对冻结日志版本化重放
mutation 未触发/未传播	负向变体是否真的改变并到达观察点	保持 GAP，修复刺激或 observation， 不宣称敏感
量测不合格	counter 是否守恒、身份是否 稳定、观测是否非干扰	拒绝性能候选，先修量测合同或探针
signal/证据/清理不完整	是否到达预定证据末端	保留 FAIL 与 stage/signal，禁止自动 PASS
hard gate 回归	候选是否破坏上级合同或 guardrail	回滚到最近合格基线，并复验恢复

7.3 内层算法与停止条件

内层闭环必须在执行前规定停止语义，否则反馈循环会退化为无界重试。代码 17 把终态分为证据缺口、硬门回滚、限域接受、父级晋级和不可判定 GAP；预算只限制循环长度，不改变这些终态的含义。

代码 17: Loop-Work 的最小执行算法

```
1 x <- bind(design, config, inputs, oracle, maturity)
2
3 while budget_available:
4     r <- freeze_claim_and_gates(x)
5     open <- classify_unmet_gates(x, r)
6     a <- select_by_priority_then_cost(open, contract_order)
7     y <- execute_and_observe(x, a)
8     A <- seal(identity, commands, rc, raw_output, y)
9
10    if identity_mismatch(A) or evidence_incomplete(A):
11        return GAP(A, "binding/evidence")
12
13    e <- compare(r, y)
14    if violates_hard_gate(e):
15        rollback_to_last_qualified_state()
16        verify_rollback()
17        record(REJECTED_OR_ROLLED_BACK, A, e)
18        continue_or_stop()
19
20    if scoped_error_is_zero(e):
21        if promotion_requested and parent_gates_pass_same_identity(A):
22            return PROMOTED(A)
23        return SCOPED_ACCEPTED(A, promotion=false)
24
25    if error_is_actionable(e):
26        x <- transition(x, a, e)
27        continue
28    return GAP(A, e)
```

算法强调三个不等式：

SCOPED_ACCEPTED $\neq$ PROMOTED,

replay PASS $\neq$ raw live-run PASS,

benchmark completed $\neq$ measurement qualified.

7.4 外层能力闭环：交付单元与跨轮知识固化

有效产物不是“Agent 的回答”，而是可重建状态迁移的最小证据包。表 14 先固定每轮必须留下的对象，再讨论其中哪些稳定反馈可以进入外层能力；这使表格成为本节论证的输入，而不是紧贴标题出现的孤立清单。

表 14: 每个 Loop-Work 的最低产物

产物	内容	下游用途
task/contract	分类、目标、允许路径/命令、成功条件	约束 Agent 动作和审查范围
input/identity manifest	RTL、配置、工具、workload、oracle 的路径与哈希	判断证据能否比较或复用
raw observation	log、trace、counter、wave、exit code、signal	保留未被摘要改写的事实
verification receipt	正向、负向、marker、计数与边界	判断 oracle 与门禁是否合格
decision trace	hypothesis、竞争假设、反例、回滚与选择理由	使下一轮不再盲目重复
status/publication	PASS、FAIL、GAP、UNQUALIFIED 及 stage/cleanup/review	限定可消费的完成主张
reusable update	稳定结论进入 memory/skill/instruction/checker	改善下一轮环境而非堆积临时日志

最后一行使 Loop-Work 同时成为环境更新循环。不是所有反馈都应写入同一处：本轮 design-id、风险和已知事实进入 retained memory；一次运行与原始失败进入 task-run/evidence index；可复用的人类决策规则进入 instruction，短小且需要重复操作的流程才封装为 Skill；能机械裁决的字段进入 JSON schema、checker 或 assertion；需要自动发现和执行的能力再接入 profile 与 e2e function。其数据流可写为

engineering counterexample → stable rule extraction
→ instruction/Skill/checker → profile wiring
→ qualified execution → next engineering action.

这种分流避免两种相反错误：其一，把所有历史过程塞进 memory，使下一轮被过时细节淹没；其二，只在某次最终回复里记住规则，导致换会话或换模型后能力消失。若一条稳定规则已经写入 instruction 但没有 profile/checker 能发现，环境仍处于“知道但做不到”；若 profile 能执行但没有 mutation 或反例证明其会拒绝错误输入，则

处于“能跑但不会判”。只有闭合这两个缺口，工程经验才形成 Agent 可执行、可检验的外部能力。

失败、回滚和未晋级也是有效产物。它们排除了一个候选方向，保护了基线，并为下一轮提供更窄的搜索空间。只有“既没有改变判断、也没有留下可验证状态”的循环才是无效消耗。

附录表 36 给出四条“工程反例—方法增量—后续复用”的过程链。它们说明新结构确实进入后续任务，但由于模型、代码和任务难度同时变化，只能作为外层闭环的过程证据，不能作为独立因果效应估计。

7.5 内外闭环的触发关系

不是每一次 FAIL 都应修改 AI 环境。单次 RTL 实现错误通常只在内层修复；只有缺口具有重复性、可迁移性，且无法由当前合同中的局部动作消除时，才进入外层。操作规则是：单次实现错误只修当前候选；跨节点重复且可抽象为稳定不变量的错误进入 instruction/Skill；公共 runner、oracle、环境或历史污染类缺口分别进入执行/证据、知识/E2E、环境适配或有界召回接口。完整分流见附录表 37。

外层修改完成后不能追溯改变旧 task-run。它必须形成新版本规则或执行器，先通过自身正例和针对性反例，再由后续业务任务实际复用；否则只是一项环境候选，而不是能力扩展。这样，图 1 上方返回通道的起点和终点都有可检验条件。

7.6 Selector 驱动的中等粒度 CPI/PPA 优化闭环

处理器性能优化不是“修改 RTL 后观察 CPI 是否下降”的单步任务。CPI（每条退休指令的周期数）可能因测量窗口、退休端点、统计逻辑或 workload 身份改变而下降；PPA（性能、功耗、面积）也可能在功能错误、时序未约束或 memory macro 缺失时得到更好看的代理数字。该工作流在完整设计点的成熟度晋级之外，增加一个中等粒度优化切片 Selector：它把“下一步改什么”先改写为“下一项能够区分竞争解释、并能独立关闭的工程动作”，再把动作结果反馈给下一轮选择。

本文所谓中等粒度，不由代码行数或文件数量定义，而由四个条件共同界定：（1）一次切片只承载一个主要因果假设；（2）其 RTL、配置、workload 与工具依赖形成有界依赖锥；（3）必须预登记不能破坏的功能、架构、周期和实现硬门；（4）可以用一条独立收据结束为接受、回滚、RESEARCH_REQUIRED 或 GAP。它大于单信号试改，因为需要覆盖与假设直接相关的观测和验证；又小于无约束的全核联合搜索，因为不得在结果出现后持续扩大修改范围。

这里的 Selector 是工作流控制面的“优化切片选择器”，不是 RTL 中从 issue queue 选择就绪微操作的 issue selector。后者决定本周期哪些指令占用功能单元，是被分析

和优化的微体系结构对象；前者只根据工程事实决定下一项测量、资格化或可回退实验，无权改变 RTL 语义。其数据流可写为

`{current receipts, catalog, policy}` → 同身份事实 → 决策类
→ {非支配前沿, RESEARCH_REQUIRED}
→ selected slice.

固定 policy、活动目录与派生决策分别位于 `optimization-slice-selector-policy-v1.json`、`optimization-slices-current.json` 和 `evidence/optimization-slice-current.json`。代码 18 保留了该 policy 的真实决策次序和权限边界。派生决策绑定所有输入 SHA-256；它既不能回写 authority receipt，也不能替代完整设计点的 promotion checker。

代码 18: 中等粒度 Selector policy 的真实字段摘录

```

1 "decision_class_order": [
2   "correctness_closure", "state_reconciliation",
3   "causal_measurement", "ppa_qualification",
4   "rtl_experiment", "diversity_experiment"
5 ],
6 "selection_modes": {
7   "information": {
8     "maximize": ["information_gain", "evidence_confidence",
9                 "reversibility"],
10    "minimize": ["execution_cost", "functional_risk",
11                "scope_width"],
12    "ambiguous_front_action": "RESEARCH_REQUIRED"
13  },
14  "multiobjective": {
15    "maximize": ["performance_ratio", "area_efficiency_ratio",
16                "qualified_power_efficiency_ratio"],
17    "scalarization_scope": "same_measured_pareto_front_only"
18  }
19 },
20 "claim_boundary": {
21   "selector_authorizes_rtl_semantics": false,
22   "selector_authorizes_promotion": false,
23   "front_py_remains_promotion_authority": true
24 }

```

信息型切片最大化信息增益、证据可信度与可回退性，同时最小化成本、功能风险和影响范围；出现多个互不支配项时返回 RESEARCH_REQUIRED，不能依 catalog 顺序或隐藏权重消歧。只有同 design-id 的实测候选才进入多目标比较；时序属于不可交易的 hard gate，而不是能被 CPI 或面积收益抵消的软目标。通过时序门后，性能、面积效率和已资格化的功耗效率才形成 Pareto 前沿，最终晋级仍由独立的 front.py 裁决。

该规则不是散文建议。代码 19 节选自 optimization_slice_selector.py：信息型切片必须在全部“应最大化/应最小化”字段上不差且至少一项严格更好；实测候选采用区间保守支配，左候选的最坏值低于右候选的最好值时，就不能宣称确定支配。因此，epsilon 只抑制数值噪声，不能把重叠区间强行排序。

代码 19: Selector 中的信息支配与区间保守支配

```

1 no_worse = all(a[name] >= b[name] for name in INFO_MAXIMIZE)
2 no_worse = no_worse and all(
3     a[name] <= b[name] for name in INFO_MINIMIZE)
4 strict = any(a[name] > b[name] for name in INFO_MAXIMIZE)
5 strict = strict or any(a[name] < b[name] for name in INFO_MINIMIZE)
6 return no_worse and strict
7
8 # robust_dominates: left worst-case vs. right best-case
9 for name in axes:
10     a = left[name]
11     b = right[name]
12     epsilon = epsilons[name]
13     if a["low"] < b["high"]:
14         no_worse = False
15         break
16     if a["low"] > b["high"] + epsilon:
17         strict = True
18 return no_worse and strict

```

表 15 把 Selector 的中间决策与完整设计点晋级放入同一条链。节点之间是硬依赖：前一节点没有形成可消费收据，后一节点只能保持缺口，不能由 Agent 依据“趋势合理”跳过。

表 15: RV64 双发射核切片选择与 CPI/PPA 晋级边界

节点	必需输入	形成的产物	不能晋级的条件
架构发现 ARCH_DISCOVERY	冻结 RTL、接口、配置、已有缺陷与验证入口	模块/事务图、债务清单、待闭合 owner 与观察点	调用链未知、设计身份未冻结
架构闭合 ARCH_CLOSED	逐项债务、接口不变量和定向验证	每项债务的修复、反例或明确 GAP	只通过局部正例，没有跨模块事务解释
历史缺陷下沉与架构门	已知 Linux/DiffTest/全核反例	最低充分层的 TB、断言或可编译负向 RTL，以及全量回归	历史错误只能在昂贵系统层复现，或判定器无法检出变体
架构稳定 ARCH_STABLE	同一设计快照上的功能、乱序、恢复和副作用证据	带 design-id 的稳定架构收据	production RTL、配置或验证身份仍在变化
性能量测合同	架构稳定收据、workload、ROI 与计数器定义	起止端点、重复次数、守恒式、非干扰检查和原始状态协议	ROI 或 lane 端点含混，统计开关改变 cycles/retired

续下页

表 15: RV64 双发射核切片选择与 CPI/PPA 晋级边界 (续)

节点	必需输入	形成的产物	不能晋级的条件
性能基线 PERF_BASELINE	同一合同下的重复执行与独立核算	可重放 cycles、retired、CPI/IPC、计数器栈及限定收据	运行不重复、端点不守恒、原始 FAIL 被覆盖
瓶颈普查 (bottleneck census)	PERF_BASELINE 能力收据、选定的 CPI 诊断目标与计数器所有权	驻留层级、竞争解释、可证伪预测与区分实验目录	未发起 CPI 诊断任务，嵌套驻留被相加，或把相关性直接写成根因
因果量测切片	冻结 census、同身份 workload/ROI、观测器合同	owner 区间、occupancy、守恒式、A/B 原始日志与量测收据	探针改变基线、事件身份非法或收据不守恒
假设分析切片	合格且同身份的因果量测收据	唯一可检验假设，或显式 RESEARCH_REQUIRED	多个解释仍非支配，却主观挑选一个 RTL 方向
参考 PPA 资格	可选择的因果假设、同源 netlist、SDC、library 与 corner	可重复 P/A 与 timing 参考；不合格 power 继续标记未知	网表、路径集、约束或物理条件不可比较
可回退 RTL 实验	一个主要假设、参考 PPA、success/stop/rollback	有界 candidate、正负向验证及同身份性能/PPA 观测	影响锥静默扩大，或用中间切片申请 promotion
多目标处置与独立晋级	全部硬门、逐 workload floor、实测区间与完整设计点	接受、回滚、RESEARCH_REQUIRED、Pareto 位置及独立 front.py 裁决	timing 失败、区间重叠或 power 未资格化却被隐藏评分越过
分层系统再认证	同源 L0-L3 workload、终态与独立审查	独立系统收据；完整成熟度合取后形成 SYSTEM_RECERTIFIED/PROMOTABLE	把系统 PASS 写成 PPA PASS，或把 PPA 缺口写成系统 FAIL

7.6.1 架构发现、闭合与稳定

架构发现首先围绕待优化事务建立可验证的生产者—消费者图，并以该图限定随后需要读取的模块、接口和观测点。对于 memory stall 的 CPI 候选，分析依次回答：请求由哪个模块建立，reservation、bridge、AXI response 和 commit 各由哪个 owner 持有，flush 或 replay 能否清除它，以及哪个计数器能观察到每段生命周期。输出是带路径和未知项的债务表，而不是提前生成的 RTL 候选。

架构闭合逐项处理这些债务，并把历史系统缺陷下沉为低成本检查。若一项 Linux 故障可以由模块 TB 或断言稳定复现，后续 Agent 就不必用完整 Ubuntu 回放筛掉同

类候选；若只能在系统组合中出现，则保留在系统层而不伪造低层覆盖。架构门要求功能回归、双发射/乱序合同、精确恢复、异常与副作用同时闭合，并用能够通过编译的负向 RTL 检查判定器是否真的对目标错误敏感。只有 production RTL 与配置冻结后，才形成 ARCH_STABLE；它表示性能测量的架构前提稳定，不表示 PPA 已经合格。

7.6.2 量测合同与性能基线

正式测量首先冻结 workload 二进制、输入、编译器、起止 PC/marker、cache/TLB 初态、memory latency、seed、计数器版本、重复次数和阈值。基本量为

$$\text{CPI} = \frac{\text{测量窗口内的核心周期数}}{\text{测量窗口内退休的架构指令数}}, \quad \text{IPC} = \frac{1}{\text{CPI}}.$$

双发射核每周期有两个退休槽。令 C 为冻结 ROI 的核心周期数， $R > 0$ 为退休指令数， $l_s, l_e \in \{0, 1\}$ 分别为日志裁剪起止端点的 lane 编号。若裁剪协议把可见槽跨度定义为

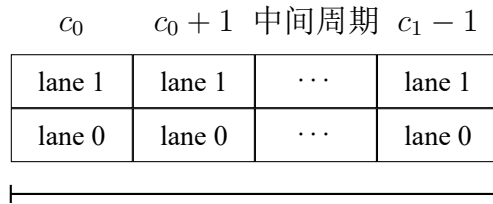
$$S = 2C + l_e - l_s,$$

且 $U_{\text{span}} = S - R$ 为该裁剪跨度内的未使用槽，则由 $2C = R + U_{\text{span}} + l_s - l_e$ 可得

$$\text{CPI} = \frac{C}{R} = \frac{1}{2} + \frac{U_{\text{span}} + l_s - l_e}{2R}.$$

因此，原先不含端点项的写法只有在 $l_s = l_e$ ，或把 $U_{\text{corr}} = U_{\text{span}} + l_s - l_e = 2C - R$ 明确定义为“端点修正后的未使用槽”时才成立：

$$\text{CPI} = \frac{1}{2} + \frac{U_{\text{corr}}}{2R}.$$



固定半开窗口 $[c_0, c_1)$ ： $C = c_1 - c_0$ 个周期，共 $2C$ 个退休槽
 事件端点裁剪从 (c_0, l_s) 到 (c_1, l_e) 时，可见槽跨度为
 $S = 2C + l_e - l_s$ ；端点差必须进入未使用槽修正。

图 7: 双发射退休槽、半开周期窗口与事件端点修正的关系

为避免 lane 被误解为半个物理周期，新的量测合同优先使用固定半开周期窗口 $[c_0, c_1)$ ，令 $C = c_1 - c_0$ 、 $U_{\text{full}} = 2C - R$ ；此时上式直接成立且不再需要端点 lane。本文把同一历史性能基线的两次原始独立执行简称为 A1/A2；两者 task-run 均保持 FAIL。

A1/A2 不是算法版本或验收等级，端点规则修订只属于后续 versioned replay，不会追改 A1/A2。两份原始日志仍按当时的裁剪协议保留；新收据则必须显式保存 $l_s, l_e, U_{\text{span}}$ 与 U_{corr} ，不能混用。

历史 performance-baseline 任务中的 A1/A2 曾因 Dhrystone 的 ROI 从 lane1 结束、在 lane0 重新开始而被旧判定器拒绝；后续版本化勘误只替换错误的“起止 lane 必须相等”代理条件，并保留原始 FAIL 与冻结 replay。该事件用于修订量测合同，却不再充当冻结设计基线。当前冻结设计的 PERF_BASELINE 采用 FRESH_CURRENT_DESIGN_DIRECT 模式重新执行：CoreMark 与 Dhrystone 各三次统计开启运行 bit-exact，统计关闭运行的 cycles/retired 又与开启时相同。由此，该收据既继承了端点修正规则，又绑定新的 design-id、simulator、workload 与 manifest；它证明量测接口可重复且统计逻辑没有观察到干扰，不证明 CPI 已经因某次 RTL 修改而改善。

7.6.3 瓶颈普查、因果量测与 Selector 实例

性能基线形成后，证据面先发布能力收据，声明当前设计能够在冻结 workload/ROI 上重复量测 cycles、retired 与计数器，同时明确该收据不授权任何优化方向。控制面随后若选择 CPI 诊断任务，Agent 才针对该任务建立 lost-slot census（退休槽损失普查）；若目标是功能正确性或 PPA 资格，则分别进入验证或实现支路，并不要求先生成 census。CPI 诊断中，总 cycles、unused retire slots、I/D-cache miss、控制等待、LSU 等待、AXI 等待和事务 owner 驻留分别记录；这些计数可能在同一周期重叠，不能简单相加为总周期。该 census 在 CoreMark 与 Dhrystone 上均给出 memory_latency $\rightarrow$ request_outstanding $\rightarrow$ axi_write_response 的驻留层级，但同时保持 HYPOTHESIS_NOT_PROVEN 与 optimization_candidate_authorized=false。换言之，驻留排序只决定应观测哪个事务区间，不能直接授权“增加 buffer”或“隐藏 B response”等 RTL 方案。

某次只读 owner 探针完成了六次 benchmark，却产生 complete=0 与非零 invalid 事件，因而在 receipt-build 阶段失败。该反例促使工作流把 owner 身份、区间开始/结束、直方图守恒、observer noninterference 和清理状态纳入同一个量测合同。后续切片不再复用该结论，而是对 CoreMark10 与 Dhrystone10000 分别执行三次参考构建和三次诊断构建。owner-timing 输入 manifest 的 148 个条目在前后保持一致，且 production_filelist_modified=false；同一冻结身份的 canonical production RTL inventory 为 146 个文件。两者属于不同清单口径，不能把差值解释为新增或删减 RTL。统计开关不改变 cycles/retired，owner 区间与状态计数守恒，非法事件和溢出均为 0，运行时产物完成清理。

表 16: 冻结设计 owner-timing 中等粒度量测切片的主要观测

观测	CoreMark10	Dhrystone10000	解释边界
CPI / retired	1.653110 / 3,183,617	2.294462 / 4,250,000	同一冻结 PERF_BASELINE 的描述值，不是候选收益
AW/W→B 完成数	144,201	600,000	区间完整且无 right-censored 事件
B response 平均区间 /	4.000 cycle /	4.000 cycle /	强相关本身不等于生产 RTL 中的唯一因果机制
ROB-head 比例	1.000	1.000	
双写 in-flight 比例	0	0	两个 workload 未观察到双写并发；不单独证明应扩容
reservation 平均区间 /	1.850 / 0.414	2.013 / 0.255	给 H4 提供竞争解释，不能与 H1 驻留相加
ROB-head 比例			
三次重复与观察器非 干扰	bit-exact / true	bit-exact / true	只授予量测可消费性，不授予因果和 PPA 资格

量测 runner 因而以 PASS 结束，并生成 OWNER_TIMING_WORKLOAD_MEASURED 收据；但代码 20 中的权限字段仍全部为 false 或 UNQUALIFIED。这只是 owner A/B 节点的终态：测量动作已经成功，H1–H4 获得了可比较观测，后续节点仍需用受控干预区分竞争解释。

代码 20: 初始 owner-timing A/B 收据的授权边界

```

1 "status": "OWNER_TIMING_WORKLOAD_MEASURED",
2 "authorization": {
3   "hypothesis_selection_authorized": false,
4   "optimization_candidate_authorized": false,
5   "ppa": "UNQUALIFIED",
6   "promotion_eligible": false
7 },
8 "unknowns": [
9   "owner correlation does not by itself prove a causal RTL mechanism",
10  "frequency, area and power were not measured in this run"
11 ]

```

初始因果分析仍将 H1（B-response 延迟）与 H2（写并发）判为耦合，并返回 RESEARCH_REQUIRED。随后，诊断构建仅受控增加两个 B-response 周期：CoreMark 的 cycles 从 5,262,868 增至 5,502,804，增加 239,936；Dhrystone 从 9,751,462 增至 10,951,411，增加 1,199,949；两者 retired 均保持不变，且每个点三次 bit-exact。该干预把结果限定为 H1_B_RESPONSE_LATENCY_SENSITIVE，从而授权 H1 作为下一分析方向；它是诊断延迟注入，不是 production RTL 性能候选，也没有证明 H1 是唯一根因。因而收据仍将

因果状态记为 HYPOTHESIS_NOT_PROVEN, 并把 production_rtl_change_authorized、optimization_candidate_authorized 与 promotion_eligible 全部置为 false。

证据接入后, canonical Selector 已无 stale reference, 并形成 decision=SELECT; 其选择对象是 analyze.current-timing-recovery-candidate, 即聚类 5 ns 违例路径并定义一个可回退时序候选或明确 GAP。该动作只允许读取并归类 current path family, 明确不修改 production RTL。当前参考实现量测可重复, 但 5 ns timing hard gate 失败, 故 ppa_reference_available=false、ppa_qualified=false。早期双 memory 发射实验发生在多个关键模块尚未满足目标时序的阶段, 其 CPI 数字不作为这条冻结证据链的基线或候选比较。

7.6.4 PPA 资格、分层系统证据与可交付边界

联合资格按固定全序裁决: 同源设计与完成定义、功能、双发射/乱序、精确恢复与副作用、精确 timing tier、逐 workload floor、面积/功耗资格、Pareto 前沿, 最后才执行晋级检查。Timing 在这里是硬门: 确定低于门槛的候选直接拒绝, slack 区间跨越门槛时先补 STA, 不能把频率或 slack 与 CPI、面积压成同一个补偿分数。通过硬门后, Pareto 前沿才比较性能、面积效率和已资格化的功耗效率; 所谓前沿, 是不存在另一个合格设计点在这三项上均不差且至少一项确定更好。平均 CPI、generic cell 数或一个隐藏加权评分都不能抵消前面的硬门。

物理资格至少要求同源 netlist、SDC、library/corner、完整 I/O delay、真实 memory macro、有效切换活动、时钟树/寄生与对应 timing 报告。所记录的 Yosys/OpenSTA 结果能够用于结构筛选, 却因 I/O 未约束、缺少 SPEF、CTS、OCV、真实宏和功耗活动而保持 UNQUALIFIED。这种非晋级状态不是“尚未整理好的成功”, 而是一个可消费的工程结论: 允许继续定位面积或逻辑深度, 不允许宣称频率、面积、功耗签核, 也不允许进入 PPA promotion。

系统证据与 PPA 资格是独立坐标。L0 module/transaction、L1 完整核、L2 mini-system 与 L3 轻量 Linux 可以在同一 RTL identity 上形成分层系统收据; 它们证明相应软件和设备事务, 并不把代理综合变成 PPA_QUALIFIED。反过来, PPA 保持 UNQUALIFIED 也不抹去已经闭合的 L0-L3 功能事实, 只阻止成熟度越级到 PPA/PROMOTABLE。完整 Ubuntu 22.04 是需要用户显式授权的可选低成本再认证, 不是每个切片或系统层失败后自动触发的默认门。

表 17: 现有证据对 CPI/PPA 工作流的支持范围

已观察事件	若无分层门禁可能得到的错误结论	该工作流实际形成的结论及其意义
A1/A2 旧判定器拒绝、冻结重放通过	删除 FAIL，只保留新的绿色基线	原始 FAIL 与新收据并存；证明判定器版本可修订但历史事实不可改写
三次重复与统计开关非干扰	一次 CPI 数字即可比较候选	只授予可重复量测基线；为后续同身份 A/B 提供共同分母
历史 owner 事件无效；冻结 A/B 与 +2-cycle 干预可重复	workload 完成或敏感性成立即证明生产根因	旧量测失败关闭；新量测授予 MEASURED，干预只选择 H1 分析方向，production RTL、PPA 与晋级仍关闭
历史 Selector verifier 哈希漂移	单元测试通过即可继续消费旧决策	canonical verify 曾拒绝存量决策；新决策消除 stale reference，但只授权下一项路径聚类动作
双 memory CPI 退化	局部回归通过即可保留“优化”	候选回滚并保留负实验；后续 Agent 不再重复同一搜索方向
代理综合和时序数字可读	直接写成目标频率、面积或功耗改善	保持 PPA 未资格化；把缺失物理条件转化为后续输入而非夸大结论

因此，本节证据支持的是“工作流能把量测错误、可重复观测、受控敏感性、负优化、版本漂移和物理条件不足导向不同终态”，并形成下一实验所需的输入；它尚不支持“B-response 已被证明为 production RTL 中的唯一根因”“该方法已经带来稳定 CPI 提升”或“冻结核已经完成物理 PPA 签核”。这种区分使性能工程的有效产物不仅包括更优设计，也包括被可靠淘汰的候选、被替换的过期决策和不能晋级的边界。

8 案例研究设计与证据协议

前述章节给出了方法及其实现接口；在进入工程里程碑之前，本章先固定“哪些材料可以作为证据、怎样选择和编码主张、冲突怎样处理”。这样，后文的时间顺序只用于过程追踪，而不会被直接解释为方法产生效果的因果顺序。本文采用单仓库纵向嵌入式案例研究，并借鉴软件工程案例研究和过程追踪对案例边界、分析单位与诊断性证据的要求 [33, 34]。

8.1 案例边界与分析单位

案例材料来自 `ysyx-workbench` 近四个月的持续开发流程及其留存日志；分析单位不是目录、文件或一次聊天，而是任务 `episode` 中可独立判真的工程主张。一个 `episode` 具有明确目标、待判断对象、可恢复验收条件，以及 `accept`、`reject`、`rollback`、

GAP、UNQUALIFIED、interrupted 或 MISSING 中至少一个终态。版本化源码、合同、task-run 状态、原始日志、结构化收据和证据身份进入分析；每项观察只在其绑定的设计、配置、workload 与判定器范围内成立，只存在于未归档对话中的推测不进入证据集。

8.2 主张选择、编码与冲突处理

本文所称“30条目的性主张”，是采用目的性抽样（purposive sampling）形成的30个可独立判真的论断，不是30项完成成果，也不是从全部任务中随机抽出的成功样本。“目的性”表示研究者主动让方法机制、工程压力、正反结果和证据状态都进入审计集，而不是预先挑选有利结论。候选来自近四个月的任务终态、里程碑报告、失败记录和机器收据；重复发布合并为一个 task episode，task episode 即具有同一目标、待判断对象、验收条件和终态的一次连续工程事件。早期候选全集及逐条排除日志没有完整保留，因此30条只能用于逐项证据审计，不能估计仓库中所有任务的成功率或覆盖率。

每条主张统一规范为“工程对象—实验身份—验收谓词—结论—边界”五元组。例如，C29的对象是冻结 RV64 核上的 owner-timing 量测，身份是同一 design/workload/oracle，验收谓词包括三次重复、retired 守恒和探针非干扰，结论是“B-response 延迟敏感”，边界则是“不授权 production RTL、PPA 或晋级”。若缺少最后一项边界，同一观测就容易被误写成“已经找到唯一根因”。

选择时同时覆盖 RQ1（领域能力构成）、RQ2（目标到任务）、RQ3（设计—验证闭环）、RQ4（能力演进）和 RQ5（工程产物与边界），并对六类已登记失配作最大差异选择：功能尚不支持、局部结果越级为系统结论、旧身份或跨设计串证、判定器盲区、未资格化代理，以及发布不完整。纳入项必须可被反驳、有精确来源路径、能恢复至少两类身份/执行信息，并具有明确的接受、拒绝、回滚或未资格边界；纯进度文字、配置不可比的数字和无法恢复决策边界的临时日志被排除。表 18 按分析用途归纳全部 30 个编号，并给出能够直接看懂其内容的代表例。

表 18: 30 条目的性主张的组成与代表例

分析用途	主张编号	代表主张的直白含义	为什么进入审计集
环境与方法演进	C01–C05	C02 提问“反馈闭环是否因果上优于单次 Prompt”；当前缺少同模型、同任务和同预算对照，故为 MISSING	防止把纵向共现或工具存在直接写成方法优势；主要限定 RQ1、RQ4

分析用途	主张编号	代表主张的直白含义	为什么进入审计集
身份、验证与回滚	C06–C13	C09 记录局部 FENCE 检查通过但全核/PPA 仍保持非晋级；C11 则把 “mutation 全过即可证明 RTL 正确” 编码为待反驳强命题	同时保留限定 PASS、身份冲突、回滚和判定器盲区；覆盖 RQ2–RQ4
综合、时序与性能可比性	C14–C17、C20	C14 只声称代理 STA 在给定约束下得到正裕量；C15 的 “因此已完成 200 MHz 物理签核” 被同一证据直接反驳	展示同一原始结果怎样支持弱主张却拒绝越级强主张；限定 RQ5
系统软件里程碑	C18、C19、C22、C23、C26	C22 支持一次可回查的 Ubuntu 系统事务；C23 拒绝把冻结 replay 改写为新的 live PASS 或架构/PPA 晋级	区分 OpenSBI 交接、rootfs、用户态、自然关机和发布资格；覆盖 RQ3、RQ5
模块与工程形态演进	C21、C24、C25	C24 说明 RV32 SoC 的 AM 地址图、NPC target 与 NEMU reference 如何形成 39/39 DiffTest 闭环	使 “从 single 到 SoC、从 RV32 到 RV64” 落到具体对象而非时间口号；覆盖 RQ3、RQ5
当前 CPI/PPA 入口	C27–C30	C29 支持 B-response 延迟敏感性但关闭生产 RTL 权限；C30 保存前一轮非法 owner 量测被拒绝的原始 FAIL	同时展示合格基线、因果诊断、下一动作选择和失败关闭；覆盖 RQ2、RQ3、RQ5

机器可读 codebook 把字段分为五组：主张与分类，episode 与身份，来源与执行，结果与反例，以及外推边界。证据状态评价的是 “这句话由现有材料支持到什么程度”，不是任务运行成功率：

- **SUPPORTED**：对象、来源和边界共同支持原句；如 C10 仅声称 23 个指定的 compile-success 负向 RTL 被对应判定器拒绝，不外推到其它故障。
- **PARTIALLY_SUPPORTED**：机制或结果存在，但强度、分母、身份或因果归因不完整；如 C07 只证明同一发布内存在身份元组冲突，不能恢复权威元组。
- **OBSERVATIONAL_ONLY**：只支持共现或纵向变化；如 C01 只能观察到 workflow 构件随工程压力共同演进，不能证明单一构件具有因果作用。
- **CONTRADICTED**：原始证据直接拒绝强表述；如 C15 的代理 STA 不足以证明 200 MHz 物理签核。这类反驳是边界检查结果，不等于一次系统运行失败。
- **MISSING**：现有材料不能回答该主张；如 C02 尚无固定模型、任务和预算的对照实验。

只有在 design、configuration、workload 与 oracle 适用域相同而结论不同时，才构成证据冲突；新判定器可以生成版本化 replay，但必须保留原始状态。完整 30 行及其来源、数字、反例、回滚和外推边界位于机器可读主张表，正文的代表例不替代逐行记录。

第二 Agent 对预先固定的 12/30 主张执行隔离、只读的反例复核，检查状态、原始路径、数字、返回码、回滚和声明边界。复核者能够看到主编码，因而这是独立反例审查而非盲法双编码；初审阻断并修订了 C01（早期阶段描述过强）、C07（同一发布内身份冲突）和 C30（原始路径覆盖不足）。三条记录的修订轨迹保留在独立复核表中，最终状态仍纳入 30 条审计集。样本规模和非盲条件决定本文不报告人类编码一致性指标 [35]。

8.3 数据保留与可复核入口

公开仓库为 mcsaver/oscpu，机器可读主张表、选择协议、独立复核和构建入口分别由 claim-evidence-map.tsv、case-selection-protocol.md、independent-claim-review.tsv 与 paper-reproduction.md 承担，完整相对路径和 schema 见附录 C。可版本控制的合同、状态、收据、摘要、索引和哈希直接保留；重型日志或波形由 manifest 记录路径、尺寸、SHA-256、时间和 marker。原始对象丢失时登记 MISSING，受访问限制时登记 RESTRICTED，二者均不由摘要替代。

早期 episode 缺少统一的模型快照、seed、采样参数、完整系统提示、工具序列、上下文截断状态和部分重型原始对象。这些记录仍可说明接口如何演进，但不进入模型级复现、效率比较或严格总体率的分母。由此固定证据协议后，下一章才沿验证压力追踪方法如何从 RV32 局部运行扩展到 RV64 系统事务与 OoO 性能资格化。

9 Loop-Work 的工程实践与能力里程碑

9.1 验证压力与环境能力的共同演进

本文所称“一生一芯 C 阶段至 B 阶段”沿用项目课程记录中的阶段标签，用于界定这段连续实践的范围；论文中的 ARCH_STABLE、PERF_BASELINE 和 PPA 状态另由机器合同定义，二者不能互相替代。

早期把“环境建设”和“芯片开发”混在同一条非结构化对话中，会产生三个直接问题：修改 RTL 后无法判断旧日志是否仍属于当前设计；系统启动现象容易被概括为超过证据层级的结论；反复出现的调试经验仍停留在个人上下文，下一任务只能重新发现。Loop-Work 将二者通过合同接口分开：芯片节点只负责形成当前设计的候选和业务证据，环境节点只负责把已验证且可迁移的规则、Skill、执行入口和证据协议版本化；两者通过有类型的输入输出相连，而不共享模糊的“完成”状态。

这种拆分没有形成两条互不相关的路线。每当处理器任务暴露稳定失效模式，外层闭环就增加一个可复用环境能力；新能力随后缩小下一类任务的可执行范围，并提高其证据要求。表 19 按验证压力列出这种共同演进。它支持“同一接口在更强任务上持续形成可追踪产物”的过程结论，不构成某个环境构件单独提升生产率的因果实验。

Design Conductor 报告的 12 小时 RV32I+Zmmul spec-to-GDSII 案例说明长时间自治已经能够贯穿 RTL、验证、时序优化与后端工具 [25]。本文不把该结果与本仓库的 RV64 OoO 或 Yosys 代理 PPA 作速度、频率或质量对比：两者的 ISA 范围、微架构、工艺、工具与 oracle 均不相同。它在这里承担的作用只是界定外部能力上界；表 19 仍按本案例实际经历的验证压力记录可声明范围，不能用外部端到端成功替代本地系统长跑、身份一致性或物理资格。

附录表 38 进一步追踪方法构件为何进入系统。分析单位不是文件首次出现的时间，而是“哪种失效反复阻断交付、为此在四面之间增加了什么接口、该接口后来被什么任务复用”。这避免把 Git 时间戳误写成因果证据，也使方法演进能够脱离当前目录结构进行复现。

正文保留三条承重过程链：NEMU/Ubuntu 的多节点压力推动“profile 闭包 → 原子证据包”；RV64 跨模块任务推动“架构事实 → 机器合同 → 隔离复核”；十九小时长跑推动“运行状态 → 证据完成位/信号/清理合取 → 限定终态”。它们分别扩展了调度、作用域控制和失败关闭能力；其余构件的触发压力与上下游位置见附录表。

以下里程碑只追踪“工程压力—接口增量—后续调用”的过程链；环境构件的独立生产率效应留给受控实验评价。

9.2 从静态候选到局部动态语义闭合

在实现 RV32I 的 I/S/B/U/J 译码、ALU 与分支比较、LB/LH/LW 和 SB/SH/SW、唯一写回点以及 redirect 后旧路径抑制时，最初冻结记录包含一个非流水线 NpcCore 与 8 个辅助 RTL 文件。Agent 能较快产生这些局部结构并通过 Verilator lint，但当时尚无 testbench、程序镜像、存储模型、退出协议和参考对拍，因而结果最多说明“语法与静态结构成立”。随后的任务不是继续增加 RTL 行数，而是把 AM hello image、DPI/C++、pmem、串口和退出码连接为运行链，并继续增加 itrace、mtrace、dtrace 与批量重放。Agent 的介入由“一次生成候选”变为“分析现有通路—生成—运行—观察—修正”，工程师则开始把原本隐含的完成定义写成可执行接口。

该组证据的有效范围是最小动态闭环，而非完整 ISA、异常或 DiffTest；其方法意义在于把 lint 之后仍缺少的运行接口显式转化为下一轮输入。

相关记录已经出现有界历史检索、子 Agent、独立复核与任务档案等协作结构，但这些结构尚未同 AM image 构建、target/reference、动态 oracle 和可执行完成门连接

表 19: 工程压力、AI 环境演化与可声明里程碑

工程对象与验证压力	暴露的工程盲区	环境标准化与可执行产物	最高可声明范围
RV32 single	lint 无法回答“能否运行”	rv32-bringup 图把 AM image、pmem、DPI、trace、exit 串成动态反馈	给定镜像的最小动态闭环
RV32 SoC	地址图、设备与 target/reference 身份混淆	npc/sim backend 状态、SoC profile、NEMU reference 与 DiffTest 接线	给定测试集上的 SoC 仿真一致性
RV64 target/reference	宽度、CSR、特权和配置变化使旧证据失效	task contract 绑定 RTL/spec/TB/config; 40/40 与 CoreMark DiffTest 分层留证	指定配置的 target/reference 一致性
OpenSBI/Linux/Ubuntu	固件 handoff、rootfs mount、shell 与关机容易被同称为“启动”	Linux 证据八层、virtio/rootfs 合同、long-run status 与 frozen replay	指定 design-id 的系统事务
OoO architecture	owner、恢复、内存序跨模块，局部 PASS 易越级	interface-first、RTL 拓扑、debt ledger、compile-success mutation、独立 reviewer	指定 cohort 的 ARCH_STABLE
CPI/PPA	counter 可干扰、嵌套等待可重复计数、proxy 易冒充 signoff	量测合同、退休槽损失普查 (lost-slot census)、同源 PPA 身份与资格状态	当前仅 PERF_BASELINE; PPA 未合格

成统一数据流。人的主要工作仍包括解释运行输出和决定能否接受；Agent 虽能生成、复核并参与短反馈，却不能依据统一合同独立形成动态语义收据。正是 lint PASS 与运行闭环之间的落差，促使后续把 image build、reference、target、compare 和 record 固化为图节点，并把“没有动态证据就不得宣称功能成立”写成 instruction。第一项环境扩展扩大的是执行与解释最小 CPU 事务的闭合能力，而不是从零开始建立所有 Agent 协作结构。

这一里程碑的证据上限是给定 hello 镜像的最小运行语义；完整 ISA、异常、DiffTest 和 RV64 功能由后续层级继续验收。

9.3 target/reference 闭合：从 RV32 single 到 RV64 SoC

随后，npc/sim 被设计为统一入口，按配置分流 npc/single 与 npc/soc。SoC 数据路径为：

代码 21: RV32 SoC 的 target/reference 数据流

```
am-kernels -> AbstractMachine(riscv32-ysyxsoc image)
-> npc/sim(BACKEND=soc) -> npc/soc -> ysyxSoCFull
                        \-> NEMU(CONFIG_SOC_SIM) reference
-> commit-level DiffTest
```

SoC 接通又暴露出另一类环境问题：同一个“运行测试”动作可能在 NEMU reference、NPC single、NPC SoC 上具有完全不同的地址图和结论。这个经验后来进入 E2E 场景隔离规则：nemu-dev* 展开后只允许 NEMU/software-flow 节点，npc-dev 只允许 NPC/software-flow 节点，而显式 integrated profile 保留跨栈闭环；--validate-profile 与真实 dispatch 共用边界检查。这样，profile 不是把模块分开管理，而是防止一个平台的 PASS 在数据流中被误送给另一个平台。

MROM 入口为 0x20000000，SRAM 为 0x0f000000..0x0f001fff；cpu-tests 形成 39/39 DiffTest，char-test 观察 UART，mem-test 在 PC=0x200002c6 到达 GOOD TRAP。这里的关键产物不是另一个 CPU 文件，而是同一镜像在 target 与 reference 两侧可比较。

随后 NPC 先扩展 PC/GPR/CSR/AXI/DPI 到 RV64，关闭尚未资格化的 cache 与 DiffTest，得到 38/38。NEMU 补齐 RV64 CPU state、CSR 与 RV64I/M/B/C/W reference 后，形成 40/40 和 CoreMark DiffTest。这两个冻结点必须区分：38/38 说明 NPC 在给定自测中运行，40/40 才增加 target/reference 一致性证据。

这些数字是演进中的冻结点，不是能力上限。随着整数、浮点、原子、压缩指令、特权态、Sv39 和乱序提交路径继续接入，AM 测试从少量 RV64 定向程序扩展为自动发现且逐项留证的 61 项 inventory，official riscv-tests 扩展为 177 项，模块测试扩展为 113 项；同一冻结 design-id 下三组 inventory 均全部通过，DiffTest mismatch 为 0。测

试增长与 RTL 增长被同一合同约束：新增架构语义若现有 workload 不能激活，就先增加 AM 程序或定向 TB；新增 oracle 又必须用可编译负向 RTL 检查其拒绝能力。因此，从 40/40 到分层合取并非简单增加测试数量，而是 Agent 同时扩展参考语义、工作负载和 RTL 观测的结果。

从 32 位扩展到 64 位还改变了 Agent 的上下文组织方式。PC/GPR/CSR/AXI/DPI 不再适合由一个宽泛任务同时修改，局部 reviewer 也不能只读主 Agent 的摘要。为此，后来的 RTL task Skill 规定把 RV64 子任务拆为 read-only-review、implementation、verification 与 ppa-analysis 四类，并要求配套生成器形成显式列出 RTL、spec、TB、evidence、写路径和命令，并让子 Agent 在隔离上下文中工作。它没有减少源码探索；相反，它使多 Agent 能够并行复核不同债务项，同时保证每个结论仍能追溯到具体 module/signal/transaction 与测试配置。

39/39 与 40/40 的证据范围分别绑定各自冻结测试集和配置：SoC 全量、Linux、FPGA、物理芯片与跨位宽性能属于更高层验收。

9.4 系统级闭合：NEMU、NPC 与十九小时 Ubuntu 事务

系统级研究先以 OpenSBI v1.8、内嵌 DTB 和 S-mode mini payload 形成固件交接；mini payload 的“S”与 GOOD TRAP 只证明 M-mode 到 S-mode handoff，不等于 Linux。随后，NEMU reference 先进入 Ubuntu 22.04.5 的非交互 shell，NPC 则到达 2 GiB virtio block、EXT4 rootfs、systemd PID1 和 Ubuntu banner。

如果没有证据分层，以上现象很容易被统一写成“Ubuntu 已启动”。.github/instructions/rv64-linux-bringup.instructions.md 因而把路径拆成 env-built、QEMU reference、OpenSBI handoff、kernel progress、init executed、Ubuntu probe、initramfs shell 与真实 rootfs 八级；.github/instructions/virtio-rootfs.instructions.md 又要求 virtio-mmio register、vring descriptor/avail/used、host block backend、UART/virtio 多源 PLIC、DTB、kernel config、/dev/vda 和 mount 证据同时闭合。rootfs 模板必须按内容寻址复制为 run-owned 工作副本，模板前后 SHA 保持不变，guest 合法写入只能改变副本。instruction 在这里直接改变了日志的解释方式：NEMU shell 与 NPC banner 被保留为不同层级，而不是由 Agent 用更顺耳的自然语言合并。

表 20 展示该 instruction 的承重内容。八级不是八条近义描述，而是八个不同验收谓词；上一级的产物只能作为下一级输入，不能自动继承更强结论。例如，QEMU 通过只证明镜像和启动参数存在可用参考，不证明 NPC target；rootfs 构建完成也不证明 guest 已挂载并执行它；使用 RV64IMAC/LP64 的探针更不能替代正式 RV64GC/LP64D 用户态。

表 20: RV64 Linux bring-up instruction 的八级证据与结论边界

级别	状态名称	必须观察到的对象	仍不能声称
1	环境构建完成	固件、内核、DTB、initramfs/rootfs 的构建身份与哈希	任一模拟器已经执行镜像
2	QEMU 参考通过	同一镜像在 QEMU 上达到约定终点	NEMU/NPC target 已通过
3	OpenSBI 交接	固件启动并进入 S-mode payload 或内核入口	Linux 内核已经持续运行
4	内核推进	Linux 解压、设备枚举或内核阶段 marker 持续前进	init 或 Ubuntu 用户态已执行
5	init 已执行	指定 /init 或 PID1 的可归属输出	正式 Ubuntu rootfs 已挂载
6	Ubuntu 探针可见	Ubuntu 用户态文件、动态链接器或版本探针命中	已获得可交互 shell
7	initramfs shell	shell 命令成功且输出/退出码可核验	真实块设备 rootfs 与持久写已闭合
8	Ubuntu rootfs	/dev/vda、EXT4 mount、systemd/服务、读写回验和终止事务	未覆盖配置、设备、服务和长期稳定性均正确

代码 22: NEMU 与 NPC 两个平台的分层日志节选

```
[NEMU]
virtio_blk virtio0: [vda] 4194304 512-byte logical blocks
VFS: Mounted root (ext4 filesystem) on device 254:0
PRETTY_NAME="Ubuntu 22.04.5 LTS"
[ysyx-rootfs] /bin/sh -c exit=0
[ysyx-rootfs] launching /bin/sh

[NPC]
virtio_blk virtio0: [vda] 4194304 512-byte logical blocks
VFS: Mounted root (ext4 filesystem) on device 254:0
Run /lib/systemd/systemd as init process
systemd 249.11-0ubuntu3.21 running in system mode
Welcome to Ubuntu 22.04.5 LTS
Hostname set to <ysyx-ubuntu2204>
```

NEMU 的里程碑是 shell；该次 NPC 的里程碑是 rootfs mount 与 systemd/banner，二者不能合并为同一结论。随后，NPC A3（第三份、也是首份达到完整终态的冻结系统事务证据）把指定 design-id 推进到：

A3 的 host 流水线墙钟约 19 h 14 min 43 s，simulator 自报约 18 h 48 min 22 s；guest 在 49.720532 s 虚拟时间发起 power down。它完成了 OpenSBI、Linux、2 GiB EXT4、Ubuntu systemd、rootfs 写入/同步/回读、virtio direct read/IRQ 增长、有序关机与 clean exit。旧 checker 却把 benign 的 printk: debug: 中 bug: 误识别为 BUG:，所以 source 状态保持 FAIL rc=1，strict 16/17。修正后的 oracle 对冻结 console 重放为 PASS，

代码 23: NPC A3 Ubuntu 22.04 系统事务的关键 marker

```
OpenSBI v1.8
Linux version 6.6.0
virtio_blk virtio0: [vda] ... 2.00 GiB
VFS: Mounted root (ext4 filesystem) on device 254:0
systemd 249.11-0ubuntu3.21 running in system mode
__NPC_CHECK_PASS__:rootfs-write-sync-readback
reboot: Power down
npc: HIT GOOD TRAP
exit via system-reset, code=0,
    cycles=5071521696, commits=1223536213
CPI (cycles/instruction) = 4.145
```

但不改写原始 FAIL，也不授权新 RTL、architecture 或 PPA 晋级。这是 Loop-Work 中“DUT 终态”和“oracle 状态”分离的典型产物。

十九小时运行也反向改变了 AI 环境。短任务可以把进程退出近似看作完成，系统长跑却必须区分 execution/terminal/artifact/assertion/oracle/replay 六种状态；否则一次 HUP、前台 timeout 或 checker 修改都可能把不完整任务留成绿色。由此形成的 task-run-status.sh 显式维护 evidence-complete latch、stage、signal 与 cleanup，agent-system 的 fail-closed 节点再用 early exit、cleanup failure 和 signal 定向测试守住该实现。这项环境能力随后不只服务 A3，也成为所有长仿真、综合、STA 与系统回放的公共交付接口。

NEMU shell、NPC banner 与 A3 系统事务分别属于不同证据层；A3 replay 只重新解释同一冻结观测，设备覆盖、长期稳定、架构晋级和 PPA 资格仍由各自门禁决定。

9.5 跨模块事务合同：排空、FENCE 与 memory owner

OooPendingDrainResolveGate.v 的三个不可变快照展示了工程合同如何从隐含条件变成可验证 owner 边界。

这一演化由两层环境共同约束。接口 instruction 先要求回答“谁产生 stop、谁保持 outstanding、哪类 flush 可以清什么、已发 AXI 和 committed store 是否仍有 owner”；RTL instruction 再要求把需求、状态机、不变量、数据通路与时拍优先级写清。具体复核节点则由 task Skill 限定到该 gate、CoreGlue/ControlPlane 连接、相关 testbench 与证据路径，并保留 scope extension。于是 Agent 不是看到一个布尔表达式后继续叠加条件，而是必须解释每个条件对应的 transaction lifecycle 与反例。

代码 24: 快照 A: 基本后端排空

```
assign backend_drained_o = (rob_count_i == 0) &&
    (issue_count_i == 0) &&
    (core_retire_count_i == 0) &&
    !synth_lane1_ret_pending_i &&
    !synth_lane1_branch_drop_pending_i;
```

快照 A 将 ROB、issue queue、当拍退休和 synthetic lane1 pending 合取为 drain; 它没有显式表示退休 store 仍驻留 SQ 的情况, 也没有完整 memory owner 静默条件。快照 B 增加 mem_retire_quiet_i, 并根据合法状态域证明 rob_count==0 蕴含 retire_count==0, 删除冗余条件:

代码 25: 快照 B: 退休/SQ 静默与冗余条件消除

```
// commit0_fire -> count_q != 0
// commit1_fire -> commit0_fire
// therefore rob_count == 0 -> retire_count == 0
assign backend_drained_o = (rob_count_i == 0) &&
    (issue_count_i == 0) &&
    !synth_lane1_ret_pending_i &&
    !synth_lane1_branch_drop_pending_i &&
    mem_retire_quiet_i;
```

该切片对 16-entry ROB 的有限二值合法域枚举 2176 个用例。枚举器不是把随机输入跑 2176 次, 而是先用 commit1_fire -> commit0_fire、commit0_fire -> count_q!=0 和 commit valid=commit fire 排除不可达组合, 再比较删除前后谓词。附录表 39 给出其中五类代表状态; 正文只保留其语义边界: ROB 空、head 阻塞、单退休、双退休和“lane1 独立退休”这一被排除的非法组合。

在这个声明的有限域内, 原谓词与删除 core_retire_count_i 后的谓词逐例等价; 删除 count guard 和在顶层增加 || 1'b1 的两个故障变体则被拒绝。因此证据支持“该项在枚举域内冗余且检查器对两种旁路敏感”, 不等于完整形式化验证, 也不覆盖 ROB 数据域、所有时序轨迹或全核状态空间。快照 C 再为 ordinary FENCE 增加完整 memory owner graph 静默, 但不把相同条件施加到所有 system event:

代码 26: 快照 C: ordinary FENCE 等待完整 memory owner 静默

```
wire pending_fence_mem_quiet_w =
    !pending_system_fence_i || mem_idle_i;

assign drain_complete_o =
    stop_pending_i && backend_drained_o &&
    pending_control_ready_i && !pending_replay_wait_o &&
    pending_fence_mem_quiet_w;
```

两个可编译负向 RTL 分别删除 FENCE full-memory-idle 谓词和常量化 CoreGlue 到 ControlPlane 的 mem_idle 连接, 均被唯一目标 marker 拒绝; 正向 focused 2/2、module 109/109、validator 12/12。项目内部状态记为 FENCE-G1=CLOSED, 只表示上述局部合同、用例与两个目标 fault model 形成范围限定的回归闭合; 历史 full-core 仍为 GAP、PPA 为 UNQUALIFIED。三快照之间同时存在其他 RTL 变化, 因此它们证明合同显式化的过程, 不构成单变量性能实验。

代码 27 保留了两个变体的关键 shell 转录。二者都先通过 Verilog 编译, 随后分别在“MIQ/bridge reservation 尚未空闲”和“CoreGlue 连接不再传递真实 mem_idle”的定向场景中被拒绝, 外层 make 均返回 2。由此可以排除“变体只是语法错误”的替代解释; 唯一失败 marker 又把拒绝归因到目标合同, 而不是无关回归。它仍只证明这两个 fault model 可见。

代码 27: 两个编译成功的 FENCE 负向 RTL 及其动态拒绝记录 (删节)

```
[fence_full_memory_idle_removed]
[COMPILE] iverilog ... tb_ooo_pending_drain_resolve_gate
[CHECK-FAIL] fence waits for MIQ bridge reservation idle got=1 expected=0
[FAIL] tb_ooo_pending_drain_resolve_gate errors=1
[RESULT] FAIL status=1

[core_glue_fence_mem_idle_binding_constantized]
[COMPILE] iverilog ... tb_ooo_priv_system
[CHECK-FAIL] fence control plane consumes core memory idle got=0 expected=1
[FENCE-G1-PROGRAM] ... mem_idle_binding=0 ... FAIL
[FAIL] tb_ooo_priv_system errors=1
[RESULT] FAIL status=1
```

2176 个合法域用例与两个 FENCE mutant 覆盖声明的有限状态域和 fault model; FENCE-G1=CLOSED 是项目内部的局部门状态和 full-core 闭合输入, 不表示 FENCE.I 语义完备、全状态空间覆盖或形式证明。三快照只作为合同演化证据, 不构成单变量性能实验。

9.6 反例驱动的候选淘汰与搜索空间收缩

早期双 memory 发射实验发生在多个关键模块尚未满足目标时序的阶段, 因而不复承担当前 RV64CORE 的性能比较。较新的全核候选更直接体现闭环的淘汰作用: branch+ret 虽通过 focused test, Verilator 随后报告 UNOPTFLAT 组合环, 候选被回滚; 打开分支推测后, 局部 ROB-walk 测试也可以通过, 但全核形成“branch speculation 冻结 commit—JALR 等待 backend drain—drain 等待 ROB 排空”的环形等待。spec-on 的

riscv-tests 为 255 PASS/16 FAIL、AM 为 25 PASS/32 FAIL；回退 mode=0 后 riscv-tests 271 PASS/0 FAIL、module 113/113 PASS。

同一反馈纪律也出现在 NEMU，而不只在 NPC RTL。nemu-ubuntu-profile 将 basic block、wide ifetch、decode cache、host-vaddr fast path、Sv39 TLB 和诊断 counter 做成显式 runtime flags；detail counter 默认关闭且默认 run 要求其为 0，从环境层保证“用于定位的探针”不自动成为性能基线。一个 64K decode-cache detail run 将 hit rate 提到 94.52%，但得到 43.19 MIPS，低于同容量 default run 的 43.57 MIPS，工作区据此保留“容量 footprint 抵消 miss 减少”的负结论；只提前识别 c.sdsp/c.ldsp 的 RVC stack fast path 为 44.31 MIPS，也低于相邻 baseline 46.95 MIPS，候选撤回。这些 host MIPS 受 WSL 波动影响，不能跨 run 任取峰值作稳定收益；但 profile 的开关、counter 非干扰和相邻反例使“继续扩大 cache/只优化单条压缩指令”成为可审计的被拒方向。这里，E2E 既调度真实系统 workload，又把负实验转成下一轮 Agent 不应重复的搜索边界。

在后续定向回归中，IFU、FENCE、STORE/AMO 共 23 个 compile-success 负向 RTL 变体被判定器动态拒绝（19+2+2）。这些结果不证明全状态空间正确，却证明在指定 fault set 内，测试不是只能对正确版本打印 PASS。对工程师而言，回滚是一项带复验证据的候选淘汰结果；对 Agent 而言，它把下一轮搜索限制在仍然满足基线合同的空间内。

这些回滚把给定配置下的不合格方向从后续搜索空间中排除；保留基线的全局最优性和未建模回归仍是独立问题。

9.7 性能基线、诊断与未晋级边界

在当前 RV64 双发射乱序核的冻结架构基线上，成熟度收据将同一 RTL、配置与 workload 组合标记为 ARCH_STABLE 和 PERF_BASELINE，同时把 PPA 状态保持为 UNQUALIFIED，并禁止直接晋级。这里的身份合同只用于保证表中各次运行可比较，不把内部哈希或版本号当作论文结论。

表 21: 冻结设计的可重复性能基线

workload	cycles	retired	CPI	IPC	资格边界
CoreMark10	5,262,868	3,183,617	1.653110	0.604921	三次 bit-exact，另有 stats-off 非干扰对照
Dhrystone10000	9,751,462	4,250,000	2.294462	0.435832	三次 bit-exact，另有 stats-off 非干扰对照

这组数字来自同一冻结对象的直接重跑，而不是从旧设计的成功日志中挑选。每个 workload 保留三份独立 stats-on 日志和一份 stats-off 非干扰对照，run manifest、量

测合同、counter schema、ARCH_STABLE pre/post-flight 与 simulator identity 一并进入收据；重复中未删除 outlier。历史 A1/A2 的原始 FAIL 与 endpoint-corrected replay 继续作为判定器演化证据保存，但不参与该基线 CPI 的分子、分母或重复性统计。这里，Skill 约束审查输入，performance instruction 约束判定，receipt/schema 约束可消费结果，三者共同形成表 21，而不是由 Agent 自行选择一个更好看的 CPI。

冻结性能基线的 CPI stack 显示两个 workload 的 memory latency、request outstanding 与 AXI write response 分别为 CoreMark 的 2,414,173/995,957/551,102 cycles，以及 Dhrystone 的 6,550,362/3,910,144/2,370,000 cycles。这些计数是嵌套驻留，不能相加成总周期；它们只为后续 CPI 诊断任务形成竞争假设，不能单独证明根因。

为进一步区分事务阶段，NpcOooOwnerTimingProbe.sv 通过 SystemVerilog bind 接入只读观测逻辑。某次探针运行曾因非零 invalid 事件在 receipt-build 阶段失败，该反例随后进入量测合同。后续同身份 A/B 中，owner-timing 输入 manifest 的 148 个条目在观测前后保持一致，且 production filelist 未被诊断构建修改；canonical production RTL inventory 另按功能签核口径统计为 146 个文件。CoreMark 与 Dhrystone 各三次 reference/diagnostic 重复均 bit-exact，observer noninterference、owner 区间/状态守恒和 runtime cleanup 全部通过，收据形成 OWNER_TIMING_WORKLOAD_MEASURED。然而该收据仍显式写入 hypothesis_selection_authorized=false、optimization_candidate_authorized=false 和 promotion_eligible=false；量测成功并未被扩写为因果或 RTL 候选成功。

初始 owner 收据不授权假设选择；独立因果分析随后返回 H1/H2 耦合和 RESEARCH_REQUIRED。受控 +2-cycle B-response 延迟实验保持两个 workload 的 retired 不变，并分别观察到 239,936 和 1,199,949 个 cycle 增量，由此只授权 H1 作为后续分析方向。该诊断干预没有产生 production RTL 候选，因果状态仍为 HYPOTHESIS_NOT_PROVEN。

历史 Selector decision 曾因 verifier 哈希漂移被 canonical verify 拒绝；新的同身份 decision 已消除 stale reference，并只选择“聚类 current 5 ns 违例路径、定义最小可回退时序候选或明确 GAP”这一下一步动作。参考实现量测能够重复，但 5 ns timing hard gate 失败；当前证据没有形成可接受的 PPA reference，也没有足以替代旧代理报告的完整面积、功耗和物理时序数字。因此该案例的成熟度仍停在 PERF_BASELINE：B-response 延迟敏感性已被观察，production RTL 收益归因和 PPA 晋级均未闭合，PPA 保持 UNQUALIFIED。

这些里程碑给出了方法在不同验证压力下的具体实例。据此，工程产物可以按 NEMU、AM、NPC 与方法构件加以综合，而无需再次复述时间线。

10 工程成果与证据综合

10.1 NEMU、AM 与 NPC：方法形成的实体工程产物

Loop-Work 的结果不能只用“工作流已建立”概括。表 22 按工程对象汇总同一工作区中的实体产物、可复核结果以及它们对方法论证的意义。这里的计数来自冻结 inventory 和分层验证收据，而不是把不同时期的最优数字拼接为一个结果。

这些产物之间存在可追踪的生产关系。AM 生成的同一 BIN 同时进入 NEMU 和 NPC；NPC 的退休状态消费 NEMU 提供的参考状态；模块 TB 和负向 RTL 先检查周期级合同，L2/L3 再检查跨特权级、地址转换、设备与系统软件；只有同一 design-id 的功能与架构门闭合后，RTL 才进入 Yosys/OpenSTA。这些生产关系表明四平面与双闭环已在本仓库形成可执行实例：程序、参考模型、RTL、验证器和实现工具之间存在可由下一节点消费或拒绝的实体对象。它们不能区分该方法的作用与开发者经验、模型变化或工具演进的作用，因而不作为“Loop-Work 比其他流程更有效”的因果证据。

冻结验证收据把 production RTL 的 146 个文件绑定到同一 design-id: L0 为 113/113 模块测试和零 RTL assertion failure; L1 为 61/61 AM、113/113 module、177/177 official 与 DiffTest mismatch=0; L2 为 OpenSBI/mini-system 的完整 all case; L3 为 Linux 6.6/PID1 的完整 all case。四层共同支持“冻结对象在规定压力层级下通过”，却不自动推出完整 Ubuntu 再认证、所有未建模设备正确或物理 PPA 签核。保留这种边界正是领域 Agent 能够继续安全迭代的条件。

10.2 可迁移的工程构件

面向团队协作的 RTL 项目通常由架构、前端、验证、性能、后端和发布角色共同推进。单个 Agent 回答很难成为这些角色的稳定接口；身份绑定的合同与证据包则可以。本文方法提供六类可复用工程构件：

在这种组织中，instruction、Skill 和 E2E 本身也是可版本化工程资产，而不再是某位工程师账号中的个人提示词。commercial-delivery-readiness 节点已经能构建相对路径包，核对三层 policy、Skill、维护脚本和数据库入口，排除目录重复嵌套，并扫描本机 home、Windows 用户路径和本地标识。该节点 PASS 只表示 AI 开发环境达到既定可迁移包验收合同（packaging contract），不代表处理器或组织流程获得认证；但它说明同一套方法可以把“如何使用 Agent”从口头经验变成可审查、可供迁移验证的内部开发基础设施。

这些构件在机制上允许团队把交接对象从截图和口头结论转为可重建实验，把系统层反例沉淀为定向回归，并在 checker-only 变化时重放冻结证据。本文案例实际观察到的是：旧身份消费、局部 PASS 晋级、不合格 owner 量测、历史 Selector decision

表 22: NEMU、AM 与 NPC 的工程产物、验证结果和结论边界

对象	形成的能力与产物	直接证据	对 Loop-Work 的说明及边界
NEMU	RV64 M/A/F/D/C 指令解释；M/S/U 系统态；UART、virtio-blk、PLIC 相关中断、RTC、syscon；既可独立运行 Linux/Ubuntu，也可编译为 NPC DiffTest reference	Ubuntu 22.04.5 root shell、Linux 6.6、systemd running 与 0 failed units 的独立参考记录；DiffTest API 可同步 memory、GPR/PC、CSR/FPR、单步和中断	说明 Agent 能维护“系统参考机”和“逐退休参考模型”两种角色；NEMU 启动 Ubuntu 不能替代 NPC RTL 的系统证据
AM/am-kernels	依据 ARCH 组合 ISA/platform，编译 C/C++/汇编并链接 AM/klib，产生 ELF、BIN 和反汇编；提供 CPU、异常、设备、虚存及性能 workload	61/61 项 AM CPU 测试；代表用例包括 RV64 指令/CSR 审计、W/D 两宽度 9 类 AMO、2048 轮乱序访存顺序、mini-system handoff、PLIC/UART/Sv39/fence.i 等	说明测试程序是与 RTL 同级的任务产物，并可在 NEMU/NPC 之间复用；workload 通过不等于未覆盖状态正确
NPC single/SoC	RV32 五级流水、Verilator/DPI、PMEM/MMIO、AM 镜像运行；经 AXI bridge 接入 sysxSoCFull，从 MROM 取指并形成 SoC 运行入口	single 可执行 C/AM 应用并依据 trap/exit 终止；SoC lint/run 与指定测试的 GOOD/BAD TRAP、DiffTest 记录	说明 Agent 能把局部 RTL 扩展为软件可执行 target 和平台接口；不等于完整 SoC 外设签核
NPC RV64 OoO	RV64IMAFDC、Zicsr、Zba/Zbb/Zbc/Zbs，M/S/U、Sv39、PTW、I/D TLB、PMP16，整数与浮点 rename/PRF/IQ/ROB，取指/译码/派发 2 宽、双完成和双退休	同一 design-id 下 module 113/113、official 177/177、AM 61/61、DiffTest mismatch=0；L0/L1/L2/L3 均 PASS，架构冻结为 ARCH_STABLE	说明方法可实例化到跨模块、系统和性能前置层；该核尚未覆盖 L2/coherence/ECC 等完整工业特性，且 PPA 仍 UNQUALIFIED

表 23: 方法构件、工业问题与可交付产物

构件	对应问题	直接产物	案例证据边界
主张规范化规则（把“完成”写成五元组）	“完成”在不同角色间含义漂移	工程对象、实验身份、验收谓词、结论与边界	尚未形成跨组织标准
分层实验身份	不同设计、工具或 workload 串证	清单与 design、build、workload、evidence 四类身份	历史早期记录不完整
失败反馈路由（按对象归责）	所有 FAIL 都被错误地导向 RTL 修改	根因/竞争假设、冻结重放、回滚与证据缺口	依赖可观察性与判定器质量
负向资格化（用可编译错误版本检查判定器）	单次 PASS 无法证明测试敏感	编译成功的负向 RTL 与唯一拒绝 marker	只覆盖声明的故障集合
失败关闭发布	长跑中断或证据缺失仍遗留绿色状态	stage、signal、cleanup 与证据完成收据	不替代功能或物理签核
成熟度阶梯（局部、架构、性能、物理实现分层）	局部功能、系统与 PPA 结论相互越级	ARCH_STABLE、PERF、PPA、SYSTEM 状态	PPA 尚不具备晋级资格

漂移和代理 PPA 越级被相应门禁拒绝；重建后的 Selector 也只获准选择下一分析动作。由于没有对照组，本文不推断因此节省了多少人时、算力、缺陷逃逸或上市周期。

10.3 四项工程准则的案例观察与证据边界

方法章已经定义 D1–D4 四项证据与控制准则及其违规事件，本节只报告案例中的对应观察，避免把实现遵从性误解为准则的普遍有效性或方法的因果效果。

表 24: D1–D4 在纵向案例中的观察与证据上限

准则	案例观察	证据上限
D1 身份拒绝	V90 身份元组冲突被降级；A3 原始 FAIL 与冻结 replay 分开保存和消费	覆盖已记录的错配类型，尚无总体逃逸率
D2 负向资格	IFU、FENCE、STORE/AMO 的 23 个可编译目标变体均被对应门禁拒绝	只说明该 fault set 下的判定器敏感性
D3 失败关闭	A3 原始状态未被重放覆盖；TERM、marker/cleanup 故障与 verifier 哈希漂移均未遗留可消费绿色终态	覆盖已注入的终止、发布与版本漂移故障
D4 有界晋级	branch speculation 候选回滚；owner 量测为 MEASURED，H1 只获分析授权，production RTL 仍关闭；代理 PPA 保持 UNQUALIFIED	这些个案未越级；跨任务固化条件尚未系统校准

这些观察符合工程设计准则规定的期望约束，并给出了相应违规结果被拒绝的案例；相对生产率、跨模型恢复成本和跨团队迁移收益仍由后续受控实验回答。

在实体工程产物、可迁移构件和四项工程准则之外，本章再按研究问题与描述性指标回收同一批案例证据。案例边界、主张选择、编码冲突与数据保留协议已经在前置的案例研究设计章固定，因此下列统计承担审计责任，不能脱离其选择规则解释为总体成功率。

10.4 研究问题的案例回答与描述性审计

表 25 给出纵向案例能够回答到的范围。表中不再只写编号：RQ1–RQ4 分别追问知识怎样组织、目标怎样编译、设计与验证怎样闭环、经验怎样固化，RQ5 则汇总实体产物及其不能越过的证据边界。

同一批案例材料还接受机器可读主张台账的描述性审计。30 条主张中有 18 条 SUPPORTED、4 条 PARTIALLY_SUPPORTED、1 条 OBSERVATIONAL_ONLY、6 条 CONTRADICTED、1 条 MISSING。CONTRADICTED 不等于“系统失败六次”：其中包括论文主动构造并由现有证据反驳的强命题，例如“代理 STA 已证明 200 MHz 物理签核”。同理，SUPPORTED 是逐条主张状态，不是方法成功样本。

表 25: 研究问题、观测材料与案例回答边界

问题	关键观测材料	案例回答	不能推出的结论
RQ1 领域能力构成	AGENTS、instruction、Skill、E2E、spec、memory 与工具入口的分层接线	分散知识可按不变量、程序化能力、任务合同和执行图组织为可发现环境	该分层是唯一可能的知识组织方式
RQ2 目标到任务	RV32 lint/动态闭环差异；接口先行、RTL 四段式、机器合同与代次控制	自然语言目标可先转为事务图、修改边界、成功条件和授权节点，再进入 RTL	相对直接提示的平均速度提升
RQ3 设计—验证闭环	AM image 分叉到 NEMU/NPC；模块 TB、23 个 compile-success mutant、DiffTest 与 L0-L3	Agent 可以协同生成 RTL、测试和反例，并消费参考状态与分层工具反馈保持冻结身份的一致性	对未建模故障的完备拒绝率或全状态空间正确
RQ4 能力演进	接口合同、场景 profile、长跑状态协议和历史缺陷下沉的过程链	重复反例可以固化为 instruction、Skill、E2E、测试或门禁，并进入后续业务任务	每个环境构件的独立生产率因果贡献
RQ5 产物与边界	NEMU/AM/NPC 实体产物；113/113、177/177、61/61、DiffTest=0 与 L0-L3	同一方法已在所列 RV32/RV64、single/SoC/OoO 与系统压力层级实例化并形成可复核产物	跨仓库普遍可迁移、完整工业 CPU 或物理 PPA signoff

为使描述性审计和第 11 节的受控实验使用同一指标语言，本文定义 FCR（False Completion Rate，假完成率）、VSR（Verified Success Rate，经验证成功率）和 UPR（Unsafe Promotion Rate，不安全晋级率）。令 $\mathcal{I}$ 为按预注册纳入规则形成的合格运行集合， $N = |\mathcal{I}|$ ；对每个 $i \in \mathcal{I}$ ， $C_i = 1$ 表示 Agent 声明完成， $Y_i = 1$ 表示独立隐藏 oracle 的全部强制门通过， $P_i = 1$ 表示结果已发布或晋级。以下 $[\cdot]$ 为示性函数：条件成立取 1，否则取 0。严格指标为

$$\begin{aligned} \text{FCR} &= \frac{\sum_{i \in \mathcal{I}} [C_i = 1 \wedge Y_i = 0]}{\sum_{i \in \mathcal{I}} [C_i = 1]}, \\ \text{VSR} &= \frac{\sum_{i \in \mathcal{I}} [Y_i = 1]}{N}, \\ \text{UPR} &= \frac{\sum_{i \in \mathcal{I}} [P_i = 1 \wedge Y_i = 0]}{\sum_{i \in \mathcal{I}} [P_i = 1]}. \end{aligned}$$

记 $n_C = \sum_{i \in \mathcal{I}} [C_i = 1]$ 、 $n_P = \sum_{i \in \mathcal{I}} [P_i = 1]$ 。FCR 仅在 $n_C > 0$ 时有定义，VSR 仅在 $N > 0$ 时有定义，UPR 仅在 $n_P > 0$ 时有定义；相应分母为零时统一报告 N/A，不得记为 0。若任一必需字段未登记，或隐藏 oracle 未按协议冻结，则指标保持 UNQUALIFIED，不得把缺失值按 0 计入。

表 26 把三个指标先用于主张表的资格审计。结果不是填入一个看似精确的百分比，而是确认现有字段能否构成合法分子和分母。

表 26: FCR、VSR、UPR 在 30 条主张上的资格状态

指标	必需字段与分母	案例材料	资格结论
FCR	真实的完成声明 C 、独立隐藏真值 Y ，且 $n_C > 0$	台账混合原始声明、限定重写和为审计构造的反命题；没有统一隐藏 oracle	UNQUALIFIED； 6 条 CONTRA-DICTED 不得直接作为分子
VSR	每个正式 run 的同预算隐藏验收 Y ，且 $N > 0$	只有部分事件存在冻结 replay，且未尝试重放的主张没有统一登记	UNQUALIFIED； 不能把 18 条 SUPPORTED 写成 18/30 VSR
UPR	下游消费/晋级位 P 、对应隐藏真值 Y ，且 $n_P > 0$	若干关键事件明确保持 GAP/UNQUALIFIED，但 30 行没有统一的 publication-consumed 字段	UNQUALIFIED； 不能用“未晋级个案”推断总体 UPR

这个负结果本身完成了描述性审计：案例材料足以给出 30 条证据状态，却不足以估计严格 FCR/VSR/UPR。后文的受控实验设计将 C, Y, P 设为逐 run 必填字段；在

此之前，本文只报告 A3 原始 FAIL、owner-timing 量测可重复、H1 敏感性获分析授权但 production RTL 仍关闭、历史 Selector decision 被拒绝和 PPA UNQUALIFIED 等可逐项复核事件，不把它们汇总为因果效果。

11 研究不足与下一步攻克方向

前文只报告已经存在的仓库对象、运行记录和限定结论。本章集中列出尚未实现、尚未形成充分证据或尚未完成外部复制的内容，并把每一项缺口转换为可执行的研究问题、实验设计和验收产物。以下内容均是后续研究计划，不构成本文已经取得的实验结果。

11.1 当前证据的适用边界

1. 方法效果尚不可归因。案例中模型、任务难度、RTL、验证平台与开发者经验共同演进，且没有随机化对照或同预算基线。现有材料支持“接口已被实例化”和“若干违规结果被拒绝”，不支持 Loop-Work 相对直接提示、一般工具 Agent 或人工流程具有更高成功率、更低成本或更少缺陷逃逸。
2. 责任分解尚无最小性与完备性证明。四平面能够区分本文列出的案例状态对，但尚未建立 soundness、completeness、状态不变量或复杂度结论，也没有排除三平面、五平面及其他编码方案。D1-D4 是工程设计准则，不是已经被普遍验证的科学定律。
3. 验证集合不等于覆盖率闭合。113/113 模块测试、177/177 official tests、61/61 AM、DiffTest mismatch=0 与 L0-L3 PASS 只描述所列集合；本文没有统一的 line、branch、toggle、FSM、assertion 与 functional coverage 模型，也没有完整 coverage waiver。
4. 反例规模和故障模型有限。23 个 compile-success mutant 只说明对应 oracle 对该 fault set 敏感，不能覆盖全部协议、跨周期状态、时序、功耗或物理故障。FENCE.I 的结果同样是指定 single-hart SMC 路径上的范围限定回归闭合，而非对全部实现错误的完备证明。
5. 系统与性能重复性不足。CoreMark 和 Dhrystone 的三次 bit-exact 证明仿真观测在该配置下确定，不构成跨 workload 统计稳定性；约十九小时 Ubuntu 材料来自一次指定 design/config/rootfs 的完整执行及其冻结重放，不能估计独立启动成功率或时长分布。owner-timing 已观察到 B-response 延迟敏感性，但 production RTL 候选、收益归因和晋级仍未授权，因而尚未形成可归因的 CPI 优化结果。

6. 物理实现与签核尚未闭合。Yosys、OpenSTA 和开放 PDK 已进入反馈接口，但缺少完整 I/O、真实 memory macro、floorplan、CTS、route、SPEF、OCV/MMMC、活动率功耗、多 PVT、DRC/LVS、EM/IR 与 DFT。PPA 因而保持 UNQUALIFIED，L0-L3 功能 PASS 不能补偿这一缺口。
7. 外部效度和复现信息有限。材料来自一个 RISC-V 仓库和一位主要开发者；尚未在第二个 HDL 项目、多人团队、商业 EDA 或不同工艺节点复制。早期 model ID、seed、temperature、完整 context 与部分 raw payload 没有统一冻结，工程证据可重放不等于模型采样轨迹可重建。

30 条目的性主张可以被逐项复核，但 12 条非盲第二 Agent 审查不构成人类双编码，且 C, Y, P 字段不足使 FCR、VSR、UPR 仍为 UNQUALIFIED。由此，本文回答的是“特定工程状态怎样形成可审计主张”，而不是“该方法平均提高了多少生产率或正确率”。

11.2 未实现事项与研究验收矩阵

表 27: 尚未实现内容、下一步研究动作与完成条件

未闭合问题	现有证据边界	下一步研究动作	可接受的验收产物
方法因果效果	单仓库纵向共现，无同预算基线	执行 B0-B3 随机化对照，并在 B3 上做 D1-D4 单组件消融	预注册协议、隐藏 oracle、盲审、效应量、置信区间与失败分布
四平面理论边界	只有案例内不可区分状态对	定义不变量与语义映射；比较三/四/五平面和现有 ACI、PROV、GSN 表达	定理或明确反例、机器检查模型及适用条件
覆盖率模型	测试通过计数，没有统一覆盖分母	建立语句、分支、翻转、FSM、断言与功能覆盖模型及 waiver 流程	分维度报告、交叉覆盖、不可达证明和审计过的 waiver
形式与等价验证	DiffTest、断言和定向 mutation	对 ready/valid、flush、replay、精确异常、提交顺序和访存序建立性质；对保持功能的重构做等价检查	proof/cex、等价日志、性质—模块—设计身份映射
Mutation 规模	23 个功能逻辑目标变体	扩展到不少于 100 个分层 fault，覆盖握手、owner、优先级、跨周期状态和恢复协议	生成规则、触发记录、kill matrix、存活 mutant GAP

续下页

表 27: 尚未实现内容、下一步研究动作与完成条件（续）

未闭合问题	现有证据边界	下一步研究动作	可接受的验收产物
性能泛化	CoreMark/Dhrystone 与单一 ROI	增加嵌入式、访存、分支、原子并发和 Linux 系统 workload, 冻结 binary/输入/ROI	workload manifest、计数器原始记录、重复分布与置信区间
Ubuntu 重复性	一次完整执行及同一证据 replay	在相同 design/config/rootfs 下执行多次冷启动, 并预注册超时、登录、服务与自然终止 oracle	独立 run 身份、完整日志、启动时长分布和失败分类
物理实现与 PPA	预布局综合/STA 诊断, PPA 未资格化	完成 floorplan、CTS、route、SPEF、MMMC、功耗、DRC/LVS、EM/IR 与 DFT 路径	物理实现 manifest、时序/功耗/面积/物理验证与扫描报告
外层固化阈值	θ 只有参数化接口, 历史字段不完整	比较 replay、跨合同复用和稳定窗口阈值, 测量误晋级与维护成本	阈值敏感性曲线、预登记选择依据和逐构件审计表
跨项目复现	单仓库、单主要开发者	在第二 RTL 项目和多人团队复制, 并固定模型、预算和工具权限	跨仓库任务集、交接记录、成本分母与复现实验报告

11.3 受控对照与组件消融

方法效果的首要研究问题是: Loop-Work 的知识、控制、执行和证据机制是否相对于直接提示产生可重复、可归因的交付收益, 以及收益来自哪个组件而非额外 Token 或工具预算。后续实验采用四个递增条件: B0 为冻结提示的一次生成; B1 加入受预算约束的工具反馈与迭代; B2 再加入有界召回、任务合同和显式成功条件; B3 最后加入实验身份、证据新鲜度、mutation、rollback 与原子发布。任务按局部功能、跨层逃逸、性能回退、身份错误、半发布和控制事务风险分层, 在同一模型、解码参数、初始提交、工具权限、Token/时间预算和执行后端下随机分配。

隐藏 oracle、mutation 集和停止规则必须由被测工作流之外的过程预先冻结, 结果由盲化人工审查处理; B3 上再分别移除 D1–D4 所对应的身份、负向资格、状态提交或晋级约束, 以估计单组件作用。每次 run 必须填写完成声明 C 、隐藏真值 Y 和下游消费位 P , 并按上述零分母与缺失字段规则计算 FCR、VSR 与 UPR, 同时报告任务闭合率、耗时、Token、计算成本、效应量和置信区间。experiment-plan.md 仅作为协议草案; 其中的 512 次是资源规划值, 正式样本量仍需在不观察结果的前提下由功效分析确定。

11.4 验证闭合、形式性质与规模化反例

下一步首先建立 verification plan，把 ISA、特权、MMU、异常、乱序、存储序、设备与系统交互映射到 functional coverpoint、cross、assertion 和可执行测试；代码覆盖与功能覆盖分别报告，不用单一百分比替代。覆盖缺口必须绑定不可达证明或经独立复核的 waiver。对于 ready/valid、flush、replay、异常、提交顺序和 memory owner 等局部不变量，采用模型检查或性质证明；不改变功能的重构采用等价性检查，改变流水级或延迟的优化则使用与该时序边界相符的顺序等价或专门性质。

Mutation campaign 从现有 23 个目标变体扩展到不少于 100 个，按数据通路、握手/反压、错误 owner、刷新竞争、异常优先级、CSR/特权、内存顺序和跨周期状态更新分层。每个 mutant 都要证明可编译、语义非空、能够触发并传播到观察点；“测试/性质—mutant”杀伤矩阵和存活变体共同发布，任何存活项形成显式 GAP。时序、功耗和物理故障不能由 compile-success RTL mutation 代替，必须在相应实现层另建故障模型。

11.5 性能、系统重复性与外层阈值校准

性能研究将 workload 扩展到嵌入式程序、STREAM 或 pointer-chasing 访存负载、分支密集程序、原子与锁竞争测试，以及文件系统、进程创建、中断和系统调用等 Linux 场景。每项实验固定 ELF/BIN、编译选项、输入、ROI 和初态，除 CPI/IPC 外同步保存退休、分支失误、cache/TLB 和 owner 事件；报告逐次分布与置信区间，而不把三次 bit-exact 写成总体性能稳定。

Ubuntu 验证采用多次独立冷启动，区分“同一原始观测的 checker replay”和“新的系统执行”。每次运行都绑定 design、simulator、kernel、DTB、rootfs 与 PID1 身份，预注册超时、登录、关键服务、设备事务和自然终止条件，失败与超时不得从统计中删除。与此同时，对 $\theta = (r_{\min}, u_{\min}, t_{\min})$ 进行敏感性分析，比较不同 replay、复用和稳定窗口对误晋级、漏晋级与维护成本的影响；未经校准前，环境改动只能称为候选能力或已被后续任务调用的接口。

11.6 从 RTL 实现反馈推进到物理签核

物理方向的研究问题是：Yosys/OpenSTA 形成的预布局网表和时序诊断能否预测布局布线后的频率、面积与功耗，以及 Selector 选择的候选在真实物理约束下是否仍位于合格 Pareto 前沿。后续流程固定工艺库、I/O、时钟、利用率、宏单元和实现脚本，依次完成 floorplan、power plan、placement、CTS、route 和寄生提取；基于 SPEF 执行多模式多角 STA，并用代表性 workload 的 VCD/SAIF 估算功耗。物理验收还包括拥塞、天线、DRC/LVS、EM/IR 以及 scan/ATPG 等 DFT 工作。

验收产物至少包括 floorplan 与利用率、CTS skew、route/congestion、SPEF/MMMC、活动率绑定功耗、DRC/LVS、EM/IR 和 DFT/扫描覆盖报告，并把工艺角、工具版本、随机种子、约束与 design-id 写入同一 manifest。所有候选必须与同一基线共享可比较身份；timing、物理验证、功耗或 DFT 任一硬门缺失时，PPA 继续保持 UNQUALIFIED。完成该链后，才有条件研究 RTL 级优化是否具有后端可持续性。

11.7 研究优先级

近期优先执行 B0-B3 小规模预注册试验并建立 coverage 基线，因为二者分别决定方法是否具有可归因效果以及现有验证究竟覆盖了什么。其后扩展形式性质与 mutation fault set，再进行多 workload 与 Ubuntu 独立重复；这些步骤为物理实现提供稳定功能和性能基线。最后推进完整物理签核并在第二项目、多人团队中复制。这样的次序避免在功能、覆盖和实验身份仍不稳定时，过早把一次综合网表或长跑日志解释为方法效果或芯片级结论。

12 结论

本文提出 Loop-Work，一种面向处理器 RTL 功能开发与分层验证的仓库原生领域 Agent 构建方法。其基本观点是：基础模型提供推理与候选生成能力，仓库环境提供专业知识、受控执行、验证反馈和长期记忆；二者需要作为同一工程系统接受设计和审查。控制面把目标编译为对象、权限和任务图，知识面选择 instruction、Skill、spec 与历史事实，执行面协同产生 RTL、测试、反例和工具观测，证据面再限定这些结果能够支持的结论。内层闭环完成受约束设计任务，外层闭环把稳定反例固化为下一轮可调用的环境能力。

在 ysyx-workbench 中，这一方法已经贯通真实工程数据流：AM 把 C/汇编、运行时和平台适配构建为可复用 workload；NEMU 同时承担系统参考机与 NPC 的逐退休 reference；NPC 从 RV32I 五级流水和 ysyxSoC 接入演进为支持 RV64IMAFDC、Zicsr、Zba/Zbb/Zbc/Zbs、M/S/U、Sv39、PTW/TLB、PMP16、浮点和原子路径的双发射乱序核。同一冻结设计身份下，113/113 模块测试、177/177 official riscv-tests、61/61 AM、DiffTest mismatch=0 和 L0-L3 分层验证共同形成 ARCH_STABLE；Yosys/OpenSTA 已进入反馈链，但因实现条件尚未完备，PPA 保持 UNQUALIFIED。

这些结果说明，在人工定义的目标、规则与验收门下，Agent 可以在同一方法中辅助产生并消费架构合同、RTL、AM 程序、testbench、负向 RTL、参考状态和实现报告，而不只是输出代码。23 个目标负向变体被对应 oracle 拒绝、系统原始 FAIL 与冻结重放并存、branch speculation 候选被回滚、owner-timing 从 MEASURED 推进到 H1 敏感性分析但未授权 production RTL，以及历史 Selector decision 被 canonical verifier 拒绝，表明参考实现中的若干安全约束在这些个案中实际触发并阻止了越级消费。候

选资格化在这里承担安全阀作用，但方法的主体仍是领域知识、工程工具、分层验证和反馈资产怎样共同构成 Agent。

Loop-Work 的潜在可迁移价值在于把个人提示经验转化为可版本化、可执行并可由团队审查的工程接口：模型可以变化，任务仍由合同、工具和验证责任约束；项目可以扩展，新的系统反例仍可下沉为 AM 测试、模块 TB、断言、mutation 或固定门禁。单仓库纵向材料只支持方法的可实例化性、过程链和工程产物边界；相对优势、四平面理论边界、覆盖率闭合、系统重复性、跨项目复现和物理签核均尚未完成，其研究路径已在第 11 节集中给出。

研究台账与复现材料位于 docs/research-paper/；公开仓库为 mcsaver/oscpu。

参考文献

- [1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 50528–50652, 2024. DOI: 10.52202/079017-1601.
- [2] M. Liu, N. Pinckney, B. Khailany, and H. Ren, “Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation,” in *2023 IEEE/ACM International Conference on Computer Aided Design*, pp. 1–8, 2023. DOI: 10.1109/ICCAD57390.2023.10323812.
- [3] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, “ChatEDA: A Large Language Model Powered Autonomous Agent for EDA,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 10, pp. 3184–3197, 2024. DOI: 10.1109/TCAD.2024.3383347.
- [4] J. Wu, Y. Lei, Z. Zhang, X. Meng, D. Yang, P. Li, J. He, and X. Mao, “Mantra: Mutation Testing of Hardware Design Code Based on Real Bugs,” in *2023 60th ACM/IEEE Design Automation Conference*, pp. 1–6, 2023. DOI: 10.1109/DAC56929.2023.10247962.
- [5] D. Gil, J. Gracia, J. C. Baraza, and P. J. Gil, “Study, Comparison and Application of Different VHDL-Based Fault Injection Techniques for the Experimental Validation of a Fault-Tolerant System,” *Microelectronics Journal*, vol. 34, no. 1, pp. 41–51, 2003. DOI: 10.1016/S0026-2692(02)00128-3.
- [6] O. C. Z. Gotel and A. C. W. Finkelstein, “An Analysis of the Requirements Traceability Problem,” in *Proceedings of the First IEEE International Conference on Requirements Engineering*, pp. 94–101, 1994. DOI: 10.1109/ICRE.1994.292398.
- [7] Object Management Group, *Structured Assurance Case Metamodel, Version 2.3*, OMG Document formal/23-05-08, 2023. Official PDF.

- [8] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, “in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes,” in *28th USENIX Security Symposium*, pp. 1393–1410, 2019. USENIX paper page.
- [9] SLSA Community, *SLSA Specification, Version 1.2*, 2025. Official specification.
- [10] G. Schermann, J. Cito, P. Leitner, and H. C. Gall, “Towards Quality Gates in Continuous Delivery and Deployment,” in *2016 IEEE 24th International Conference on Program Comprehension*, pp. 1–4, 2016. DOI: 10.1109/ICPC.2016.7503737.
- [11] IEEE and IEC, *IEEE/IEC International Standard–SystemVerilog–Part 2: Universal Verification Methodology Language Reference Manual*, IEEE/IEC Std 62530-2-2023, 2023. DOI: 10.1109/IEEESTD.2023.10287892.
- [12] International Organization for Standardization, *Road Vehicles–Functional Safety–Part 5: Product Development at the Hardware Level*, ISO 26262-5:2018, 2nd ed., 2018. ISO catalogue page.
- [13] Y. Zhao, H. Zhang, H. Huang, Z. Yu, and J. Zhao, “MAGE: A Multi-Agent Engine for Automated RTL Code Generation,” in *2025 62nd ACM/IEEE Design Automation Conference*, 2025. DOI: 10.1109/DAC63849.2025.11133191.
- [14] P. Yubeaton, S. Garg, and C. Hegde, “Exploring the Agentic Frontier of Verilog Code Generation,” *arXiv preprint arXiv:2603.19347*, 2026. Primary preprint page.
- [15] N. Pinckney, C. Deng, C.-T. Ho, Y.-D. Tsai, M. Liu, W. Zhou, B. Khailany, and H. Ren, “Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification,” *arXiv preprint arXiv:2506.14074*, 2025. Primary preprint page.
- [16] F. Cui, H. Hou, Z. Luo, C. Yin, and Y. Liang, “HWE-Bench: Benchmarking LLM Agents on Real-World Hardware Bug Repair Tasks,” *arXiv preprint arXiv:2604.14709*, 2026. Primary preprint page.
- [17] Y. Zhao, Z. Wu, B. Yuan, Z. Yu, H. Zhang, W. Ni, C.-T. Ho, H. Ren, and J. Zhao, “PRO-V-R1: Reasoning Enhanced Programming Agent for RTL Verification,” *arXiv preprint arXiv:2506.12200*, 2025. Primary preprint page.
- [18] X. Xin, J. Lou, J. Li, J. Yan, P. Xiao, D. Wu, H. Li, W. Lu, W. Fan, X. Qu, Y. Zhao, M. Yu, Z. Di, and Y. Lin, “GoGoTB: Agentic RTL Verification with Specification-Grounded Coverage Closure,” *arXiv preprint arXiv:2607.26181*, 2026. Author manuscript.
- [19] C. Deng, Z. Yu, G.-T. Liu, N. Pinckney, B. Khailany, and H. Ren, “ACE-RTL: When Agentic Context Evolution Meets RTL-Specialized LLMs,” *arXiv preprint arXiv:2602.10218v2*, 2026. Primary preprint page.

- [20] N. K. Juluru, C. Deng, C. Yu, N. Pinckney, and B. Khailany, “NVIDIA Nemotron 3 Ultra Leads Open Models on Accuracy and Efficiency in Agentic RTL Coding,” *NVIDIA Developer Technical Blog*, 26 July 2026. Official technical release.
- [21] J. Deng, Z. Chen, X. Wei, T. Xing, C. Wen, Q. Sun, and C. Zhuo, “Can AI Agents Really Complete RTL-to-GDS? Lessons from Benchmarking Tool-Interactive EDA Workflows,” *arXiv preprint arXiv:2607.17528v3*, 2026. Primary preprint page.
- [22] R. M. Ghorab, E. Parisi, C. Gutierrez, M. Alberti-Binimelis, M. Moreto, D. Garcia-Gasulla, and G. Kestor, “NotSoTiny: A Large, Living Benchmark for RTL Code Generation,” *arXiv preprint arXiv:2512.20823*, 2025; presented as a Research Manuscript at DAC 2026. Primary preprint page.
- [23] B. Skakov, S. Abduraimov, O. Nurman, and N. Kabylkas, “Firefly: Shedding Light on Verification Gaps Using LLM-Powered Mutation Testing,” in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD*, pp. 1–10, 2025. DOI: 10.1109/MLCAD65511.2025.11189125.
- [24] S. Jasper, M. Luu, E. Pan, A. Tyagi, M. Quinn, J. Hu, and D. K. Houngrinou, “BugGen: A Self-Correcting Multi-Agent LLM Pipeline for Realistic RTL Bug Synthesis,” in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD*, pp. 1–9, 2025. DOI: 10.1109/MLCAD65511.2025.11189127.
- [25] The Verkor Team: R. Krishna, S. Krishna, and D. Chin, “Design Conductor: An Agent Autonomously Builds a 1.5 GHz Linux-Capable RISC-V CPU,” *arXiv preprint arXiv:2603.08716*, 2026. Primary preprint page.
- [26] H. Zhang, Z. Yu, C.-T. Ho, H. Ren, B. Khailany, and J. Zhao, “LLM4Cov: Execution-Aware Agentic Learning for High-Coverage Testbench Generation,” accepted to *The 43rd International Conference on Machine Learning*; camera-ready author manuscript, arXiv:2602.16953v3, 2026. Author manuscript.
- [27] Siemens Digital Industries Software, “Siemens Advances Self-Verifying Agentic AI Workflows for Semiconductor and PCB Design,” official corporate product announcement, 26 July 2026. Official industry release.
- [28] R. Berke, “Introducing the LBC-bench Leaderboard: A New Public Benchmark for LLMs in Chip Design,” *Silicon Integration Initiative*, 21 July 2026. Official benchmark announcement.
- [29] L. Moreau and P. Missier, eds., *PROV-DM: The PROV Data Model*, W3C Recommendation, 30 April 2013. Official specification.
- [30] SCSC Assurance Case Working Group, *Goal Structuring Notation Community Standard, Version 3*, SCSC-141C, Safety-Critical Systems Club, 2021. Official standard page.

- [31] E. Singh, K. Devarajegowda, S. Simon, R. Schnieder, K. Ganesan, M. R. Fadiheh, D. Stoffel, W. Kunz, C. Barrett, W. Ecker, and S. Mitra, “Symbolic QED Pre-silicon Verification for Automotive Microcontroller Cores: Industrial Case Study,” in *2019 Design, Automation & Test in Europe Conference & Exhibition*, pp. 1000–1005, 2019. DOI: 10.23919/DATE.2019.8715271.
- [32] S. Chattopadhyay, K. Devarajegowda, B. Zhao, F. Lonsing, B. A. D’Agostino, I. Vavelidou, V. D. Bhatt, S. Prebeck, W. Ecker, C. Trippel, C. Barrett, and S. Mitra, “G-QED: Generalized QED Pre-silicon Verification beyond Non-Interfering Hardware Accelerators,” in *2023 60th ACM/IEEE Design Automation Conference*, 2023. DOI: 10.1109/DAC56929.2023.10247903.
- [33] P. Runeson and M. Höst, “Guidelines for Conducting and Reporting Case Study Research in Software Engineering,” *Empirical Software Engineering*, vol. 14, pp. 131–164, 2009. DOI: 10.1007/s10664-008-9102-8.
- [34] D. Beach and R. B. Pedersen, *Process-Tracing Methods: Foundations and Guidelines*, 2nd ed., University of Michigan Press, 2019. DOI: 10.3998/mpub.10072208.
- [35] C. O’Connor and H. Joffe, “Intercoder Reliability in Qualitative Research: Debates and Practical Guidelines,” *International Journal of Qualitative Methods*, vol. 19, 2020. DOI: 10.1177/1609406919899220.

补充材料说明

以下附录保存正文主张所依赖的文件级追踪关系、实现节选、机器可读主张表与复核入口，供审稿和复现时按需查阅；它们不属于正文的方法贡献或新增实验结果。正文只保留理解 Loop-Work、案例设计、工程结果及其证据边界所必需的内容。

A 参考实现的追踪表

本附录保存参考实现的逐文件、逐字段映射和承重实现节选。正文以四面接口、数据流和判定边界为主；这里的材料用于复核“抽象方法由哪些仓库载体实现”，不把文件数量、代码存在或环境自测结果当作方法有效性证据。

表 28: 四面方法接口与参考实现的上下游映射

主归属面、方法算子与实现载体	上游输入	下游输出	在闭环中的意义
控制面：任务生命周期控制 <code>agent-flow.c/.sh</code>	用户目标、任务类别、显式修改路径、证据指针	<code>generation</code> 、固定门禁、候选/正式终态与有界档案	使对话式工作变成有代次、可失效、可终止的控制过程
执行面：软环境入口 <code>agent-env.sh</code> 、 <code>agent-run.sh</code>	仓库根、工具链安装与场景开关	可追踪的 NEMU、AM、NPC 与 EDA 环境和确定命令入口	使非交互节点不依赖个人 shell 隐式状态
知识面：规则、spec 与 instruction	架构目标、接口、Linux 层级、CPI/PPA 裁决规则	不可省略的工程不变量、任务分类和验证顺序	把工程师的隐性判断变成模型可发现的声明式策略
知识/控制接口：Skill 与合同生成器	专业策略、任务目标、RTL、spec、TB 与 evidence 范围	已校验合同、结构化命令和可派发提示	把“应该怎样做”编译为可重复操作，而非继续扩写 Prompt
控制/执行接口： <code>profile</code> 、 <code>agent-e2e.sh</code> 、 <code>module library</code>	合同、依赖闭包、owner、节点函数	按序节点执行、逐节点日志和状态事件	把单点能力组合为从局部修改到端到端判定的任务图
执行/证据接口： <code>task-run-status.sh</code>	命令返回码、证据完成位、清理返回码与信号	原子 PASS 或带 stage 的 FAIL	阻止“进程退出即工程成功”
证据面： <code>report.sh</code> 、 <code>task-run</code> 、 <code>manifest</code>	节点元组、raw evidence、身份与时间	<code>receipt</code> 、 <code>evidence index</code> 、 <code>marker</code> 、 <code>publication</code>	将一次执行转化为可审计且不能半发布的交付单元
证据/知识接口： <code>dev_memory</code> 与索引 API	稳定 memory、 <code>task-run</code> 摘要、evidence 元数据	<code>profile/focus</code> 约束的 <code>context brief</code> 与历史指针	连接上一轮证据面和下一轮知识面，而不全量回灌历史
外层能力闭环： <code>agent-maintain.sh</code> 与 <code>package builder</code>	控制器、Skill、 <code>profile</code> 、 <code>schema</code> 、 <code>policy</code>	<code>quick/final/release</code> 资格结果与可迁移包	方法自身也经过正负向 gate，避免规则存在但不可执行

表 29: agent-flow 命令与状态产物

命令	动作	主要产物
begin	建立 task、分类、档位和 generation=1	meta、events.log
record	登记显式路径与固定 gate pointer; 使旧代次失效	paths.log、gates.log
evidence	登记 PASS/FAIL/GAP 与已存在的 artifact	evidence.tsv
decision	登记 hypothesis、evidence、counterexample 与 rollback	decisions.tsv
reclassify	用户扩展范围时改变 task class 并递增代次	分类轨迹、更新后的 meta
finish	执行同代次门禁, 形成待独立复核候选	CANDIDATE_PASS 摘要
--candidate finish	复用同代次合格门禁并正式结束	PASS/BLOCKED 与 none/compact/durable 档案

表 30: 专业知识载体在编译链中的分工

载体	编码的问题	进入下一节点的方式	单独不能证明什么
instruction/spec	架构、接口、Linux 层级、CPI/PPA 全序等不变量	由分类与路径选择, 并被合同字段、assertion 或 checker 机械化	文件存在不等于规则被执行
Skill	某类任务的可复用操作、参数、模板和反例出口	规定生成、校验、渲染的次序及失败出口	Skill 自测不等于 DUT 正确
machine contract	本轮对象、路径、命令、上下文、成功条件和边界	作为 Agent 节点唯一授权与审查基准	合同完整不等于工具已经运行
profile/E2E	节点 owner、依赖、函数、输入输出和端到端完成条件	递归展开闭包并逐节点执行、留证	图可展开不等于业务门通过
oracle/mutation	如何把 raw observation 转为限定裁决	生成正向结果、唯一拒绝 marker 与 receipt	给定 fault set 被杀死不等于全状态空间完备

表 31: RV64 子任务 instruction 中的关键字段及其工程作用

规则字段	机器约束	被阻止的失效
工程域与任务类型	固定为本地 RV64 RTL，并区分只读审查、实现、验证和 PPA 分析	软件问题或写型任务借用只读权限
可读/可写路径	规范化仓库相对路径；写路径必须属于可读路径，审查任务写集为空	越过模块边界修改、依赖未登记文件
规范命令三元组	每项绑定命令、读写模式和固定用途；禁止自由用途文本扩大权限	临时拼接更容易通过的验证命令
必读材料	列出完成节点必需的 instruction、spec、RTL、TB 和证据	只根据主 Agent 摘要作结论
交付与成功条件	规定具体产物、反例出口、未知项和客观验收谓词	被迫给出 PASS，或把自然语言总结当证据
合同路径与哈希	渲染任务说明时绑定同一 JSON 的路径和 SHA-256	派发提示与已审查合同发生静默漂移

表 32: 同一 RV64 合同能力在 Skill 与 E2E 中的接线

接线位置	实际绑定对象	可检验结果
SKILL.md	能力分档、最小上下文、canonical 调用次序、未知项和范围扩展出口	Agent 对同类任务遵循同一操作规约；不把口头追加当作新权限
合同生成器	instruction、策略 JSON、schema 与 create/validate/render 入口	合同可生成；越权组合被校验器拒绝；提示绑定合同哈希
E2E profile	节点 rtl-task-contract、实现函数、责任主体，以及规则/Skill/生成器输入	该能力能够被任务图发现并调度，而非孤立脚本
E2E 节点函数	生成器审计、自测、命令行自测和合同外路径 mutation	正向用法可运行，目标越权变体会产生非零结果与独立日志
业务验证	合同列出的 RTL testbench、DiffTest、综合、STA 或 PPA gate	决定被测设计是否合格；不由 Skill/E2E 的环境自测代替

表 33: 执行面主要工程后端、原始产物及其证据用途

后端或工具	在本工程中的角色	执行面直接产生的原始产物	证据面可支持的结论与边界
AM 与 RISC-V 交叉工具链	AM (Abstract Machine, 抽象计算机运行时与平台适配层) 把同一测试程序适配到不同后端; 交叉编译器生成 RISC-V 目标码	带哈希的 ELF/二进制镜像、反汇编、链接布局和构建配置	冻结 workload 身份并保证两端消费同一程序; 仅构建成功不证明程序已在任一处理器上正确运行
NEMU	本工程的 RISC-V 软件指令集模拟器, 在 DiffTest 或系统启动中作为功能参考端	退休指令与架构状态序列、异常/设备事件、参考端控制台和终止状态	为 NPC 的指令级状态比较提供参考, 也可建立 Linux 参考路径; NEMU 自身通过不能证明 RTL target 正确
NPC/Verilator	把本地处理器 RTL 编译为可执行仿真目标, 是被验证的 target 而非参考模型	target 退休流、周期计数、总线与设备事务、波形、控制台日志及终止 marker	支持局部 TB、DiffTest 和 Ubuntu 系统事务; 只有与冻结参考、配置及 oracle 同时闭合后才能形成限定 PASS
Yosys/OpenSTA	分别承担 RTL 逻辑综合与静态时序分析, 观察设计实现结构和时序约束	门级网表、单元/位数统计、关键路径、时序裕量及约束覆盖报告	支持结构筛选和代理时序比较; 缺少真实宏、寄生参数、完整 I/O 与工艺角时, PPA 必须保持 UNQUALIFIED
JDK/Chisel 工具链	在需要 ysyxSoC/Chisel 生成路径时, 把 Scala/Chisel 描述展开为可供后续编译的 RTL	生成的 Verilog/SystemVerilog、SoC wrapper 与生成日志	证明生成入口和接口产物可复现; 不替代生成 RTL 的仿真、综合或接口验收

表 34: 环境自验证图的责任分组而非文件逐行解释

方法责任	代表节点	所验证的边界
启动上下文与场景身份	discovery、tool environment、 backend status	规则、工具和当前 target 可发现，但不证明 RTL 正确
知识/运行物边界	三层环境合同（规则、Skill 与运行物分层）、运行时产物边界	active rule、Skill、日志与重型产物没有互相冒充
控制与审查责任	state traceback、 reviewer/inspector、RTL task contract	状态可回溯、实现者不能自证、越界合同被拒绝
证据与交付资格	fail-closed status、delivery readiness、profile index	不完整长跑不能 PASS，能力可打包且确实有调度入口

A.1 正文下沉的故障注入与过程证据

本节保存第 5–7 章中用于复核实现和过程链、但不承担方法定义的完整表格。正文保留真实 RTL 扩展的四面传递顺序，附录集中保存失败关闭、反馈路由、触发规则与代表性工程链。

表 35: 失败关闭提交协议的定向故障注入结果

注入条件	命令 rc	证据完整	清理成功	信号/写入	实测终态
正常到达证据末端	0	是	是	无	PASS
提前正常退出	0	否	是	无	FAIL, 保留当前 stage
主命令失败	7	否	是	无	FAIL rc=7
清理失败	0	是	否	无	FAIL rc=9
PASS 状态写入失败	0	是	是	write rc=73	回退写 FAIL, 记录 status-write rc
HUP 中断	129	否	未授权	HUP	FAIL signal=HUP
INT 中断	130	否	未授权	INT	FAIL signal=INT
TERM 中断	143	否	未授权	TERM	FAIL signal=TERM

表 36: 工程反例、方法增量与后续复用的过程追踪

暴露的失败模式	外层闭环固化的机制	后续复用场景	可观察结果及边界
lint 无法回答镜像能否运行	动态 image/pmem/DPI/trace/exit 图节点	RV32 single 与 SoC 动态执行	从语法推进到给定镜像；不等于完整 ISA
target/reference 与配置易串证	场景 profile、分层实验身份和任务合同	RV64 自测、40/40 DiffTest 与 CoreMark 对拍	能区分自测和对拍；无 prompt-only 对照
完整系统日志被旧 oracle 误判	原始状态/版本化 replay 分离，长跑提交失败关闭	Ubuntu A3 replay、HUP/TERM 与 early-exit 自测	保留 source FAIL 并形成限定重放；未测节省时数
owner 量测曾身份非法、B 延迟敏感性已识别，PPA 时序硬门失败	量测合同、守恒式、受控干预、Selector 与 PPA 资格全序	PERF_BASELINE、owner A/B、敏感性实验与下一动作选择	H1 只获分析授权，不授权 production RTL 或晋级；PPA 保持 UNQUALIFIED

表 37: 同一证据事件对内层与外层闭环的不同触发

证据面观测	当前任务的内层动作	触发外层的附加条件	外层修改对象
单次实现或周期错误	定位首个错误事务、修复并回归	无；当前合同足以处理	不修改环境
多次出现同类接口误判	当前节点返工并保留反例	跨节点重复，且可抽象为稳定不变量	知识面的 instruction/Skill, 必要时控制面合同字段
缺少可执行 oracle	当前任务保持 GAP	同类任务均无法形成机器判定	知识面的 oracle/E2E 与证据面的资格规则
长跑退出码与证据矛盾	停止并保留 stage/signal/cleanup	问题来自公共 runner 终止语义	执行/证据接口的状态协议与故障注入
工具或环境能力缺失	当前节点 BLOCKED 或截断	不是单一主机偶发故障，且存在稳定适配方案	执行适配器、环境发现与对应 E2E
历史上下文污染或预算溢出	缩小当前查询并保持来源	在多次任务中重复引入过期或无关结论	知识面的 bounded recall 与证据索引策略

表 38: 工程压力驱动的方法构件演进及其上下游作用

方法构件	触发压力	新增方法算子	上游/下游位置	扩大的交付能力
图执行与原 子证据	NEMU/Ubuntu 任务已无法由一条命令表达	profile 闭包、逐节点状态和不可半发布的证据包	合同 → 节点图 → 收据	多节点场景可调度、可定位失败并留证
确定环境与 有界召回	非交互环境漂移、历史记录持续增长	工具环境装载、活动索引和受预算约束的历史摘要	环境合同 → 工具；证据索引 → 知识包	新会话可重建必要环境而不吞入全量历史
环境自验证	规则、Skill 与实现入口可能彼此漂移	静态审计、正例、mutation 和发布资格	方法资产 → E2E → 版本化能力	AI 环境自身成为受验证产物

表 38: 工程压力驱动的方法构件演进及其上下游作用（续）

方法构件	触发压力	新增方法算子	上游/下游位置	扩大的交付能力
RTL 任务合同	RV64 跨模块任务需要隔离派发与专业复核	作用域、规范命令、交付和验收的机器合同	架构事实 → Skill → 子 Agent/TB/EDA	多 Agent 仍绑定 module、signal、transaction 与写路径
长跑提交协议	十九小时系统运行不能把进程退出等同于成功	证据完成位、stage、signal、hash 与 cleanup 合取	运行状态 → 资格化 → PASS/FAIL	中断、旧判定器与不完整证据不能遗留绿色状态
轻量生命周期控制	高频开发中全量 profile 成本过高且门禁选择易漂移	任务分类、代次失效和显式路径到固定门禁的路由	分类/路径 → 门禁 → 候选/复核/终态	日常任务运行最小责任门集，同时保留正式复核

表 39: 2176个 ROB 合法域用例中的代表组合与期望

ROB/退休条件	合法域约束	期望退休数	检查目的
count_q=0	lane0、lane1 均不得 fire	0	证明 ROB 空时退休数必为 0
count_q>0, head 被阻塞	两条 lane 均不 fire	0	区分“有条目”和“本拍可退休”
lane0 fire, lane1 阻塞	lane1 不得独立越过 lane0	1	覆盖单退休合法路径
count_q>1, 两 lane 均 fire	lane1 fire 蕴含 lane0 fire	2	覆盖双退休合法路径
lane1 fire、lane0 不 fire	违反提交顺序，排除出合法域	不适用	防止用不可达输入伪造反例

A.2 知识编译与复核合同

代码 28 说明配套生成器如何按照 Skill 约束把字段写入可校验 JSON；代码 29 给出一个只读性能复核节点的权限、输入和允许结论。二者分别对应通用合同构造与一次具体实例，不能替代 DUT 的功能或性能 oracle。

代码 28: 配套生成器按照 Skill 规则构造 RV64 子任务合同的关键字段

```
1 allowed_paths = unique([
2     *args.allow_path, *required_context, contract_relative_path
3 ])
4 allowed_commands = [{
5     "command": command.strip(),
6     "mode": "read-only",
7     "purpose": purpose_catalog[command.strip()][ "purpose" ],
8 } for command in args.allow_read_command]
9
10 contract = {
11     "task_kind": args.task_kind,
12     "scope": {"allowed_paths": allowed_paths,
13              "write_paths": unique(args.write_path or []),
14              "allowed_commands": allowed_commands},
15     "context": {"required_files": required_context},
16     "deliverables": unique(args.deliverable),
17     "success_criteria": unique(args.success_criterion),
18 }
```

代码 29: PERF_BASELINE 独立复核合同的承重字段

```
1 task_kind = read-only-review
2 goal = recompute(endpoint, cycle_slot_conservation,
3                  three_run_bit_exact, stats_off_equivalence,
4                  raw_A1_A2_state, PPA_qualification)
5 allowed_paths = {
6     performance-counter-schema-v4.json,
7     performance-baseline-current.json,
8     versioned raw-run v1,
9     versioned corrected-run v2
10 }
11 write_paths = {}
12 allowed_commands = {
13     (rg, read-only, search-allowed-paths),
14     (sed, read-only, view-selected-lines),
15     (sha256sum, read-only, hash-declared-artifact)
16 }
17 acceptable_outputs = {PASS, GAP, inconclusive}
```

B 工程能力层级、执行端与 Agent 角色变化

表 40: 各能力层级的执行端记录与职责重心

能力层级与 证据对象	可核验执行端	Agent 主要动作	工程师主要责任
局部生成与 动态接通: RV32 single target / reference 与 位宽扩展: SoC、RV64	task-report 多记录 GitHub Copilot; 精确 底层模型未统一冻结 商业模型快照未稳定 保存	局部生成、报错解释、RTL 铺开 跨文件候选、Cache/OoO 修 改、定向调试	定义接口、提供上下文、补 建动态验证 冻结基线、比较候选、决定 回滚
系统级事务: Linux/Ubuntu	进入 Codex/Agent 式 环境; 逐 run model ID 仍不完整	跨模块追踪、Linux bring-up、长链调试	建立 profile、有界召回和证 据结构
架构合同: OoO、 FENCE 与 memory owner	Codex/Agent 式环境; 精确模型未统一保存	身份绑定、mutation、冻结 重放、分层验证	定义 fault set、hard gate 和晋 级边界
多目标资格: CPI/PPA	Codex/Agent 式环境; 精确模型仍不完整	基线重放、CPI 分解、外置 探针	冻结 workload/ROI, 拒绝不 合格量测

能力层级以验证压力和方法接口为主。执行端名称只能证明使用表面，不能反推出底层模型版本。早期缺失 provider、model ID、seed、temperature 和完整 context snapshot，因此本文不补造“当时是什么精确模型”。可以确认的是 Agent 的角色随环境成熟而变化：从候选生成器，逐步成为在工程合同内执行、验证、回滚和留证的节点。

使用规模背景。累计使用 227.5 亿 Token，单日峰值 14.6 亿，最长单次聊天为 24 小时 11 分，连续使用 28 天。这些指标只说明交互规模与持续性，并解释为什么上下文边界、有界召回、证据压缩和任务恢复成为实际工程问题；Token 数不等于代码量、正确率、生产率或方法效果。

C 机器可读主张—证据附录

本附录不把 30 条长记录重复排入 PDF，而是固定机器可读对象及其 schema。权威数据文件为 docs/research-paper/claim-evidence-map.tsv；C27–C29 与正文

性能结果绑定同一冻结的 canonical 证据 cohort, C30 则有意保留前一轮不合格 owner 量测作为失败关闭反例。该文件与同一仓库版本共同确定证据身份, 校验工具在构建时重算摘要; schema/path 校验与证据新鲜度复核是两个独立步骤, 前者通过不能替代后者。正文不固定会随修订漂移的“提交—哈希”组合。首行 schema 按原始顺序为:

代码 30: 主张—证据表的 22 列 schema

```
claim_id, claim_text, research_axis, evidence_status, date,
task_episode, run_id, profile, source_path, raw_evidence_path,
git_commit, design_id, configuration, command, exit_code,
metric, negative_evidence, rollback_or_rejection, claim_boundary,
reproducibility, paper_section, notes
```

这些列承担四类不同责任: claim_id 至 evidence_status 固定主张与编码; date 至 configuration 绑定 episode 和实验身份; command 至 rollback_or_rejection 保存执行、观测与反例; claim_boundary 至 notes 保存外推边界和复现缺口。缺失的 design_id 或 git_commit 在 TSV 中以连字符显式表示, 不由论文补造。

表 41: 冻结主张表的状态分布与解释边界

状态	数量	在本研究中的含义
SUPPORTED	18	对象、来源和边界支持当前限定措辞; 不是方法成功次数
PARTIALLY_SUPPORTED	4	机制或结果存在, 但强度、分母、身份或归因仍不完整
OBSERVATIONAL_ONLY	1	只支持共现或纵向变化, 不支持因果解释
CONTRADICTED	6	原始证据直接否定所编码的强命题; 其中包含主动审计的反命题
MISSING	1	当前公开材料不足以裁决, 不以摘要或路径存在替代证据
合计	30	目的性审计集, 不是随机样本或运行成功率分母

选择与去重规则由 docs/research-paper/case-selection-protocol.md 固定。独立反例审查样本为 C01、C02、C03、C04、C07、C10、C11、C14、C15、C22、C23、C30; 复核结果位于 docs/research-paper/independent-claim-review.tsv。初审阻断 C01、C07、C30, 分别暴露早期阶段被过度简化、同一发布内身份元组冲突和原始路径覆盖不足; 最终裁定保留了措辞收紧、状态降级和证据补齐的轨迹。该复核用于发现转录与边界错误, 不构成人类编码一致性或方法效果证据。

早期候选全集及逐条排除日志没有完整保留，这是本附录仍不能补偿的选择偏差。因而 TSV 允许读者复算状态分布、追到现有来源并检查边界，但不能复原所有未入选候选，也不能计算严格 FCR、VSR 或 UPR。

D 复核入口与复现边界

本文只复核已有工程证据，不复跑生产 RTL、Linux、综合或 STA。工程日志、配置与 design-id 的可重放不等于 Agent 采样轨迹可复现；PPA 代理缺少物理条件时统一保持 UNQUALIFIED。公开材料见 <https://github.com/mcsaver/oscpu>。

表 42: 文章中主要方法与工程证据的复核入口

对象	工作区入口
Workflow 导航与三层 policy	AI_ENVIRONMENT.md、 .github/ai-env/contracts/agent-env-policy.json、.github/instructions/agent-env-layer-contract.instructions.md
instruction/Skill 执行规则	.github/instructions/rtl-generation-workflow.instructions.md、.github/instructions/rv64-ppa-optimization-workflow.instructions.md、 .github/skills/agent-env-maintenance/SKILL.md
轻量任务与代次状态 profile 十一节点闭包与发布实现	scripts/agent-flow.c、scripts/agent-flow.sh .github/e2e/profiles/agent-system.tsv、 .github/e2e/profiles/discovery.tsv、scripts/agent-e2e.sh、 scripts/e2e/lib/report.sh
agent-system 实际完成机制	.github/e2e/profiles/agent-system.tsv、 scripts/e2e/lib/report.sh、 docs/research-paper/evidence-ledger.md
bounded recall 与 evidence API	scripts/dev_memory/api.py、scripts/github_index_db.py、 .github/ai-env/contracts/agent-env-schema-contract.json
RTL 子任务 Skill 与生成器	.github/skills/prepare-rtl-task-contract/SKILL.md、 .github/skills/prepare-rtl-task-contract/scripts/rtl_task_contract.py
PERF_BASELINE 与分层设计身份	npc/rv64/eval/ppa/evidence/performance-baseline-current.json、npc/rv64/eval/ppa/evidence/layered-system-signoff-current.json
owner-timing、敏感性与 Selector	.github/task-runs/2026-08-07-rv64-v15q-owner-timing-current-337-a2/evidence/owner-timing-workload-ab/result.json、 .github/task-runs/2026-08-07-rv64-v15q-owner-b-latency-sensitivity-current-337-a1/evidence/owner-b-latency-sensitivity/result.json、 npc/rv64/eval/ppa/evidence/optimization-slice-current.json
长跑失败关闭	scripts/task-run-status.sh、 scripts/tests/test-task-run-status.sh
里程碑与证据台账	docs/research-paper/evidence-ledger.md
单模块三快照	docs/research-paper/module-evolution-evidence.md
NPC A3 十九小时里程碑	docs/research-paper/ubuntu-19h-milestone-evidence.md
30 条主张映射	docs/research-paper/claim-evidence-map.tsv
构建复现	docs/research-paper/paper-reproduction.md